

TUGAS AKHIR

PERANCANGAN APLIKASI SISTEM INFORMASI HIMAPL
BERBASIS *MOBILE* PADA FAKULTAS KOMPUTER
UNIVERSITAS UNIVERSAL



Diajukan sebagai salah satu syarat untuk menyelesaikan
Pendidikan program sarjana

Disusun oleh:

Alex Ferguson

2022133011

Pembimbing:

Ilwan Syafrinal, S.Kom, M.Kom.

PROGRAM STUDI TEKNIK PERANGKAT LUNAK
FAKULTAS KOMPUTER
UNIVERSITAS UNIVERSAL
TAHUN 2025/2026

HALAMAN PERSETUJUAN TUGAS AKHIR

Nama : Alex Ferguson
NIM : 2022133011
Program Studi : S1 TEKNIK PERANGKAT LUNAK
Judul Tugas Akhir : PERANCANGAN APLIKASI SISTEM INFORMASI
HIMAPL BERBASIS *MOBILE* PADA FAKULTAS
KOMPUTER UNIVERSITAS UNIVERSAL

Telah disetujui untuk dipertanggung jawabkan di depan dewan penguji pada Sidang Tugas Akhir pada Program Strata Satu (S1) Sarjana Komputer Program Studi Teknik Perangkat Lunak Universitas Universal.

Batam, 2026
Pembimbing 1

Ilwan Syafrinal, S.Kom, M.Kom.
0427019401

Mengetahui:
Kordinator Program Studi

Kaharuddin, S.Kom., M.Kom.
1009059501

HALAMAN PENGESAHAN TUGAS AKHIR

**PERANCANGAN APLIKASI SISTEM INFORMASI HIMAPL
BERBASIS *MOBILE* PADA FAKULTAS KOMPUTER
UNIVERSITAS UNIVERSAL**

Disusun oleh:

Alex Ferguson

2022133011

Pembimbing 1

Ilwan Syafrinal, S.Kom, M.Kom.

Tanggal: 26 Juni 2026

Batam, 2026

Program Studi Teknik Perangkat Lunak

Universitas Universal

Koodinator Program Studi

Kaharuddin, S.Kom., M.Kom.

1009059501

SURAT PERNYATAAN KEASLIAN TULISAN

Saya yang bertanda tangan di bawah ini:

Nama : Alex Ferguson

NIM : 2022133011

Program Studi : Teknik Perangkat Lunak

Judul TA : PERANCANGAN APLIKASI SISTEM INFORMASI
HIMAPL BERBASIS *MOBILE* PADA FAKULTAS
KOMPUTER UNIVERSITAS UNIVERSAL

Menyatakan dengan sebenar-benarnya bahwa tugas akhir yang saya tulis ini adalah benar-benar karya saya sendiri, bukan hasil jiplakan (plagiat), belum pernah diterbitkan atau dipublikasikan dimanapun atau dalam bentuk apapun, serta belum pernah diajukan untuk memperoleh gelar kesarjanaan di suatu perguruan tinggi.

Atas pernyataan ini, saya siap menerima sanksi apabila di kemudian hari ditemukan pelanggaran terhadap tugas akhir saya ini.

Demikian surat pernyataan ini saya buat dengan sesungguhnya.

Batam, 2026

Yang Membuat Pernyataan

Alex Ferguson

2022133011

ABSTRAK

Sistem Informasi memiliki peran penting dalam mendukung kelancaran aktivitas organisasi, termasuk Himpunan Mahasiswa Teknik Perangkat Lunak (HIMAPL) di Fakultas Komputer Universitas Universal. Saat ini, penyampaian informasi dalam organisasi masih dilakukan melalui grup WhatsApp, yang sering menyebabkan pesan penting tertimbun dan mengakibatkan penyebaran informasi yang tidak merata. Selain itu, pendokumentasian kegiatan masih sulit ditelusuri karena tidak tersip dengan baik, serta proses pendataan anggota dilakukan secara manual sehingga kurang efisien. Berdasarkan permasalahan tersebut, penelitian ini bertujuan untuk merancang aplikasi Sistem Informasi HIMAPL berbasis *mobile* yang berfungsi untuk mempermudah pengurus dalam mengelola kegiatan, memfasilitasi distribusi informasi secara lebih terstruktur, dan melakukan digitalisasi data organisasi. Penelitian ini menggunakan metode penelitian kualitatif untuk memperoleh data kebutuhan pengguna secara lebih detail dan akurat melalui observasi serta wawancara. Proses perancangan dan pembangunan aplikasi dilakukan menggunakan metode Agile Sprint, yang memungkinkan pengembangan fitur secara bertahap dan adaptif. Aplikasi ini dirancang menggunakan *Framework* Ionic sebagai teknologi utama pada sisi frontend, sedangkan teknologi backend akan ditentukan pada tahap pengembangan. Fitur yang dirancang meliputi login dan *role* pengguna, pengumuman, notifikasi, manajemen kegiatan, data anggota dan struktur organisasi, riwayat kegiatan beserta dokumentasi, pembagian tugas, profil pengguna, serta agenda kegiatan. Penelitian ini diharapkan menghasilkan aplikasi *mobile* yang dapat membantu HIMAPL dalam mengelola data dan kegiatan secara digital, mempercepat penyebaran informasi, serta meningkatkan efisiensi administrasi organisasi. Aplikasi juga diharapkan mampu menjadi solusi terintegrasi yang dapat digunakan sebagai sarana komunikasi dan dokumentasi resmi organisasi.

Kata Kunci: Sistem Informasi, Aplikasi *Mobile*, *Agile Sprint*, *Framework Ionic*, Organisasi Mahasiswa.

ABSTRACT

Information systems play a crucial role in supporting the smooth running of organizational activities, including the Software Engineering Student Association (HIMAPL) at the Faculty of Computer Science at Universal University. Currently, information within the organization is still disseminated through WhatsApp groups, which often results in important messages being buried and resulting in uneven distribution. Furthermore, documenting activities is difficult to trace due to poor archiving, and the manual member data collection process is inefficient. Based on these challenges, this study aims to design a mobile-based HIMAPL Information System application that will simplify activity management, facilitate more structured information distribution, and digitize organizational data. This study uses qualitative research methods to obtain more detailed and accurate user needs through observation and interviews. The application design and development process uses the Agile Sprint method, which allows for gradual and adaptive feature development. This application is designed using the Ionic Framework as the primary technology for the frontend, while the backend technology will be determined during the development phase. The designed features include user login and roles, announcements, notifications, activity management, member data and organizational structure, activity history and documentation, task allocation, user profiles, and activity agendas. This research is expected to produce a mobile application that can assist HIMAPL in digitally managing data and activities, accelerating information dissemination, and improving organizational administrative efficiency. The application is also expected to be an integrated solution that can be used as a means of communication and official organizational documentation.

Keywords: *Information System, Mobile Application, Agile Sprint, Ionic Framework, Student Organization.*

KATA PENGANTAR

Segala puji dan rasa syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa. Atas segala nikmat, karunia dan kasih sayang-Nya yang tidak terhingga, karena atas berkat rahmat-Nya penulis diberikan kemudahan dalam menyelesaikan tugas akhir ini.

Adapun penulisan tugas akhir ini merupakan salah satu syarat untuk menyelesaikan jenjang Sarjana Strata 1 Teknik Perangkat Lunak pada Universitas Universal Batam. Pada kesempatan ini, penulis ingin mengucapkan terimakasih kepada pihak-pihak yang telah memberikan bantuan, dukungan, bimbingan, saran dan dorongan baik secara moril maupun materil dari awal sampai akhir penyusunan tugas akhir ini kepada:

1. Bapak Dr. techn. Aswandy, M.T., Selaku Rektor Universitas Universal
2. Bapak Suryo Widianoro, S.T., M.MSI., M.Com. (IS), Selaku Dekan Fakultas Komputer Universitas Universal
3. Bapak Kaharuddin, S.Kom., M.Kom., Koordinator Program Studi Teknik Perangkat Lunak Universitas Universal.
4. Bapak Ilwan Syafrinal, S.Kom, M.Kom., selaku Pembimbing tugas akhir
5. Bapak/Ibu Dosen Program Studi Teknik Perangkat Lunak Universitas Universal.
6. Kedua orang tua dan keluarga saya
7. Rekan-rekan mahasiswa Program Studi Teknik Perangkat Lunak Universitas Universal, khususnya angkatan 2022.

Penulis menyadari bahwa laporan ini masih sangat jauh dari kata sempurna, besar harapan penulis semoga tugas akhir ini dapat bermanfaat bagi semua pihak yang membutuhkan.

Batam, 26 Juni 2026

Alex Ferguson

DAFTAR ISI

HALAMAN PERSETUJUAN TUGAS AKHIR.....	ii
HALAMAN PENGESAHAN TUGAS AKHIR.....	iii
SURAT PERNYATAAN KEASLIAN TULISAN	iv
ABSTRAK	v
<i>ABSTRACT</i>	vi
KATA PENGANTAR	vii
DAFTAR ISI.....	viii
DAFTAR GAMBAR	xii
DAFTAR TABEL	xvi
BAB I PENDAHULUAN	17
1.1 Latar Belakang Penelitian	17
1.2 Identifikasi Permasalahan	19
1.3 Rumusan Masalah	20
1.4 Ruang Lingkup Penelitian.....	20
1.5 Tujuan Penelitian	21
1.6 Manfaat Penelitian	21
1.6.1. Manfaat Teoritis	21
1.6.2. Manfaat Praktis	22

BAB II TINJAUAN PUSTAKA.....	24
2.1 Penelitian Terdahulu	24
2.2 Landasan Teori.....	29
2.2.1 Sistem Informasi	29
2.2.2 <i>Mobile</i>	32
2.2.3 <i>Android</i>	35
2.2.4 <i>Ionic Framework</i>	38
2.2.5 <i>Vue.js</i>	40
2.2.6 <i>Supabase</i>	42
2.2.7 <i>Agile</i>	45
2.2.8 <i>Sprint</i>	47
2.2.9 <i>Java Script</i>	50
2.2.10 Basis Data	53
2.2.11 Teori Pengolahan Kegiatan.....	57
2.2.12 Teori Usability dan <i>User Experience</i>	61
2.2.13 <i>Unified Modeling Language (UML)</i>	64
2.2.14 <i>PostgreSQL</i>	72
2.2.15 <i>Black Box Testing</i>	74
2.2.16 <i>User Acceptance Testing (UAT)</i>	76
BAB III METODE PENELITIAN.....	80

3.1	Gambaran Umum Objek	80
3.2	Metode Penelitian.....	81
3.2.1	<i>Product Backlog</i>	82
3.2.2	<i>Sprint Backlog</i>	83
3.2.3	<i>The Sprint</i>	83
3.2.4	<i>Completed Product</i>	84
3.3	Pengumpulan <i>Data</i>	84
3.4	Pengolahan <i>Data</i>	85
3.5	Jadwal Penelitian.....	90
BAB IV HASIL DAN PEMBAHASAN		92
4.1	Analisi Kebutuhan Sistem.....	92
4.1.1	Analisis Kebutuhan Fungsional	93
4.1.2	Analisis Kebutuhan Non-Fungsional	95
4.2	<i>Product Backlog</i>	96
4.3	<i>Sprint Backlog</i>	102
4.3.1	Ruang lingkup <i>Sprint 1</i>	103
4.3.2	Ruang lingkup <i>Sprint 2</i>	105
4.4	Analisis dan Perancangan Sistem.....	107
4.4.1	<i>Use Case Diagram</i>	108
4.4.2	<i>Class Diagram</i>	109

4.5	<i>The Sprint</i>	111
4.5.1	<i>Sprint 1</i>	111
4.5.2	<i>Sprint 2</i>	144
4.6	<i>Completed Product</i>	194
4.7	Pengujian Pada Sistem	195
4.7.1	Skenario dan Hasil Pengujian <i>Black-Box</i>	196
4.7.2	Hasil <i>User Acceptance Testing</i>	197
BAB V PENUTUP.....		199
5.1	Kesimpulan	199
5.2	Saran.....	200
DAFTAR PUSTAKA		203
KONSULTASI TUGAS AKHIR.....		209
DAFTAR RIWAYAT HIDUP.....		210

DAFTAR GAMBAR

Gambar 3.1 Agile Sprint	82
Gambar 4.1 <i>Use Case Diagram</i>	108
Gambar 4.2 <i>Class Diagram</i>	110
Gambar 4.3 <i>Activity Diagram Login</i>	114
Gambar 4.4 <i>Activity Diagram Register</i>	115
Gambar 4.5 <i>Activity Diagram</i> Membuat Divisi	117
Gambar 4.6 <i>Activity Diagram</i> Kelola Profil	118
Gambar 4.7 <i>Sequence Diagram Login</i>	120
Gambar 4.8 <i>Sequence Diagram Register</i>	121
Gambar 4.9 <i>Sequence Diagram</i> Membuat Divisi	123
Gambar 4.10 <i>Sequence Diagram</i> Kelola Profil.....	125
Gambar 4.11 Desain Halaman <i>Login</i>	127
Gambar 4.12 Desain Halaman Profil	128
Gambar 4.13 <i>Database</i> Tabel <i>Users</i>	129
Gambar 4.14 <i>storage bucket avatars</i>	130
Gambar 4.15 Autentikasi Akun <i>Google</i>	130
Gambar 4.16 <i>Database</i> Tabel Divisi.....	131
Gambar 4.17 Halaman <i>Login</i>	132
Gambar 4.18 Halaman <i>Register</i>	133
Gambar 4.19 Halaman Profil	134
Gambar 4.20 Halaman Kelola Profil.....	135
Gambar 4.21 Halaman Kelola Divisi	137

Gambar 4.22 Halaman Peran	138
Gambar 4.23 Halaman Kelola Peran.....	139
Gambar 4.24 <i>Activity Diagram</i> Kelola Event	147
Gambar 4.25 <i>Activity Diagram</i> Delegasi Tugas	149
Gambar 4.26 <i>Activity Diagram</i> Kelola News	150
Gambar 4.27 <i>Activity Diagram</i> Dokumentasi Event.....	151
Gambar 4.28 <i>Activity Diagram</i> Kelola Notifikasi.....	153
Gambar 4.29 <i>Sequence Diagram</i> Kelola Event.....	154
Gambar 4.30 <i>Sequence Diagram</i> Delegasi Tugas.....	155
Gambar 4.31 <i>Sequence Diagram</i> Kelola News	156
Gambar 4.32 <i>Sequence Diagram</i> Dokumentasi Event.....	157
Gambar 4.33 <i>Sequence Diagram</i> Kelola notifikasi.....	158
Gambar 4.34 Desain halaman beranda.....	159
Gambar 4.35 Desain halaman <i>event sub-tab1</i>	160
Gambar 4.36 Desain halaman <i>event sub-tab2</i>	160
Gambar 4.37 Desain halaman <i>event sub-tab2</i>	161
Gambar 4.38 Desain <i>template</i> halaman detail.....	162
Gambar 4.39 Desain halaman notifikasi	163
Gambar 4.40 <i>events database</i>	164
Gambar 4.41 <i>kategori_event database</i>	164
Gambar 4.42 <i>lokasi database</i>	165
Gambar 4.43 <i>event_participants database</i>	165
Gambar 4.44 <i>event_kategori_mapping database</i>	166

Gambar 4.45 <i>event_links database</i>	167
Gambar 4.46 <i>tugas database</i>	167
Gambar 4.47 <i>tugas_assignments database</i>	168
Gambar 4.48 <i>dokumentasi database</i>	169
Gambar 4.49 <i>storage bucket dokumentasi-event</i>	170
Gambar 4.50 <i>news database</i>	171
Gambar 4.51 <i>notifikasi database</i>	171
Gambar 4.52 <i>penerima_notifikasi database</i>	172
Gambar 4.53 Halaman Beranda	173
Gambar 4.54 <i>Side Menu</i>	174
Gambar 4.55 Halaman <i>News Detail</i>	175
Gambar 4.56 Halaman <i>Events sub-tab Available</i>	176
Gambar 4.57 Halaman <i>Event sub-tab Upcoming</i>	177
Gambar 4.58 Halaman <i>Events sub-tab Completed</i>	178
Gambar 4.59 Halaman <i>Event Detail</i>	179
Gambar 4.60 Halaman Notifikasi	180
Gambar 4.61 Halaman Detail Notifikasi	181
Gambar 4.62 Halaman Kelola <i>Events</i>	182
Gambar 4.63 Halaman <i>Create Event</i>	183
Gambar 4.64 Halaman Kelola <i>News</i>	184
Gambar 4.65 Halaman <i>Create News</i>	185
Gambar 4.66 Halaman Kelola Notifikasi	186
Gambar 4.67 Halaman <i>Create Notification</i>	187

Gambar 4.68 Halaman Laporan	188
Gambar 4.69 Ikon aplikasi di <i>Android</i>	195

DAFTAR TABEL

Tabel 2.1 Penelitian Terdahulu	24
Tabel 2.2 Class Diagram	65
Tabel 2.3 Tabel Use Case Diagram.....	67
Tabel 2.4 Tabel <i>Activity Diagram</i>	68
Tabel 3.1 Jadwal Penelitian.....	90
Tabel 4.1 Tabel Kebutuhan Fungsional	93
Tabel 4.2 Tabel Kebutuhan Non-Fungsional	95
Tabel 4.3 Tabel <i>Product Backlog</i>	97
Tabel 4.4 Tabel <i>Sprint Backlog</i> 1.....	103
Tabel 4.5 Tabel <i>Sprint Backlog</i> 2.....	105
Tabel 4.6 Tabel <i>Planning</i> Sprint 1	112
Tabel 4.7 Tabel <i>Review</i> Sprint 1	140
Tabel 4.8 Tabel <i>Retrospective</i> Sprint 1.....	143
Tabel 4.9 Tabel <i>Planning</i> Sprint 2	145
Tabel 4.10 Tabel <i>Review</i> Sprint 2	189
Tabel 4.11 Tabel <i>Retrospective</i> Sprint 2	193
Tabel 4.12 Tabel Hasil Pengujian <i>Black-Box</i>	196

BAB I

PENDAHULUAN

1.1 Latar Belakang Penelitian

Perkembangan teknologi informasi (TI) dalam pengelolaan organisasi kemahasiswaan saat ini menunjukkan dampak signifikan terhadap efektivitas dan efisiensi operasional organisasi. Organisasi mahasiswa, termasuk Himpunan Mahasiswa Teknik Perangkat Lunak (HIMAPL) di Fakultas Komputer Universitas Universal, dituntut untuk memanfaatkan TI dalam memperbaiki proses penyampaian informasi, pengelolaan administrasi, serta dokumentasi kegiatan. Penggunaan aplikasi *mobile* dalam konteks ini menjadi semakin relevan, karena mahasiswa sebagai pengguna utama terbukti lebih responsif terhadap *platform* aplikasi dibandingkan media komunikasi tradisional seperti *WhatsApp* (Audy Rivand & Suwandi, 2023). Penelitian internasional juga menunjukkan bahwa aplikasi *mobile* mampu meningkatkan komunikasi internal dan pengelolaan data melalui integrasi fitur yang lebih terstruktur (Johannsen et al., 2023).

Berdasarkan wawancara kepada Kaharuddin, S.Kom., M.Kom. selaku dosen pembina organisasi, Vannesia Calista selaku ketua HIMAPL yang menjabat pada tahun 2024 hingga 2025, dan Yuriko Asinselo selaku mantan ketua HIMAPL yang menjabat pada tahun 2023 hingga 2024. Dapat diketahui bahwa kondisi saat ini, HIMAPL masih bergantung pada *WhatsApp* sebagai media komunikasi utama. Meskipun mudah digunakan, *platform* ini tidak dirancang sebagai sistem manajemen informasi yang terstruktur, sehingga pesan penting sering tertimbun oleh percakapan lain. Hal ini berpotensi menyebabkan keterlambatan dan

ketidakmerataan penyebaran informasi kepada anggota. Selain itu, dokumentasi kegiatan yang dilakukan menggunakan *Google Drive* tidak terorganisasi dengan baik, menyebabkan kesulitan dalam mencari riwayat kegiatan saat dibutuhkan untuk arsip atau pelaporan. Pendataan anggota HIMAPL yang masih menggunakan *Excel* juga menyebabkan data sulit diperbarui, rawan kesalahan, serta tidak efisien dalam pengelolaannya. Beberapa penelitian menunjukkan bahwa sistem informasi terintegrasi mampu mengatasi permasalahan tersebut dengan menyediakan fitur manajemen data anggota, pengumuman, notifikasi, serta dokumentasi kegiatan yang lebih sistematis (Sopiyan & Darajatun, 2024).

Untuk menjawab permasalahan tersebut, pengembangan aplikasi Sistem Informasi HIMAPL berbasis *mobile* menjadi solusi yang tepat. *Framework Ionic* dipilih sebagai teknologi pengembangan karena mendukung pembuatan aplikasi lintas *platform* dengan antarmuka modern dan proses pengembangan yang efisien (Azis et al., 2024). Adopsi aplikasi *mobile* juga sejalan dengan tren digitalisasi organisasi dan dapat meningkatkan aksesibilitas data oleh seluruh anggota HIMAPL (Tulvina et al., 2022). Pada sisi metodologi pengembangan, penggunaan metode *Agile* dengan pendekatan *Sprint* dianggap relevan karena memberikan fleksibilitas dalam pengembangan fitur secara bertahap sesuai kebutuhan pengguna (Sirojuddin et al., 2022).

Metode penelitian yang digunakan adalah metode kualitatif, yang memungkinkan pengumpulan data kebutuhan pengguna secara mendalam melalui observasi dan wawancara. Pendekatan kualitatif membantu memastikan fitur

aplikasi yang dikembangkan sesuai dengan kebutuhan nyata pengurus dan anggota HIMAPL (Romi, 2024). Sistem informasi ini nantinya diharapkan dapat mendukung pengelolaan kegiatan, pembagian tugas, dokumentasi, serta penyampaian informasi secara terpusat dan terorganisasi. Sistem yang terintegrasi juga terbukti dapat meningkatkan partisipasi anggota dan mengurangi ketidakteraturan dalam pelaksanaan kegiatan organisasi (Hasan et al., 2025).

1.2 Identifikasi Permasalahan

Berdasarkan latar belakang yang telah dijelaskan, beberapa permasalahan yang teridentifikasi dalam pengelolaan informasi dan administrasi pada HIMAPL Fakultas Komputer Universitas Universal adalah sebagai berikut:

1. Penyebaran informasi organisasi belum terpusat, sehingga informasi yang dikirim melalui *WhatsApp* sering tertimbun dan tidak tersampaikan secara merata kepada seluruh anggota.
2. Dokumentasi kegiatan tidak terorganisasi dengan baik, karena file dokumentasi masih tersimpan secara tidak terstruktur di *Google Drive*, sehingga riwayat kegiatan sulit ditemukan kembali ketika dibutuhkan untuk arsip atau pelaporan.
3. Pendataan anggota masih dilakukan secara manual menggunakan *Excel*, sehingga pembaruan data menjadi tidak efisien, rawan kesalahan, dan menyulitkan proses pengelolaan data anggota.
4. Belum adanya sistem yang mengintegrasikan informasi pengumuman, kegiatan, struktur organisasi, dan pembagian tugas dalam satu *platform* terpusat.

1.3 Rumusan Masalah

Berdasarkan identifikasi permasalahan tersebut, maka rumusan masalah dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana merancang aplikasi sistem informasi HIMAPL berbasis *mobile* yang dapat memusatkan penyebaran informasi dan meningkatkan efisiensi komunikasi organisasi?
2. Bagaimana merancang fitur pengelolaan kegiatan dan dokumentasi agar riwayat kegiatan dapat terdokumentasi dengan baik dan mudah diakses kembali?
3. Bagaimana membangun sistem pendataan anggota yang terstruktur dan terintegrasi, sehingga proses pengelolaan data lebih efisien dan akurat?

1.4 Ruang Lingkup Penelitian

Berikut merupakan ruang lingkup penelitian:

1. Aplikasi dibangun berbasis *mobile* menggunakan *Framework Ionic* dan *Vue.js*, dengan *backend Supabase (PostgreSQL)*.
2. Fitur yang dikembangkan mencakup *login*, manajemen *role*, pengumuman, notifikasi, pendataan anggota, struktur organisasi, manajemen kegiatan, dokumentasi, agenda, dan pembagian tugas.
3. Pengembangan perangkat lunak menggunakan metode *Agile* dengan pendekatan *Sprint*, yang mencakup tahapan *planning*, *implementation*, *daily scrum*, *review*, dan *retrospective* di setiap iterasi *Sprint*.

4. Penelitian ini berfokus pada organisasi HIMAPL Fakultas Komputer Universitas Universal, dengan cakupan data dan pengguna terbatas pada anggota HIMAPL, Mahasiswa Fakultas Komputer, dan dosen pembina.

1.5 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Merancang dan mengembangkan aplikasi sistem informasi HIMAPL berbasis *mobile* yang menjadi pusat penyebaran informasi organisasi secara lebih terstruktur dan merata.
2. Menyediakan fitur pengelolaan kegiatan dan dokumentasi yang memungkinkan pengurus menyimpan, mengakses, dan menelusuri riwayat kegiatan dengan lebih mudah dan cepat.
3. Membangun sistem pendataan anggota yang terintegrasi dengan data dan sistem, sehingga proses pencatatan, pembaruan, dan pengelolaan data dapat dilakukan secara efisien dan minim kesalahan.

1.6 Manfaat Penelitian

Manfaat yang didapat diperoleh dari penelitian ini sebagai berikut:

1.6.1. Manfaat Teoritis

Secara teoritis, penelitian ini diharapkan dapat memberikan kontribusi pada pengembangan ilmu di bidang Sistem Informasi, khususnya terkait perancangan aplikasi organisasi mahasiswa berbasis *mobile*. Penelitian ini juga dapat menjadi referensi dalam penerapan metode pengembangan *Agile Sprint* pada proyek aplikasi *mobile* serta memberikan gambaran mengenai pemanfaatan

Framework Ionic dalam pembuatan aplikasi lintas *platform*. Dengan demikian, penelitian ini dapat menambah literatur terkait digitalisasi proses komunikasi dan administrasi organisasi kemahasiswaan.

1.6.2. Manfaat Praktis

Manfaat yang bisa didapatkan dalam penelitian ini sebagai berikut:

a) Bagi Pengguna

Secara praktis, penelitian ini memberikan manfaat langsung bagi HIMAPL sebagai pengguna sistem. Aplikasi yang dikembangkan dapat membantu pengurus dalam mengelola informasi secara lebih terstruktur dan mengurangi ketergantungan pada *WhatsApp*, di mana pesan penting sering kali tertimbun oleh percakapan lainnya. Selain itu, anggota HIMAPL dapat memperoleh informasi secara lebih cepat dan terpusat melalui aplikasi. Digitalisasi data seperti pendataan anggota, dokumentasi kegiatan, agenda, serta pembagian tugas juga mempermudah pengelolaan administrasi dan meningkatkan koordinasi internal organisasi.

b) Bagi Penulis

Bagi penulis, penelitian ini memberikan pengalaman langsung dalam merancang dan membangun aplikasi *mobile* berbasis *Ionic* sehingga dapat meningkatkan kemampuan dalam pengembangan perangkat lunak modern. Penulis juga memperoleh pemahaman lebih mendalam mengenai penerapan metode *Agile Sprint* dalam proyek nyata, khususnya dalam proses iteratif

perencanaan, pengembangan, dan evaluasi. Selain itu, penelitian ini membantu penulis memahami alur kerja organisasi kemahasiswaan, seperti bagaimana informasi dikelola, kegiatan direncanakan, serta data anggota diproses, sehingga penulis dapat mengasah kemampuan analisis kebutuhan dan pemecahan masalah secara nyata.

c) Bagi Peneliti Lanjutan

Penelitian ini bermanfaat bagi peneliti lanjutan sebagai referensi atau dasar untuk pengembangan sistem yang lebih komprehensif. Hasil penelitian dapat dijadikan acuan untuk memperluas fitur aplikasi, menambah integrasi dengan sistem lain, atau mengevaluasi metode pengembangan yang digunakan. Penelitian lanjutan juga dapat mengembangkan aspek-aspek yang belum terakomodasi, seperti analitik kegiatan organisasi, otomatisasi kehadiran, atau peningkatan keamanan sistem, sehingga penelitian ini dapat menjadi pijakan awal untuk pengembangan berikutnya.

BAB II TINJAUAN PUSTAKA

2.1 Penelitian Terdahulu

Tabel 2.1 Penelitian Terdahulu

No.	Judul Penelitian	Nama Peneliti	Tahun Publikasi	Hasil Penelitian
1	<i>Adopsi Cloud Enterprise Resource Planning dengan Pendekatan Technology, Organization, and Environmental pada UMKM</i>	Annisa Nabilah Hasan, Nadhilah Amaliah Liwan, Falih Zaki Sudharma, Grace T. Pontoh, Aini Indrijawati	2025	Penelitian ini menemukan bahwa faktor teknologi adalah faktor paling berpengaruh dalam adopsi ERP <i>Cloud</i> pada UMKM. Tantangan utama adalah biaya terkait investasi awal dan operasional.
2	Analisis Desain Sistem Pendataan Fasilitas Kampus di Program Studi Universitas	Syirfa, Dzune, Salsabila, Prasetya, Zae, Wildani	2023	Sistem pendataan fasilitas kampus yang dirancang menunjukkan bahwa penggunaan aplikasi akan meningkatkan efisiensi pengelolaan fasilitas (Naiylatan Syirfa et al., 2023).
3	Analisis Situasional dan Perancangan Sistem Informasi	Asep Sopiyan, Rizki Achmad Darajatun	2024	Sistem informasi keuangan dirancang untuk membantu perusahaan dalam

	Keuangan pada PT ZMI			mencatat dan mengelola data secara lebih efisien, mengurangi kesalahan dan meningkatkan akurasi.
4	Aplikasi Keuangan Koperasi Simpan Pinjam "Permata Ngijo" Berbasis Teknologi Informasi	Yunia Mulyani Azis, Sussy Susanti, Moehammad Sarosa	2023	Pengembangan aplikasi berbasis informasi telah berhasil meningkatkan efisiensi pengelolaan koperasi dengan fitur transparansi dan kemudahan akses.
5	Dampak Efektivitas Sistem Informasi Akuntansi Pengaruh Teknologi Informasi dan Kualitas Sumber Daya Manusia Terhadap Kinerja Perusahaan	Ilham Audy Rivand, Suwandi	2023	Penelitian ini menunjukkan bahwa kualitas sumber daya manusia mempengaruhi efektivitas sistem informasi akuntansi, namun teknologi informasi tidak berpengaruh langsung terhadap kinerja perusahaan.
6	Model <i>Good Governance</i> Berbasis <i>E-government</i> dan Motivasi Pelayanan Publik	Mochamad Vrans Romi	2023	<i>E-government</i> dan motivasi pelayanan publik berpengaruh signifikan terhadap peningkatan <i>Good Governance</i> pada

	Pada Aparatur Sipil Negara Pemerintah Kota Cimahi			Pemerintah Kota Cimahi.
7	Pelaksanaan Sistem Informasi Manajemen Kepegawaian Berbasis IT di Kantor Kementrian Agama Kabupaten Aceh Timur	Rimayanda, Jauharil Maknuni	2022	Sistem informasi manajemen kepegawaian (SIMPEG) mempermudah pengelolaan data kepegawaian, meningkatkan efisiensi dan kecepatan pengolahan informasi di lingkungan pemerintah (Rimayanda & Maknuni, 2022).
8	Penggunaan <i>Extreme Programming</i> Untuk Menunjang Perubahan Kebutuhan Dalam Proses Pembangunan Sistem Informasi Produksi	Kela Tulvina, Nurfia Oktaviani Syamsiah, Weishsky Steven Dharmawan	2022	Penggunaan <i>Extreme Programming</i> (XP) berhasil memberikan fleksibilitas dalam menghadirkan sistem yang dapat menyesuaikan perubahan kebutuhan dalam pembangunan sistem produksi.
9	Peranan Sistem Informasi Manajemen	Akhmad Sirojuddin, Khus	2022	Sistem informasi manajemen (SIM) di MI Darussalam

	Dalam Pengambilan Keputusan di Madrasah Ibtidaiyah Darussalam Pacet Mojokerto	Amirullah, Muhammad Husnur Rofiq, Ari Kartiko		membantu kepala sekolah dan guru dalam pengambilan keputusan berbasis data yang lebih akurat dan cepat.
10	<i>What impacts learning effectiveness of a mobile learning app focused on first-year students</i>	Florian Johannsen, Martin Knipp, Thomas Loy, Milad Mirbabaie, Nicholas R. J. Möllmann, Johannes Voshaar, Jochen Zimmermann	2023	Penelitian ini mengidentifikasi bahwa efektivitas pembelajaran melalui aplikasi <i>mobile</i> dipengaruhi oleh kepuasan pengguna, kualitas sistem, dan kualitas informasi.

Berdasarkan telaah terhadap sepuluh penelitian terdahulu, dapat disimpulkan bahwa keseluruhan studi memiliki kesamaan pada fokus penggunaan teknologi informasi untuk meningkatkan efisiensi pengelolaan data, penyebaran informasi, dan proses administrasi dalam berbagai organisasi. Berbagai metode pengembangan perangkat lunak seperti *Extreme Programming*, pemanfaatan sistem informasi berbasis *web* dan *mobile*, serta pendekatan teknologi–organisasi–lingkungan menunjukkan bahwa para peneliti terdahulu sepakat bahwa digitalisasi merupakan kunci utama dalam

meningkatkan efektivitas suatu organisasi. Selain itu, sebagian penelitian juga menekankan pentingnya kualitas sumber daya manusia dan penerimaan teknologi sebagai aspek pendukung keberhasilan implementasi sistem.

Namun demikian, dari keseluruhan penelitian yang ditinjau, belum ada yang secara khusus mengembangkan aplikasi terintegrasi untuk mendukung operasional organisasi kemahasiswaan, terutama dalam hal manajemen kegiatan, penyebaran informasi yang lebih terstruktur, pendataan anggota, serta dokumentasi kegiatan yang mudah diakses oleh seluruh anggota organisasi. Sebagian besar penelitian hanya berfokus pada sistem keuangan, kepegawaian, fasilitas kampus, ataupun *E-government*, sehingga belum menjawab permasalahan khas dalam organisasi mahasiswa yang memiliki dinamika komunikasi cepat dan kebutuhan koordinasi yang tinggi. Oleh karena itu, penelitian ini menjadi penting untuk dilakukan karena berupaya mengisi kekosongan tersebut dengan mengembangkan *Sistem Informasi HIMAPL berbasis mobile menggunakan Framework Ionic* yang didukung metode *Agile Sprint*. Penelitian ini tidak hanya berkontribusi pada literatur sistem informasi organisasi mahasiswa, tetapi juga memberikan solusi praktis untuk mengatasi masalah penyebaran informasi yang tidak merata, dokumentasi kegiatan yang tercecer, dan proses administrasi yang masih manual, sehingga mampu meningkatkan profesionalitas serta tata kelola organisasi kemahasiswaan secara menyeluruh.

2.2 Landasan Teori

2.2.1 Sistem Informasi

Sistem informasi dapat dipahami sebagai sekumpulan komponen yang saling berhubungan untuk menghasilkan informasi yang bernilai. Dari perspektif teknis, sebuah *information system* didefinisikan sebagai seperangkat komponen yang saling terkait untuk mengumpulkan (atau mengambil), memproses, menyimpan, dan mendistribusikan informasi (Laudon & Laudon, 2022). Definisi ini menekankan bahwa sistem informasi tidak hanya berkaitan dengan perangkat komputer, tetapi mencakup aliran kerja pengolahan data menjadi informasi.

Selain perspektif teknis, sistem informasi juga dapat dipandang dari sisi bisnis dan organisasi. Sistem informasi berperan sebagai solusi manajerial dan organisasional berbasis teknologi untuk menjawab tantangan atau masalah yang muncul dari lingkungan organisasi (Laudon & Laudon, 2022). Dengan demikian, sistem informasi melekat pada cara organisasi menjalankan proses kerja, mengoordinasikan aktivitas, serta mendukung pengambilan keputusan melalui informasi yang tepat.

Dalam operasionalnya, sistem informasi mentransformasikan data mentah menjadi informasi melalui tiga aktivitas dasar, yaitu input, processing, dan output. Informasi yang dihasilkan selanjutnya dimanfaatkan untuk mendukung fungsi-fungsi organisasi seperti pengambilan keputusan, komunikasi, koordinasi, pengendalian, analisis, dan visualisasi. Melalui dukungan tersebut,

sistem informasi membantu organisasi mencapai berbagai tujuan strategis, misalnya peningkatan keunggulan operasional, pengembangan produk/layanan dan model bisnis baru, peningkatan kedekatan dengan pelanggan/pemasok, perbaikan kualitas keputusan, penciptaan keunggulan bersaing, serta pemenuhan kebutuhan operasional sehari-hari (Laudon & Laudon, 2022).

Dalam kajian akademik, terdapat pula pendekatan yang menekankan bahwa sistem informasi tidak selalu identik dengan teknologi. Alter memandang sistem informasi sebagai kasus khusus dari *work system*, yaitu sistem kerja yang menghasilkan produk/layanan bagi pelanggan atau pemangku kepentingan, dan dapat bersifat manual, sebagian terotomasi, atau sepenuhnya terotomasi (Urbach & Müller, 2012). Pendekatan ini membantu membedakan antara sistem informasi (sebagai sistem kerja) dan teknologi informasi (sebagai enabler).

Keberhasilan penerapan sistem informasi umumnya dinilai melalui dampak dan manfaat yang dihasilkan. Model kesuksesan sistem informasi dari DeLone dan McLean menekankan bahwa evaluasi sistem dapat dilihat dari kualitas sistem, kualitas informasi, kualitas layanan, penggunaan/niat penggunaan, kepuasan pengguna, dan manfaat bersih (net benefits) (DeLone & Mclean, 2003). Kerangka ini banyak digunakan untuk memahami apakah suatu sistem benar-benar memberikan nilai bagi individu maupun organisasi.

a. Komponen Sistem Informasi

Sistem informasi terdiri dari lima komponen utama yang saling berinteraksi, yaitu Perangkat Keras (*Hardware*), Perangkat Lunak

(*Software*), Data, Prosedur (*Procedure*), dan Pengguna (*People*). Semua komponen tersebut bekerja bersama untuk memastikan kelancaran pengelolaan informasi di dalam sistem.

b. Fungsi Sistem Informasi

Menurut Laudon Kenneth dan Laudon Jane (2021), sistem informasi berfungsi untuk mengumpulkan, memproses, menyimpan, dan mendistribusikan informasi guna mendukung pengambilan keputusan, koordinasi, serta pengendalian dalam organisasi, termasuk membantu proses analisis dan visualisasi informasi (Laudon & Laudon, 2022).

c. Peran Sistem Informasi dalam Organisasi

Sistem informasi memainkan peran yang sangat penting dalam operasional organisasi modern. Seiring berkembangnya teknologi, kebutuhan akan sistem informasi yang cepat dan efisien semakin meningkat. Menurut Laudon Kenneth dan Laudon Jane (2021), sistem informasi berperan sebagai rangkaian aktivitas yang menambah nilai dalam memperoleh, mengolah, dan mendistribusikan informasi sehingga manajer dapat meningkatkan kualitas pengambilan keputusan, memperbaiki kinerja organisasi, serta mendukung koordinasi dan pengendalian. Selain itu, Laudon Kenneth dan Laudon Jane (2021) juga menekankan bahwa banyak manajer menghadapi situasi *information fog* karena tidak memiliki informasi yang tepat pada waktu yang tepat, sehingga penerapan sistem informasi membantu mengatasi masalah tersebut melalui penyediaan informasi yang lebih relevan dan tepat waktu (Laudon & Laudon, 2022).

2.2.2 *Mobile*

Mobile Computing merujuk pada penggunaan perangkat *mobile*, seperti *smartphone*, *tablet*, atau *wearable devices*, untuk mengakses informasi dan menjalankan aplikasi dari berbagai lokasi. Teknologi *mobile* memungkinkan pengguna untuk tetap terhubung dengan sistem informasi tanpa harus berada di lokasi tetap, seperti kantor atau ruang kerja.

a. Definisi *Mobile Computing*

Mobile Computing menggabungkan perangkat keras, perangkat lunak, dan konektivitas nirkabel untuk memberikan kemampuan akses ke data dan aplikasi secara fleksibel. Dengan adanya teknologi ini, individu dapat mengakses aplikasi dan data kapan saja dan di mana saja, yang meningkatkan mobilitas dan produktivitas pengguna. Sistem *mobile* umumnya menggunakan jaringan nirkabel, seperti *Wi-Fi*, *Bluetooth*, dan *5G*, untuk mentransfer data antar perangkat dan server (S et al., 2025).

b. Komponen *Mobile Computing*

Mobile Computing terdiri dari beberapa komponen utama:

1. **Perangkat *Mobile*:** Termasuk *smartphone*, *tablet*, dan perangkat *wearable* yang digunakan untuk mengakses aplikasi dan data. Perangkat ini dilengkapi dengan berbagai fitur seperti **sensor**, **kamera**, dan **GPS** yang memperkaya pengalaman pengguna.

2. **Jaringan Nirkabel:** Jaringan yang memungkinkan komunikasi antar perangkat *mobile* dan server. Jaringan ini bisa berupa **Wi-Fi**, **Bluetooth**, atau **jaringan seluler** seperti **4G** atau **5G**.
3. **Perangkat Lunak (Software):** Aplikasi yang berjalan pada perangkat *mobile* dan mengelola interaksi dengan pengguna serta *backend*. Aplikasi HIMAPL yang dikembangkan menggunakan **Framework Ionic** dan **JavaScript** akan memanfaatkan teknologi *mobile* untuk mengelola data dan mendukung komunikasi antar anggota.

c. Keuntungan dan Tantangan *Mobile Computing*

Mobile Computing memiliki beberapa keuntungan yang signifikan dalam konteks organisasi, seperti:

1. **Aksesibilitas:** Pengguna dapat mengakses aplikasi dan data kapan saja dan di mana saja, meningkatkan efisiensi dan produktivitas.
2. **Fleksibilitas:** Pengguna dapat berkolaborasi dan berkomunikasi dengan tim atau anggota lainnya tanpa terbatas oleh lokasi fisik.
3. **Penghematan Waktu:** Aplikasi *mobile* memungkinkan pengguna untuk menyelesaikan tugas-tugas dan mengambil keputusan lebih cepat karena akses data yang lebih cepat dan mudah.

Namun, ada juga beberapa tantangan yang harus dihadapi dalam pengembangan aplikasi *mobile*:

1. **Keterbatasan Perangkat:** Perangkat *mobile* memiliki keterbatasan dalam hal kapasitas penyimpanan, daya baterai, dan ukuran layar, yang mempengaruhi desain aplikasi dan fungsionalitasnya.
2. **Keamanan:** Keamanan data dan privasi menjadi tantangan besar dalam aplikasi *mobile*, terutama saat data sensitif ditransfer melalui jaringan nirkabel.
3. **Kompatibilitas:** Aplikasi yang dikembangkan untuk perangkat *mobile* harus kompatibel dengan berbagai *platform* (misalnya, *Android* dan *iOS*), yang dapat meningkatkan kompleksitas pengembangan.

d. Pengaruh *Mobile Computing* pada Organisasi

Berbagai studi menunjukkan bahwa adopsi *mobile computing* dalam organisasi dapat mempercepat proses bisnis melalui akses data dan aplikasi secara real-time, sehingga meningkatkan efisiensi operasional dan pengambilan keputusan. *Mobile computing* juga mendukung kolaborasi karena memungkinkan komunikasi, koordinasi tugas, dan berbagi informasi secara instan di antara anggota yang tersebar secara geografis, termasuk dalam konteks organisasi mahasiswa. Selain itu, desain *mobile* yang menekankan kemudahan penggunaan dan pengalaman pengguna (user experience) yang baik berkontribusi pada keterlibatan dan kepuasan pengguna terhadap layanan organisasi (Ndukuyu et al., 2016).

2.2.3 *Android*

Android adalah sistem operasi berbasis *Open-Source* yang dirancang khusus untuk perangkat *mobile* seperti *smartphone*, *tablet*, dan *wearable devices*. Dikembangkan oleh *Google*, *Android* telah menjadi sistem operasi *mobile* yang paling banyak digunakan di dunia. *Android* memungkinkan pengembang untuk membuat aplikasi *mobile* yang fleksibel dan mudah diakses di berbagai perangkat dan *platform*.

a. Definisi dan Karakteristik *Android*

Android adalah sistem operasi *mobile* yang dibangun di atas kernel *Linux* dan dilengkapi lapisan *middleware*, *library*, dan *application Framework* untuk menjalankan aplikasi. Sistem ini awalnya dirancang agar aplikasi dikembangkan dengan bahasa *Java* pada *Android SDK* dan dijalankan di atas *Dalvik/ART runtime*, dengan dukungan *Kotlin* sebagai bahasa modern yang juga berjalan di lingkungan *Java virtual machine*. *Android* menyediakan himpunan *API* yang kaya untuk mengakses berbagai fitur perangkat keras dan sensor, seperti kamera, *GPS*, sensor gerak, serta layar sentuh, sehingga memungkinkan pengembangan aplikasi yang memanfaatkan kemampuan perangkat secara penuh. Arsitektur *Android* dirancang agar aplikasi dapat berjalan di beragam perangkat dengan variasi ukuran layar, kapasitas penyimpanan, dan karakteristik daya baterai, selama perangkat tersebut memenuhi persyaratan versi dan spesifikasi *Android* yang didukung (Gutiérrez et al., 2022; Joann, 2015).

b. Komponen Utama *Android*

Android terdiri dari beberapa komponen utama yang membentuk sistem operasi ini:

1. **Linux Kernel:** Merupakan inti dari sistem operasi *Android* yang bertanggung jawab untuk mengelola perangkat keras dan sumber daya lainnya.
2. **Libraries:** Merupakan kumpulan pustaka yang menyediakan fungsi-fungsi tambahan untuk aplikasi, seperti **WebKit** (untuk rendering halaman web), **SQLite** (untuk penyimpanan data), dan **OpenGL ES** (untuk grafis 3D).
3. **Android Runtime (ART):** Merupakan lingkungan eksekusi aplikasi *Android* yang menggantikan Dalvik (mesin virtual *Android* sebelumnya), dan digunakan untuk menjalankan aplikasi *Android*.
4. **Application Framework:** Memberikan berbagai *API* yang memungkinkan pengembang untuk mengakses layanan sistem dan perangkat keras. *API* ini juga memungkinkan aplikasi untuk berkomunikasi dengan aplikasi lain di dalam sistem *Android*.
5. **Aplikasi:** Aplikasi *Android* adalah program-program yang dijalankan di atas *Android Runtime* dan dapat diunduh dan diunduh dari **Google Play Store** atau sumber lain.

c. Keuntungan Menggunakan *Android* dalam Pengembangan Aplikasi

1. **Open-Source:** Sebagai sistem operasi *Open-Source*, *Android* memungkinkan pengembang untuk memodifikasi dan mengembangkan aplikasi sesuai dengan kebutuhan mereka. Ini memberikan kebebasan dan fleksibilitas tinggi dalam pengembangan aplikasi.
2. **Ekosistem yang Luas:** *Android* mendukung berbagai jenis perangkat dari berbagai produsen, yang memungkinkan aplikasi *Android* untuk dijangkau oleh audiens yang lebih luas.
3. **Integrasi dengan Google Services:** *Android* memungkinkan aplikasi untuk mengintegrasikan berbagai layanan *Google*, seperti *Google Maps*, *Google Drive*, dan *Firebase*, yang dapat meningkatkan pengalaman pengguna dan fungsionalitas aplikasi.
4. **Dukungan Komunitas yang Kuat:** *Android* memiliki komunitas pengembang yang besar, yang menyediakan banyak pustaka, tutorial, dan forum diskusi untuk mendukung pengembang dalam membangun aplikasi *mobile*.

d. Tantangan Pengembangan Aplikasi *Android*

Meskipun *Android* menawarkan banyak keuntungan, pengembangan aplikasi *Android* juga memiliki tantangan tersendiri. Salah satunya adalah **fragmentasi perangkat**, yang berarti bahwa aplikasi harus dapat berfungsi dengan baik di berbagai perangkat dengan ukuran layar, versi *Android*, dan spesifikasi perangkat yang berbeda. Hal ini memerlukan pengujian yang lebih ketat dan pengelolaan versi aplikasi yang baik.

2.2.4 *Ionic Framework*

Ionic Framework adalah sebuah perangkat pengembang perangkat lunak (*Software Development Kit/SDK*) sumber terbuka (*open-source*) yang digunakan untuk membangun aplikasi lintas *platform* (*cross-platform*) berkualitas tinggi, seperti aplikasi *mobile* (*Android* dan *iOS*), *desktop*, serta *Progressive Web Apps (PWA)*, hanya dengan menggunakan satu basis kode (*single codebase*). Pertama kali dirilis pada tahun 2013, *Ionic* berfokus pada tampilan antarmuka pengguna (*frontend* dan *User Interface/UI*) aplikasi (Ionicframework, n.d.).

Berbeda dengan pengembangan *native* konvensional yang mengharuskan penggunaan bahasa pemrograman spesifik seperti *Kotlin/Java* untuk *Android* atau *Swift/Objective-C* untuk *iOS*, *Ionic* memanfaatkan teknologi *web* standar yang sudah sangat populer, yaitu *HTML*, *CSS*, dan *JavaScript/TypeScript*. Guna mempercepat proses pengembangan dan memastikan performa yang optimal, *Ionic* dapat diintegrasikan secara penuh dengan berbagai *framework JavaScript* modern, salah satunya adalah *Vue.js*.

a. Arsitektur dan Mekanisme Kerja

Secara arsitektural, *Ionic* beroperasi sebagai aplikasi *web* yang berjalan di dalam kontainer *native* pada perangkat seluler. Untuk menjembatani kode

web tersebut dengan kapabilitas perangkat keras, *Ionic* mengandalkan komponen-komponen utama sebagai berikut:

1. *Web View*: *Ionic* merender antarmuka aplikasi melalui komponen *Web View* (seperti *WKWebView* pada *iOS* atau *WebView* berbasis *Chromium* pada *Android*). Hal ini membuat aplikasi *Ionic* pada dasarnya memiliki fleksibilitas seperti halaman *web*, namun dikemas dan berperilaku layaknya aplikasi *native*.
2. *Capacitor* atau *Cordova*: Komponen ini bertindak sebagai jembatan (*bridge*) *runtime* lintas *platform*. *Bridge* inilah yang bertugas menerjemahkan *API JavaScript* dari *Ionic* agar dapat memanggil dan mengontrol fitur-fitur hardware perangkat secara langsung, seperti kamera, *GPS*, sensor gerak, sistem notifikasi, hingga penyimpanan lokal.
3. *Web Components*: Sejak versi modernnya, *Ionic* mengadopsi struktur *Web Components*. Setiap elemen *UI* di dalam *Ionic* (seperti tombol, kartu, menu, dan form *input*) dibungkus sebagai komponen *web* standar yang terisolasi. Hal ini memastikan bahwa performa *rendering* komponen tetap cepat dan konsisten terlepas dari *framework* utama yang digunakan oleh pengembang.

b. Komponen *UI* dan Desain Adaptif

Salah satu kekuatan utama *Ionic Framework* terletak pada pustaka komponen *UI* (*UI Component Library*) yang sangat kaya dan siap pakai.

Komponen-komponen ini telah dirancang untuk menerapkan prinsip *Adaptive Styling* (Gaya Adaptif).

Ketika aplikasi dijalankan pada perangkat *Android*, Ionic akan secara otomatis menyesuaikan tampilan elemen *UI*-nya agar mengikuti pedoman *Material Design* khas *Google*. Sebaliknya, jika aplikasi dijalankan pada perangkat *iOS*, tampilannya akan otomatis berubah mengikuti pedoman *Cupertino Design* milik *Apple*. Hal ini memberikan pengalaman pengguna (*user experience*) yang familier dan terasa *native* bagi masing-masing platform tanpa memerlukan modifikasi kode tambahan dari sisi pengembang.

2.2.5 *Vue.js*

Vue.js adalah sebuah *framework JavaScript* yang bersifat progresif (*progressive framework*) dan berbasis open-source, yang dirancang khusus untuk membangun antarmuka pengguna (*User Interface*) serta *Single Page Applications (SPA)* yang dinamis dan interaktif (Pšenák & Tibenský, 2020). Diciptakan oleh *Evan You* pada tahun 2014, kata "progresif" pada *Vue.js* merujuk pada arsitekturnya yang fleksibel dan mudah diadaptasikan secara bertahap ke dalam proyek perangkat lunak.

Pengembang dapat menggunakan *Vue.js* hanya sebagai pustaka skrip minor pada sebagian halaman *web*, atau mengembangkannya secara masif menjadi sebuah ekosistem *frontend* yang utuh untuk aplikasi skala besar. Dalam pengembangan aplikasi *mobile* berbasis *Ionic*, *Vue.js* bertindak sebagai motor

penggerak logika aplikasi di sisi klien (*client-side*), mengelola aliran data (*data flow*), serta mengatur perilaku interaktif dari setiap komponen antarmuka.

a. Karakteristik Utama dan Arsitektur

Vue.js memiliki beberapa karakteristik arsitektural inti yang membuatnya sangat efisien dalam memproses performa aplikasi, antara lain:

1. *Virtual DOM (Document Object Model)*: *Vue.js* mengimplementasikan konsep *Virtual DOM*, yaitu sebuah replika memori ringan dari struktur *HTML* asli di peramban. Ketika terjadi perubahan data atau interaksi pengguna, *Vue.js* tidak langsung memperbarui seluruh dokumen *HTML* asli secara mentah, melainkan melakukan kalkulasi komparasi (*diffing algorithm*) pada *Virtual DOM* untuk menemukan bagian spesifik yang berubah. Proses pembaruan ke *DOM* asli hanya dilakukan pada bagian yang membutuhkan perubahan tersebut, sehingga menghemat konsumsi memori perangkat *Android* dan mempercepat waktu respons antarmuka.
2. *Reaktivitas Data (Reactivity System)*: *Vue.js* menggunakan sistem reaktivitas berbasis *two-way data binding* (pengikatan data dua arah). Mekanisme ini memastikan adanya sinkronisasi otomatis secara *real-time* antara lapisan data logika (*Model*) dan lapisan tampilan visual (*View*). Setiap kali terjadi mutasi data di sisi kode, tampilan

layar akan langsung diperbarui secara otomatis tanpa memerlukan manipulasi *DOM* manual dari pengembang.

3. *Single File Components (SFC)*: Kode program di dalam *Vue.js* diorganisasikan ke dalam berkas berekstensi *(.vue)* yang disebut *Single File Components*. Berkas ini menyatukan tiga pilar utama pemrograman web dalam satu tempat yang terisolasi, yaitu struktur visual, logika program *JavaScript/TypeScript*, dan dekorasi gaya.

2.2.6 *Supabase*

Supabase adalah sebuah platform *Backend-as-a-Service (BaaS)* berbasis sumber terbuka (*open-source*) yang menyediakan kumpulan perangkat dan layanan komputasi awan terintegrasi. *Platform* ini dirancang untuk memfasilitasi pengembang perangkat lunak dalam membangun arsitektur sisi peladen (*backend*) secara cepat dan terukur, tanpa perlu melakukan konfigurasi dan pengelolaan infrastruktur peladen (*server*) secara manual (Supabase, n.d.).

Berbeda dengan platform *BaaS* populer lainnya yang mayoritas bertumpu pada arsitektur basis data *NoSQL*, *Supabase* dibangun secara utuh di atas sistem manajemen basis data relasional (*RDBMS*) tingkat *enterprise* yang sangat tangguh, yaitu *PostgreSQL*. Pendekatan ini memungkinkan aplikasi untuk mengelola skema data yang kompleks, menjaga integritas relasional antar-tabel, sekaligus memanfaatkan antarmuka *API (Application Programming Interface)* yang di-generate secara otomatis oleh sistem (melalui komponen *PostgREST*).

- a. *Komponen dan Fitur Utama Supabase*

Secara umum, ekosistem *Supabase* menyediakan berbagai komponen layanan inti yang dirancang untuk mendukung kebutuhan infrastruktur backend secara terpadu, antara lain:

1. *Supabase Database (PostgreSQL)*: Merupakan jantung utama dari layanan *Supabase* yang bertugas mengelola dan menyimpan data aplikasi secara relasional. Selain fungsi penyimpanan standar, komponen ini juga mendukung pelaksanaan komputasi tingkat lanjut bawaan *PostgreSQL*, seperti *Database Triggers* dan *Functions*. Fitur ini memungkinkan sistem untuk mengeksekusi logika atau aksi terprogram secara otomatis di sisi peladen setiap kali terjadi peristiwa mutasi data (seperti *Insert*, *Update*, atau *Delete*).
2. *Supabase Auth*: Merupakan layanan manajemen identitas dan autentikasi bawaan yang terintegrasi secara langsung dengan basis data. Modul ini bertanggung jawab untuk memproses logika pendaftaran pengguna, proses masuk (*login*), validasi kredensial, serta manajemen *token* sesi keamanan berbasis *JWT (JSON Web Token)*. *Supabase Auth* juga memfasilitasi penerapan kontrol akses berbasis peran (*role-based access control*), sehingga sistem dapat mengidentifikasi, membedakan, dan membatasi hak akses dari masing-masing jenis pengguna.
3. *Supabase Storage*: Layanan penyimpanan objek (*object storage*) yang dirancang khusus untuk mengelola berkas media dengan ukuran besar (*Binary Large Objects / BLOB*). Dalam arsitektur

aplikasi, komponen ini memegang peranan vital sebagai lokasi komputasi penyimpanan berkas-berkas aset statis, seperti gambar, dokumen, atau video. Layanan ini juga dilengkapi dengan manajemen kebijakan akses (*access policies*) yang mengatur apakah sebuah berkas bersifat publik atau hanya dapat diakses oleh pengguna terautentikasi.

b. Mekanisme Keamanan *Row Level Security (RLS)*

Salah satu keunggulan arsitektural *Supabase* adalah penerapan *Row Level Security (RLS)* yang diwariskan dari *PostgreSQL*. *RLS* adalah kebijakan keamanan tingkat baris yang mengevaluasi setiap kueri eksekusi (seperti *Insert*, *Update*, atau *Delete*) berdasarkan aturan kebijakan terenkripsi sebelum aksi tersebut diizinkan menyentuh basis data.

Sebagai lapisan pelindung otorisasi, *RLS* berfungsi layaknya *middleware* keamanan yang memblokir akses secara presisi. Sistem dapat memastikan bahwa eksekusi pengubahan data hanya sah dan dapat diproses apabila pengguna aktif yang mengirimkan kueri tersebut memenuhi kriteria peran (otorisasi) yang telah ditetapkan. Jika akses dikirimkan oleh pihak yang tidak berwenang, kebijakan *RLS* akan secara otomatis menolak akses tersebut dan mengembalikan status penolakan (*Unauthorized*) ke aplikasi klien.

2.2.7 *Agile*

Agile adalah pendekatan pengembangan perangkat lunak yang mengutamakan fleksibilitas, kolaborasi tim, dan respons terhadap perubahan. *Agile* menekankan pentingnya iterasi, pengembangan bertahap, serta umpan balik yang cepat dari pengguna. Metode ini pertama kali diperkenalkan melalui ***Manifesto Agile*** pada tahun 2001 oleh sekelompok pengembang perangkat lunak yang merasa perlu mengubah pendekatan pengembangan tradisional (seperti *Waterfall*) yang lebih kaku dan linear (Quist, 2015; Wulandari & Raharjo, 2023).

a. Prinsip Dasar *Agile*

Prinsip-prinsip utama *Agile* tercantum dalam ***Manifesto Agile***, yang terdiri dari empat nilai inti:

1. **Individu dan interaksi lebih penting daripada proses dan alat:**
Fokus utama *Agile* adalah pada kolaborasi antar anggota tim dan pengguna.
2. **Perangkat lunak yang berfungsi lebih penting daripada dokumentasi yang mendetail:** *Agile* lebih menekankan pada pengembangan perangkat lunak yang dapat digunakan daripada pembuatan dokumentasi yang kompleks.
3. **Kolaborasi dengan pengguna lebih penting daripada negosiasi kontrak:** Pengembangan dilakukan dengan melibatkan pengguna secara langsung agar produk akhir lebih sesuai dengan kebutuhan.

4. MenanggAPI perubahan lebih penting daripada mengikuti

rencana: *Agile* bersifat adaptif terhadap perubahan dan lebih menekankan pada fleksibilitas daripada mengikuti rencana yang sudah ditetapkan.

b. Metode *Agile* dan Keuntungannya

Metode *Agile* mengedepankan pembagian proyek menjadi bagian-bagian kecil yang disebut **iterasi** atau ***Sprint***, yang biasanya berlangsung antara satu hingga empat minggu. Setiap *Sprint* menghasilkan sebuah bagian perangkat lunak yang dapat digunakan atau diuji oleh pengguna, dan tim akan memperoleh umpan balik untuk meningkatkan produk dalam *Sprint* berikutnya.

Keuntungan utama menggunakan ***Agile*** adalah:

1. **Fleksibilitas dan Adaptasi:** Pengembang dapat dengan cepat menanggAPI perubahan kebutuhan pengguna atau pasar.
2. **Kolaborasi yang Lebih Baik:** Dengan melibatkan pengguna secara aktif dalam setiap tahap, *Agile* memastikan bahwa hasil akhir sesuai dengan harapan pengguna.
3. **Peningkatan Kualitas:** Setiap *Sprint* mencakup pengujian dan evaluasi, sehingga kualitas produk selalu diperiksa dan ditingkatkan.
4. **Pengurangan Risiko:** Dengan pembagian proyek ke dalam *Sprint* yang lebih kecil, risiko kegagalan dapat diminimalkan karena evaluasi dilakukan secara terus-menerus.

c. Tantangan dalam Mengimplementasikan *Agile*

Meskipun *Agile* menawarkan banyak keuntungan, implementasinya juga memiliki tantangan, seperti:

1. **Komunikasi yang Intensif:** *Agile* membutuhkan komunikasi yang terus-menerus antara pengembang dan pengguna. Ini bisa menjadi tantangan jika tim pengembang tidak cukup terkoordinasi dengan baik.
2. **Keterbatasan Waktu untuk Setiap *Sprint*:** Setiap *Sprint* membutuhkan waktu yang terbatas, dan jika tugas terlalu banyak atau terlalu rumit, maka kualitas dan jadwal pengembangan dapat terganggu.
3. **Keterlibatan Pengguna:** Keberhasilan *Agile* sangat bergantung pada keterlibatan aktif pengguna dalam setiap fase *Sprint*. Kurangnya keterlibatan dari pihak pengguna dapat mengurangi efektivitas metode ini.

2.2.8 *Sprint*

Sprint adalah iterasi waktu yang pendek dalam metodologi *Agile* (terutama Scrum), di mana tim pengembang bekerja untuk menyelesaikan tugas-tugas yang telah ditentukan dalam *Sprint Backlog*. *Sprint* biasanya berlangsung selama 1 hingga 4 minggu, dan pada akhir setiap *Sprint*, sebuah Increment produk yang dapat digunakan atau diuji harus diselesaikan.

a. Definisi dan Karakteristik *Sprint*

Sprint adalah inti dari metodologi Scrum, yang merupakan salah satu kerangka kerja *Agile*. Setiap *Sprint* dimulai dengan *Sprint Planning* di mana tim menentukan tugas yang akan dikerjakan, dan diakhiri dengan *Sprint Review* untuk menilai hasil dari tugas-tugas yang telah diselesaikan. Selain itu, terdapat *Daily scrum* yang merupakan pertemuan harian untuk memonitor kemajuan dan mengatasi hambatan (Purwandi, 2024).

Beberapa karakteristik penting dari *Sprint* adalah:

1. **Durasi Tertentu:** Setiap *Sprint* memiliki durasi yang tetap (biasanya 1-4 minggu) dan diharapkan untuk menghasilkan bagian dari produk yang dapat dipresentasikan kepada pengguna.
2. **Iteratif dan Inkremental:** Setiap *Sprint* berfokus pada bagian tertentu dari produk, yang dikembangkan dan diuji secara bertahap. Dengan demikian, produk final dibangun secara bertahap melalui serangkaian *Sprint*.
3. **Kolaboratif:** *Sprint* mengedepankan kolaborasi antar anggota tim pengembang, pemangku kepentingan, dan pengguna untuk memastikan bahwa pengembangan berjalan sesuai dengan kebutuhan.
4. **Fokus pada Tujuan yang Terukur:** Pada akhir *Sprint*, tim bertujuan untuk menyelesaikan bagian fungsional produk yang dapat digunakan atau diuji oleh pengguna.

b. Tahapan dalam *Sprint*

1. ***Sprint Planning***: Di awal setiap *Sprint*, tim mengadakan pertemuan untuk merencanakan dan memutuskan pekerjaan yang akan dilakukan selama *Sprint*. ***Sprint Backlog*** yang telah disusun sebelumnya menjadi acuan utama.
2. ***Implementation (Development)***: Pengembang mulai bekerja pada tugas-tugas yang telah disepakati dalam ***Sprint Backlog***. Fase ini meliputi coding, desain, dan pengujian.
3. ***Daily scrum***: Setiap hari, tim mengadakan pertemuan singkat (biasanya 15 menit) untuk membahas progres, hambatan, dan langkah berikutnya. Tujuan dari ***Daily scrum*** adalah untuk memastikan semua anggota tim terkoordinasi dan masalah dapat segera diatasi.
4. ***Sprint Review***: Setelah *Sprint* selesai, diadakan pertemuan untuk meninjau hasil *Sprint*. Produk yang telah dikembangkan dipresentasikan dan dievaluasi. Pengguna memberikan umpan balik yang akan dipertimbangkan untuk *Sprint* berikutnya.
5. ***Sprint Retrospective***: Setelah ***Sprint Review***, tim melakukan refleksi mengenai proses *Sprint*, mengidentifikasi hal yang berjalan baik dan hal yang perlu diperbaiki untuk *Sprint* selanjutnya. Ini bertujuan untuk meningkatkan proses kerja tim.

c. Keuntungan Menggunakan *Sprint* dalam Pengembangan Aplikasi

Penggunaan *Sprint* dalam pengembangan aplikasi memiliki banyak keuntungan, antara lain:

1. **Fleksibilitas dan Adaptasi:** Dengan adanya *Sprint*, tim pengembang dapat dengan mudah beradaptasi dengan perubahan kebutuhan pengguna. Hasil *Sprint* dapat diuji dan disesuaikan dengan umpan balik pengguna.
2. **Penyelesaian Cepat:** *Sprint* memberikan batasan waktu yang jelas untuk setiap iterasi, yang memaksa tim untuk menyelesaikan tugas secara efisien dan tepat waktu.
3. **Peningkatan Kualitas:** Dengan evaluasi yang dilakukan di setiap akhir *Sprint*, kesalahan atau masalah dapat segera diperbaiki, sehingga meningkatkan kualitas produk secara keseluruhan.
4. **Kolaborasi yang Lebih Baik:** *Sprint* mendorong kolaborasi yang intens antar anggota tim, pemangku kepentingan, dan pengguna akhir, sehingga produk yang dihasilkan lebih sesuai dengan harapan dan kebutuhan.

2.2.9 Java Script

JavaScript adalah bahasa pemrograman tingkat tinggi yang digunakan untuk mengembangkan aplikasi *web* dan *mobile*. *JavaScript* memungkinkan pengembang untuk menciptakan antarmuka pengguna yang dinamis, responsif, dan interaktif. Sebagai salah satu bahasa yang paling banyak digunakan di dunia,

JavaScript mendukung berbagai paradigma pemrograman seperti ***event-driven***, ***functional***, dan ***imperative Programming***.

a. Definisi dan Karakteristik *JavaScript*

JavaScript pertama kali dikembangkan oleh **Brendan Eich** pada tahun 1995 di ***Netscape Communications*** dan sejak itu menjadi bahasa standar untuk pengembangan aplikasi *web* di sisi klien. Dalam konteks pengembangan aplikasi *mobile*, ***JavaScript*** digunakan dalam kerangka kerja **hybrid** seperti ***Ionic*** untuk membangun aplikasi yang dapat berjalan di berbagai *platform*, termasuk ***Android*** dan ***iOS*** (Wirfs-Brock & Eich, 2020).

Karakteristik utama dari *JavaScript* adalah:

1. ***Event-Driven***: *JavaScript* menggunakan ***event listeners*** untuk menunggu dan merespons berbagai peristiwa (seperti klik atau *input* pengguna), yang memungkinkan aplikasi berjalan secara asinkron.
2. ***Asynchronous Programming***: *JavaScript* mendukung pemrograman asinkron menggunakan ***callback functions***, ***Promises***, dan ***async/await***, yang memungkinkan eksekusi kode tanpa memblokir proses lainnya.
3. ***Cross-Platform***: Dengan bantuan *Framework* seperti ***React Native*** dan ***Ionic***, *JavaScript* dapat digunakan untuk mengembangkan aplikasi *mobile* yang berjalan di berbagai *platform*, seperti ***Android*** dan ***iOS***.

4. **Dynamic Typing:** *JavaScript* tidak memerlukan deklarasi tipe data secara eksplisit. Variabel dapat diubah jenis datanya pada saat *Runtime*, memberikan fleksibilitas lebih bagi pengembang.

b. Keuntungan Menggunakan *JavaScript* dalam Pengembangan Aplikasi

1. **Pengembangan Cross-Platform:** Dengan menggunakan *Ionic*, pengembang dapat menulis satu kode sumber dalam *JavaScript* yang dapat dijalankan di berbagai *platform*, seperti *Android*, *iOS*, dan *web*. Ini mengurangi waktu dan biaya pengembangan.
2. **Interaktivitas Tinggi:** *JavaScript* memungkinkan pembuatan aplikasi yang lebih dinamis dan responsif, di mana pengguna dapat berinteraksi langsung dengan aplikasi tanpa perlu memuat ulang halaman.
3. **Ekosistem yang Luas:** *JavaScript* memiliki ekosistem pustaka dan *Framework* yang sangat besar, yang menyediakan banyak fitur dan solusi siap pakai untuk mempercepat pengembangan aplikasi.

c. Tantangan Penggunaan *JavaScript*

Meskipun *JavaScript* menawarkan banyak keuntungan, ada beberapa tantangan yang perlu diperhatikan:

1. **Kompleksitas dalam Debugging:** Karena *JavaScript* bersifat dinamis dan asinkron, debugging kode dapat menjadi lebih sulit, terutama dalam aplikasi besar dengan banyak interaksi.

2. **Kompatibilitas Browser dan Platform:** Meskipun *JavaScript* dapat dijalankan di berbagai perangkat, perbedaan versi *JavaScript* atau browser dapat mempengaruhi kompatibilitas aplikasi, memerlukan pengujian lebih lanjut untuk memastikan aplikasi berjalan dengan baik di berbagai *platform*.
3. **Kinerja:** Meskipun *JavaScript* berkembang pesat, aplikasi yang sangat kompleks atau dengan banyak interaksi dapat menghadapi masalah kinerja jika tidak dioptimalkan dengan baik.

2.2.10 Basis Data

Basis Data (*database*) adalah sekumpulan data yang saling terhubung dan disimpan dalam bentuk yang terstruktur, sehingga memudahkan pengambilan, pengolahan, dan pembaruan data. Sistem basis data terdiri dari perangkat lunak yang memungkinkan pengguna untuk membuat, membaca, memperbarui, dan menghapus data (dikenal sebagai operasi **CRUD: Create, Read, Update, Delete**). Basis data digunakan dalam berbagai sistem informasi untuk mengelola informasi yang dibutuhkan organisasi (Mumtahana, 2022).

a. Definisi dan Karakteristik Basis Data

Basis data adalah kumpulan data yang diorganisir secara sistematis, memungkinkan pengguna untuk mengakses dan mengelola data secara efisien. Biasanya, data disimpan dalam tabel yang terdiri dari baris dan

kolom. Setiap baris mewakili satu entitas data, sementara setiap kolom menyimpan informasi atribut terkait.

Basis data dapat dibedakan menjadi beberapa jenis, seperti:

1. **Basis Data Relasional:** Basis data yang menyimpan data dalam tabel yang saling berhubungan. Relasi antar tabel memungkinkan data disusun dan diakses dengan cara yang lebih efisien. Sistem manajemen basis data relasional yang umum digunakan adalah *MySQL*, *PostgreSQL*, dan *SQLite*.
2. **Basis Data Non-Relasional (NoSQL):** Basis data yang tidak mengikuti struktur tabel dan relasi. Biasanya digunakan untuk aplikasi yang memerlukan skalabilitas tinggi dan fleksibilitas dalam penyimpanan data. Contoh basis data NoSQL adalah **MongoDB** dan **Cassandra**.
3. **Basis Data Terdistribusi:** Basis data yang menyimpan data di lebih dari satu lokasi fisik. Ini memungkinkan data dapat diakses secara lebih cepat dan efisien dalam lingkungan terdistribusi.

b. Komponen-Komponen dalam Basis Data

Sistem basis data terdiri dari beberapa komponen yang saling berinteraksi untuk memastikan pengelolaan data yang efisien dan konsisten:

1. **DBMS (*Database Management System*):** Perangkat lunak yang digunakan untuk membuat, mengelola, dan mengoperasikan basis data. DBMS bertanggung jawab untuk mengatur penyimpanan data,

memastikan integritas data, serta menyediakan antarmuka bagi pengguna untuk berinteraksi dengan data.

2. **Tabel (Table):** Struktur utama dalam basis data yang terdiri dari baris (record) dan kolom (field). Setiap tabel mewakili entitas data yang disimpan, seperti data anggota, kegiatan, atau pengumuman dalam aplikasi HIMAPL.
3. **Relasi (Relationship):** Hubungan antara dua atau lebih tabel yang menyimpan data terkait. Relasi ini memungkinkan informasi yang tersebar di beberapa tabel untuk dihubungkan dan digunakan secara efisien.
4. **Indeks (Index):** Struktur data yang mempercepat proses pencarian dan pengambilan data. Indeks memungkinkan pencarian data dilakukan dengan lebih cepat, terutama pada tabel besar.
5. **Query:** Bahasa yang digunakan untuk mengakses, mengubah, dan mengelola data dalam basis data. **SQL** (Structured Query Language) adalah bahasa yang paling umum digunakan untuk berinteraksi dengan basis data relasional.

c. **Keuntungan Penggunaan Basis Data dalam Pengembangan Aplikasi**

1. **Pengelolaan Data yang Efisien:** Basis data memungkinkan pengelolaan data dalam jumlah besar secara efisien. Proses pencarian, pembaruan, dan penghapusan data dilakukan dengan cepat melalui **query SQL**.

2. **Konsistensi dan Keamanan Data:** Dengan menggunakan basis data, data yang disimpan dapat dijaga konsistensinya melalui aturan dan pembatasan yang ada, seperti **constraint** dan **integritas referensial**.
3. **Skalabilitas:** Sistem basis data memungkinkan aplikasi untuk mengelola lebih banyak data seiring berkembangnya aplikasi dan organisasi. Penggunaan basis data terdistribusi atau NoSQL juga memungkinkan aplikasi mengelola data dalam skala besar.

d. Tantangan dalam Pengelolaan Basis Data

1. **Keterbatasan Perangkat:** Pada aplikasi *mobile*, keterbatasan perangkat keras, seperti kapasitas penyimpanan dan daya baterai, dapat mempengaruhi kinerja basis data. Oleh karena itu, penting untuk mengoptimalkan penggunaan basis data agar aplikasi tetap efisien.
2. **Keamanan Data:** Keamanan basis data menjadi tantangan utama, terutama ketika data yang disimpan sensitif. Proteksi data menggunakan enkripsi dan autentikasi yang tepat sangat penting untuk menjaga data tetap aman.
3. **Pengelolaan Skala Besar:** Pada aplikasi dengan jumlah data yang terus berkembang, manajemen dan pengelolaan basis data harus dilakukan dengan hati-hati untuk menghindari penurunan kinerja aplikasi.

2.2.11 Teori Pengolahan Kegiatan

Pengelolaan Kegiatan adalah proses perencanaan, pelaksanaan, pengawasan, dan evaluasi berbagai aktivitas atau program yang dilakukan oleh sebuah organisasi untuk mencapai tujuan yang telah ditetapkan. Dalam konteks organisasi seperti HIMAPL, pengelolaan kegiatan bertujuan untuk memastikan bahwa setiap kegiatan berjalan dengan baik, sesuai dengan tujuan, dan memberikan hasil yang maksimal bagi organisasi dan anggotanya.

a. Definisi Pengelolaan Kegiatan

Pengelolaan kegiatan melibatkan serangkaian langkah yang dirancang untuk mengoptimalkan penggunaan sumber daya yang ada, mengatur waktu, dan memastikan bahwa kegiatan dapat diselesaikan dengan efisien dan efektif. Proses ini mencakup beberapa tahapan, mulai dari perencanaan, pengorganisasian, pelaksanaan, pengawasan, hingga evaluasi.

Menurut (Mika et al., 2012), pengelolaan kegiatan adalah " proses perencanaan, pelaksanaan, pengawasan, dan evaluasi berbagai aktivitas atau program yang dilakukan oleh sebuah organisasi untuk mencapai tujuan yang telah ditetapkan."

b. Tahapan Pengelolaan Kegiatan

Pengelolaan kegiatan dapat dibagi menjadi beberapa tahapan utama, antara lain:

1. **Perencanaan (*Planning*):** Tahap pertama dalam pengelolaan kegiatan yang melibatkan identifikasi tujuan yang ingin dicapai,

perumusan strategi untuk mencapainya, dan pengorganisasian sumber daya yang diperlukan. Dalam konteks HIMAPL, perencanaan dapat mencakup pengaturan agenda kegiatan, pemilihan tema, pembagian tugas, serta alokasi waktu.

2. **Pengorganisasian (Organizing):** Pada tahap ini, sumber daya, baik manusia, material, maupun informasi, diorganisir sedemikian rupa agar dapat digunakan dengan maksimal untuk menjalankan kegiatan. Pengorganisasian ini mencakup pembagian tugas, penentuan struktur organisasi kegiatan, dan pembuatan jadwal.
3. **Pelaksanaan (Execution):** Pelaksanaan adalah tahap dimana kegiatan yang telah direncanakan dilaksanakan. Pada tahap ini, pengurus HIMAPL bersama dengan anggota akan melaksanakan tugas dan tanggung jawab sesuai dengan pembagian yang telah dibuat sebelumnya.
4. **Pengawasan (Monitoring and Controlling):** Pengawasan dilakukan untuk memastikan bahwa kegiatan berjalan sesuai dengan rencana. Jika ditemukan penyimpangan, pengawasan bertujuan untuk segera melakukan perbaikan agar kegiatan tetap berjalan dengan baik. Pengawasan ini dilakukan dengan memantau pelaksanaan kegiatan dan mengevaluasi kinerja.
5. **Evaluasi (Evaluation):** Setelah kegiatan selesai, evaluasi dilakukan untuk menilai sejauh mana kegiatan tersebut mencapai tujuan yang

ditetapkan. Evaluasi ini akan memberikan umpan balik untuk perbaikan kegiatan di masa yang akan datang.

c. **Model Pengelolaan Kegiatan dalam Organisasi**

Model pengelolaan kegiatan yang sering digunakan dalam organisasi adalah **Model Pengelolaan Kegiatan Berbasis Tujuan** (Management by Objectives - MBO), yang mengutamakan penetapan tujuan yang jelas dan pengukuran hasil. Model ini menekankan pada pengelolaan kegiatan secara terukur dan berdasarkan pencapaian tujuan.

MBO memiliki beberapa langkah utama:

1. **Penetapan Tujuan:** Tujuan yang jelas dan terukur harus ditetapkan di awal.
2. **Perencanaan Strategis:** Strategi untuk mencapai tujuan tersebut harus dirumuskan.
3. **Pelaksanaan:** Kegiatan dilakukan sesuai dengan strategi yang telah ditetapkan.
4. **Evaluasi dan Umpan Balik:** Setelah pelaksanaan, hasil dievaluasi dan umpan balik diberikan untuk meningkatkan kinerja di masa depan.

d. **Keuntungan Pengelolaan Kegiatan yang Efektif**

Pengelolaan kegiatan yang baik memberikan banyak keuntungan, di antaranya:

1. **Efisiensi:** Memastikan bahwa sumber daya yang ada digunakan secara optimal, sehingga kegiatan dapat diselesaikan dengan biaya dan waktu yang efisien.
2. **Peningkatan Produktivitas:** Dengan pengelolaan yang baik, kegiatan dapat berjalan lebih lancar, meningkatkan produktivitas anggota.
3. **Peningkatan Kinerja Organisasi:** Pengelolaan yang efektif membantu organisasi mencapai tujuannya dengan lebih cepat dan tepat.

e. Tantangan Pengelolaan Kegiatan

Meskipun pengelolaan kegiatan memiliki banyak keuntungan, terdapat beberapa tantangan yang dihadapi oleh organisasi, seperti:

1. **Koordinasi yang Sulit:** Dengan banyaknya anggota dan divisi dalam organisasi, koordinasi kegiatan bisa menjadi sulit jika tidak ada sistem yang jelas.
2. **Keterbatasan Sumber Daya:** Pengelolaan kegiatan yang melibatkan banyak sumber daya (waktu, tenaga, biaya) memerlukan perencanaan yang matang agar tidak terjadi pemborosan.
3. **Ketidakpastian:** Perubahan yang tidak terduga, seperti perubahan jadwal atau pembatalan kegiatan, dapat mempengaruhi jalannya kegiatan.

2.2.12 Teori Usability dan User Experience

a. Teori Usability

Usability adalah konsep yang merujuk pada seberapa mudah, efisien, dan menyenangkan suatu produk atau sistem dapat digunakan oleh pengguna dalam konteks tujuan tertentu. Dalam teori *Usability*, ada beberapa aspek yang menjadi fokus utama, yaitu efektivitas, efisiensi, dan kepuasan pengguna.

b. Definisi Usability

Menurut (ISO 9241-11, 2018), *Usability* didefinisikan sebagai "sejauh mana produk dapat digunakan oleh pengguna tertentu untuk mencapai tujuan tertentu dengan efektif, efisien, dan memuaskan dalam konteks penggunaan tertentu." Hal ini menekankan pada kemampuan pengguna untuk mencapai hasil yang diinginkan dengan sedikit usaha, waktu, dan frustrasi.

Aspek-Aspek *Usability*:

1. **Efektivitas:** Seberapa baik produk memenuhi tujuan pengguna.
Dalam konteks ini, efektivitas diukur berdasarkan kemampuan pengguna dalam menyelesaikan tugas yang diinginkan.
2. **Efisiensi:** Sejauh mana pengguna dapat menyelesaikan tugas dengan waktu dan usaha yang minimal.
3. **Kepuasan:** Sejauh mana pengguna merasa puas dengan pengalaman menggunakan produk, yang mencakup kenyamanan dan kenyamanan selama penggunaan.

Model *Usability*:

1. **Model *Usability* Jakob Nielsen:** Nielsen mengemukakan 10 heuristik untuk desain antarmuka pengguna yang digunakan untuk mengevaluasi dan memperbaiki *Usability* suatu sistem.
2. **Model Pusat-Pengguna (*User-Centered Design*):** Pendekatan ini menekankan pada keterlibatan pengguna sepanjang siklus pengembangan produk untuk memastikan bahwa produk tersebut mudah digunakan dan memenuhi kebutuhan pengguna.

c. **Teori *User Experience* (UX)**

User Experience (UX) merujuk pada keseluruhan pengalaman pengguna dalam berinteraksi dengan produk, layanan, atau sistem, yang mencakup aspek fungsional, estetika, emosional, dan psikologis dari interaksi tersebut.

d. **Definisi *User Experience***

Menurut (ISO 9241-210, 2019), UX didefinisikan sebagai "persepsi dan respons pengguna yang muncul sebagai akibat dari penggunaan atau antisipasi terhadap produk, sistem atau layanan." UX lebih luas daripada *Usability* karena mencakup aspek emosional dan subjektif dari pengalaman pengguna.

UX mencakup beberapa dimensi penting, antara lain:

1. **Kegunaan:** Dimensi yang berkaitan dengan kemudahan penggunaan produk yang juga menjadi bagian dari teori *Usability*.

2. **Estetika:** Aspek visual dan desain produk yang memberikan pengalaman yang menyenangkan atau menarik bagi pengguna.
3. **Emosi dan Kepuasan:** *UX* tidak hanya berfokus pada efisiensi tetapi juga pada pengalaman emosional yang dirasakan pengguna. Sebuah produk yang baik dapat memberikan perasaan positif yang mendorong pengguna untuk terus menggunakannya.
4. **Konten dan Relevansi:** *UX* melibatkan juga kualitas informasi dan relevansi konten yang disediakan oleh sistem untuk pengguna.
5. **Konteks Penggunaan:** Faktor lingkungan dan situasi di mana interaksi berlangsung juga mempengaruhi pengalaman pengguna.

Model dan teori *UX*:

1. **Model *Experience* (Pine dan Gilmore):** Model ini membedakan pengalaman menjadi empat kategori: hiburan, edukasi, estetika, dan *escAPIsme*. Setiap kategori memberikan jenis pengalaman berbeda kepada pengguna.
2. **Model *UX* oleh Garrett:** Pada tahun 2003, Jesse Garrett mengembangkan model yang membagi *UX* menjadi lima lapisan, yaitu: Strategi, Cakupan, Struktur, *Skelton*, dan Permukaan. Model ini menunjukkan bagaimana *UX* melibatkan berbagai aspek yang saling terkait, mulai dari perencanaan strategis hingga antarmuka visual yang ditampilkan kepada pengguna.

e. Hubungan Antara *Usability* dan *UX*

Usability adalah salah satu komponen utama dari *UX*. Produk dengan tingkat *Usability* yang tinggi dapat mendukung pengalaman pengguna yang positif, tetapi pengalaman pengguna yang memuaskan juga bergantung pada faktor lain seperti estetika, konten, dan konteks penggunaan. Dengan kata lain, meskipun sebuah produk memiliki tingkat *Usability* yang baik, tanpa perhatian terhadap aspek lain dari *UX*, pengalaman pengguna secara keseluruhan mungkin tidak optimal.

2.2.13 *Unified Modeling Language (UML)*

Unified Modeling Language (UML) adalah bahasa visual yang digunakan untuk menggambarkan, merancang, dan mendokumentasikan sistem perangkat lunak. *UML* menyediakan serangkaian diagram yang dapat menggambarkan berbagai aspek dari sistem, mulai dari struktur statis hingga perilaku dinamis, serta interaksi antar komponen. *UML* digunakan oleh pengembang perangkat lunak untuk merancang dan memahami sistem yang kompleks (Popkin Software and Systems, 1998).

a. Definisi *UML*

UML pertama kali diperkenalkan pada tahun 1997 oleh **Grady Booch**, **Ivar Jacobson**, dan **James Rumbaugh**, yang dikenal sebagai tiga "bapak" *UML*. *UML* adalah bahasa pemodelan yang bersifat **general-purpose**, yang artinya dapat digunakan untuk berbagai jenis sistem perangkat lunak, baik

sistem kecil maupun besar, serta sistem berbasis *desktop*, *web*, maupun *mobile*.

UML terdiri dari berbagai jenis diagram yang dapat digunakan untuk merancang dan meModelkan sistem perangkat lunak. Diagram-diagram ini dibagi menjadi dua kategori utama:

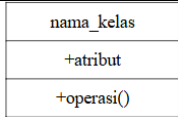
1. **Struktural Diagram:** Menunjukkan struktur statis dari sistem.
2. **Behavioral Diagram:** Menunjukkan interaksi dan perilaku dinamis antar komponen sistem.

b. Jenis-Jenis Diagram *UML*

Berikut adalah beberapa jenis diagram yang umum digunakan dalam *UML*:

1. **Diagram Kelas (Class Diagram):** Diagram yang menggambarkan struktur statis dari sistem dengan menunjukkan kelas-kelas, atribut, metode, dan hubungan antar kelas.

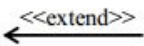
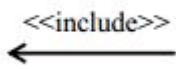
Tabel 2.2 *Class Diagram*

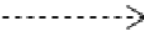
Gambar	Nama	Keterangan
	<i>Class</i>	Himpunan objek-objek dari berbagai atribut yang memiliki operasi yang sama
	<i>Association</i>	Relasi antar kelas dengan makna umum

		dan biasanya disertai <i>multiplicity</i> .
	<i>Directed Association</i>	Relasi antar kelas dengan makna kelas yang satu digunakan oleh kelas lain.
	<i>Aggregation</i>	Mengindikasikan keseluruhan bagian <i>relationship</i> disebut sebagai relasi
	<i>Generalization</i>	Relasi antar kelas dengan makna generalisasi-spesialisasi (umum khusus).
	<i>Dependency</i>	Menunjukkan operasi pada suatu <i>Class</i> yang menggunakan <i>Class</i> yang lain

2. **Diagram Use Case (Use Case Diagram):** Menunjukkan interaksi antara pengguna (aktor) dan sistem untuk mencapai tujuan tertentu. *Diagram* ini digunakan untuk menggambarkan fungsi-fungsi utama yang harus dimiliki sistem.

Tabel 2.3 Tabel *Use Case Diagram*

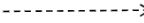
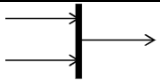
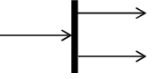
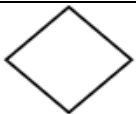
Gambar	Nama	Keterangan
	<i>Actor</i>	Pengguna atau sesuatu yang melakukan interaksi dengan sistem
	<i>Use Case</i>	Aktivitas atau perilaku dari suatu sistem secara garis besar
	<i>Association</i>	Jalur komunikasi antara actor dan <i>UseCase</i> .
	<i>Extend</i>	Perluasan perilaku dari suatu <i>UseCase</i>
	<i>Generalization</i>	Hubungan antara <i>UseCase</i> umum dan <i>UseCase</i> yang lebih yang mewarisi atau menambah fitur.
	<i>Include</i>	<i>UseCase</i> yang selalu disertakan ke dalam <i>UseCase</i> dasar.
	<i>System</i>	Representasi dari ruang lingkup sistem.

	<i>Collaboration</i>	Interaksi antar elemen yang bekerja sama menghasilkan suatu perilaku
	<i>Note</i>	Catatan yang memberikan informasi tambahan.
	<i>Dependency</i>	Hubungan di mana perubahan pada satu elemen akan memengaruhi elemen lain yang bergantung padanya.

3. **Diagram Aktivitas (Activity Diagram):** Menggambarkan alur kerja atau proses dalam sistem, termasuk aliran kontrol dan keputusan yang diambil selama pelaksanaan.

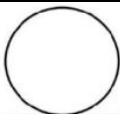
Tabel 2.4 Tabel *Activity Diagram*

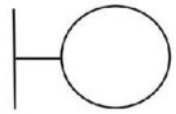
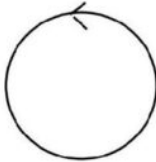
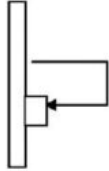
Gambar	Nama	Keterangan
	<i>Start Point</i>	Titik awal proses dari sebuah aktivitas.
	<i>End Point</i>	Titik akhir proses dari sebuah aktivitas.
	<i>Activity</i>	Tindakan atau langkah dalam sebuah proses.

	<i>Control Flow</i>	Menunjukkan urutan eksekusi antar aktivitas.
	<i>Object Flow</i>	Aliran objek dari satu <i>Activity/action</i> ke <i>Activity/action</i> lain.
	<i>Join</i>	Menggabungkan beberapa aktivitas menjadi satu aktivitas.
	<i>Fork</i>	Memecah satu aktivitas menjadi beberapa aktivitas
	<i>Decision</i>	Menggambarkan suatu keputusan/tindakan yang harus di ambil pada kondisi tertentu.

4. **Diagram Urutan (*Sequence Diagram*):** Menunjukkan interaksi antar objek dalam urutan waktu, menggambarkan bagaimana objek saling berkomunikasi dalam menjalankan fungsi tertentu.

Tabel 2.5 Tabel *Sequence Diagram*

Gambar	Nama	Keterangan
	<i>Entity Class</i>	Representasi elemen data yang menjadi dasar penyusunan basis data.

	<i>Boundary Class</i>	Mengelola interaksi antara sistem dengan lingkungan luar.
	<i>Control Class</i>	Mengatur logika proses dan mengoordinasikan aliran pesan.
	<i>Recursive</i>	Pesan yang dikirim objek kepada dirinya sendiri
	<i>Activation</i>	Durasi ketika suatu operasi sedang dijalankan.
	<i>Life Line</i>	Garis vertikal putus-putus yang menunjukkan keberadaan objek selama interaksi.

5. **Diagram Komponen (Component Diagram):** Menunjukkan struktur fisik dari sistem dan bagaimana berbagai komponen perangkat lunak saling berhubungan.
6. **Diagram Status (State Diagram):** Menggambarkan status atau keadaan yang dapat dialami oleh objek dalam sistem dan transisi antar status tersebut.

c. Manfaat Penggunaan *UML* dalam Pengembangan Sistem

1. **Visualisasi Sistem:** *UML* memungkinkan pengembang untuk menggambarkan sistem secara visual, yang memudahkan pemahaman struktur dan perilaku sistem, serta interaksi antar komponen.
2. **Standarisasi:** *UML* adalah bahasa yang diakui secara internasional dan digunakan oleh banyak pengembang perangkat lunak. Penggunaan *UML* memastikan bahwa *Diagram* dan dokumentasi sistem dapat dipahami oleh berbagai pihak yang terlibat dalam pengembangan sistem.
3. **Kolaborasi Tim:** *UML* memudahkan kolaborasi antar anggota tim pengembang, analis sistem, dan pemangku kepentingan lainnya. *Diagram UML* menyediakan cara yang jelas dan terstruktur untuk berkomunikasi mengenai desain sistem.
4. **Dokumentasi Sistem:** *UML* digunakan untuk mendokumentasikan desain dan arsitektur sistem, yang berguna untuk pemeliharaan sistem di masa mendatang.

d. Tantangan dalam Menggunakan *UML*

Meskipun *UML* memberikan banyak manfaat, penggunaannya juga memiliki beberapa tantangan, antara lain:

1. **Kompleksitas:** *UML* memiliki banyak jenis *Diagram*, dan jika tidak digunakan dengan benar, dapat menjadi sangat kompleks dan sulit dipahami.

2. **Waktu dan Sumber Daya:** Menggunakan *UML* untuk mendesain dan mendokumentasikan sistem memerlukan waktu dan sumber daya yang cukup banyak, terutama untuk sistem yang besar dan kompleks.
3. **Pemahaman Bersama:** Agar *UML* efektif, semua anggota tim harus memiliki pemahaman yang sama mengenai simbol-simbol dan prinsip-prinsip *UML*, yang dapat memerlukan pelatihan dan pengalamannya.

2.2.14 PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional-objek (*Object-Relational Database Management System / ORDBMS*) berbasis sumber terbuka (*open-source*) yang diakui memiliki tingkat keandalan, integritas data, serta ketepatan arsitektur yang sangat tinggi. Sebagai salah satu jenis basis data relasional, *PostgreSQL* beroperasi dengan menyimpan dan menyusun data ke dalam format tabel (baris dan kolom) yang saling berhubungan satu sama lain. Relasi antar-tabel ini memungkinkan pengelolaan data yang tersebar menjadi jauh lebih efisien, terstruktur, dan konsisten.

Sistem ini mendukung kepatuhan penuh terhadap prinsip transaksi *ACID* (*Atomicity, Consistency, Isolation, Durability*). Prinsip ini memastikan bahwa setiap pertukaran dan mutasi data di dalam aplikasi diproses secara aman, serta

terlindungi dari risiko korupsi atau kehilangan data apabila terjadi kegagalan sistem maupun pemutusan koneksi secara tiba-tiba (Microsoft, n.d.).

a. Peran *PostgreSQL* dalam Arsitektur Sistem

Dalam arsitektur pengembangan perangkat lunak modern, *PostgreSQL* dapat dikonfigurasi sebagai peladen basis data mandiri maupun bertindak sebagai mesin basis data (*database engine*) utama yang terintegrasi di dalam layanan komputasi awan (seperti *platform Backend-as-a-Service*).

Seluruh rancangan skema relasional yang menyokong alur operasional sebuah sistem mulai dari data pengguna, struktur organisasi, hingga pencatatan transaksi data yang kompleks disimpan dan dikelola langsung di dalam *PostgreSQL*. Kinerja basis data ini sangat esensial dalam memastikan informasi dapat diakses (*Read*), ditambah (*Create*), diperbarui (*Update*), dan dihapus (*Delete*) dengan akurasi dan kecepatan yang tinggi untuk melayani berbagai permintaan dari aplikasi klien (*frontend*).

b. Pemanfaatan Fitur Lanjutan *PostgreSQL*

Pemanfaatan *PostgreSQL* dalam sebuah sistem berskala menengah hingga besar sering kali melampaui fungsi penyimpanan data standar. Pengembang dapat mengeksplorasi kapabilitas komputasi tingkat lanjut dari *PostgreSQL* untuk menjalankan logika bisnis langsung di sisi peladen (server-side), yang meliputi:

1. *Database Triggers: Trigger* adalah sekumpulan instruksi atau fungsi prosedural di dalam basis data yang secara otomatis dieksekusi sebagai respons terhadap peristiwa tertentu (seperti *Insert*, *Update*, atau *Delete*) pada suatu tabel. Dalam arsitektur sistem informasi, *database trigger* sering digunakan untuk mengotomatisasi proses bisnis tanpa membebani aplikasi klien, seperti menghasilkan rekaman riwayat aktivitas (*log*), memperbarui perhitungan stok/kuota secara otomatis, atau memicu pembuatan baris data notifikasi baru ketika sebuah transaksi berhasil dilakukan.
2. *Row Level Security (RLS)*: *RLS* merupakan mekanisme keamanan akses tingkat lanjut yang beroperasi langsung di dalam lapisan mesin *PostgreSQL*. Konfigurasi aturan keamanan *RLS* bertugas membatasi otorisasi pertukaran data secara ketat pada tingkat baris (*row*) berdasarkan identitas atau peran (*role*) pengguna yang sedang mengakses sistem. Melalui evaluasi kebijakan ini, basis data akan secara otomatis memblokir kueri pengubahan atau pembacaan data apabila pengguna tidak memiliki otorisasi yang sah, sehingga memberikan perlindungan mutlak terhadap upaya manipulasi atau kebocoran data dari pihak yang tidak berwenang.

2.2.15 Black Box Testing

Black Box testing merupakan strategi pengujian perangkat lunak yang memandang program sebagai “kotak hitam”, sehingga penguji berfokus pada

pemeriksaan perilaku sistem melalui hubungan *input* dan *output* tanpa memperhatikan struktur internal atau logika kode di dalamnya. Dalam strategi ini, penguji berupaya memvalidasi apakah sistem telah berperilaku sesuai spesifikasi atau kebutuhan yang ditetapkan, sementara detail implementasi dianggap tidak relevan pada saat pengujian (Myers et al., 2011).

ISTQB(International Software Testing Qualifications Board) menjelaskan bahwa teknik *Black-Box* (juga dikenal sebagai teknik specification-based) didasarkan pada analisis perilaku yang dispesifikasikan dari test object tanpa merujuk pada struktur internalnya sehingga test *Case* yang dihasilkan bersifat independen terhadap cara perangkat lunak diimplementasikan. Konsekuensinya, apabila implementasi berubah namun perilaku yang dipersyaratkan tetap sama, maka test *Case* tersebut umumnya tetap relevan untuk digunakan (pwg & istqborg, 2024). Sejalan dengan itu, standar *ISO/IEC/IEEE* mendefinisikan test *Case* sebagai sekumpulan prasyarat, *input* (termasuk aksi, jika diperlukan), serta hasil yang diharapkan (*expected results*) yang disusun untuk menentukan apakah bagian yang diuji telah diimplementasikan dengan benar (Computer Soccity, 2015).

Dalam praktiknya, *Black Box* testing sering menggunakan teknik desain pengujian berbasis spesifikasi, seperti equivalence partitioning, boundary value analysis, decision table testing, dan state transition testing (pwg & istqborg, 2024). Teknik-teknik ini membantu penyusunan test *Case* secara lebih sistematis, misalnya dengan membagi data uji ke dalam partisi yang

diasumsikan diproses dengan cara yang sama (*equivalence partitioning*), menekankan pengujian pada nilai batas (*boundary value analysis*), mengevaluasi kombinasi kondisi–aksi secara terstruktur (*decision table testing*), serta memodelkan perubahan keadaan sistem (*state transition testing*) (pwg & istqborg, 2024). Selain itu, literatur klasik juga menekankan bahwa pengujian secara menyeluruh (*exhaustive testing*) umumnya tidak realistis, sehingga pendekatan yang wajar adalah merancang pengujian berbasis *Black Box* dan melengkapinya bila perlu dengan pendekatan lain (Myers et al., 2011).

2.2.16 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) merupakan fase krusial dan tahap akhir dalam siklus hidup pengujian perangkat lunak (*Software Testing Life Cycle*) sebelum sebuah sistem dirilis secara resmi ke lingkungan produksi. Berbeda dengan pengujian teknis (*System Testing* atau *Black-Box Testing*) yang dilakukan oleh pengembang untuk mencari kecacatan kode (*bug*), *UAT* dilakukan secara langsung oleh pengguna akhir (*end-user*) atau klien. Tujuan utama dari pengujian ini bukan lagi untuk menguji arsitektur sistem, melainkan untuk memvalidasi apakah perangkat lunak yang dibangun telah benar-benar selaras dengan spesifikasi kebutuhan bisnis, operasional, dan alur kerja pengguna di dunia nyata (Binus University Bekasi, n.d.).

a. Tujuan dan manfaat *UAT*

Pelaksanaan *UAT* memberikan beberapa manfaat fundamental bagi pengembangan perangkat lunak, di antaranya:

1. Validasi Kebutuhan Pengguna: Memastikan bahwa fungsionalitas yang telah dikembangkan benar-benar mampu memecahkan masalah pengguna dan mendukung aktivitas mereka sehari-hari.
2. Minimalisasi Risiko Rilis: Mengidentifikasi kekurangan atau ketidaksesuaian antarmuka dan alur kerja yang mungkin terlewat oleh tim pengembang sebelum sistem didistribusikan secara massal.
3. Pernyataan Penerimaan (*Acceptance*): Memberikan konfirmasi formal berupa umpan balik langsung dari pengguna bahwa sistem telah diuji, dipahami, dan dinyatakan layak (*accepted*) untuk diimplementasikan secara operasional.

b. Parameter dan Aspek Pengujian

Dalam pelaksanaan *UAT*, pengujian tidak dilakukan melalui skrip pengkodean, melainkan melalui skenario penggunaan langsung (*hands-on*). Parameter penilaian umumnya difokuskan pada observasi terhadap interaksi pengguna dengan aplikasi. Berdasarkan standar evaluasi perangkat lunak interaktif, aspek penilaian dalam *UAT* sering kali dikelompokkan ke dalam tiga indikator utama:

1. Desain Antarmuka (*User Interface*): Mengevaluasi tata letak visual, tipografi, kombinasi warna, dan kejelasan elemen grafis aplikasi agar selaras dengan prinsip estetika dan standar aksesibilitas.
2. Kemudahan Penggunaan (*User Experience*): Menilai seberapa intuitif alur navigasi aplikasi, tingkat responsivitas sistem, dan

kenyamanan psikologis pengguna saat menjalankan tugas tertentu di dalam aplikasi.

3. Manfaat Fungsionalitas: Mengukur kesesuaian fitur-fitur yang tersedia dengan kebutuhan praktis pengguna, serta seberapa efektif fitur tersebut dalam membantu penyelesaian tugas operasional organisasi.

c. Metode Pengukuran Berbasis Kuesioner dan Skala Likert

Untuk menghasilkan kesimpulan yang objektif dan terukur dari pengujian UAT, evaluasi kualitatif dari pengalaman pengguna perlu dikonversi menjadi data kuantitatif. Metode yang paling umum dan efektif digunakan untuk tujuan ini adalah pendistribusian kuesioner evaluasi yang mengadopsi instrumen Skala Likert.

Skala Likert adalah skala psikometrik yang digunakan untuk mengukur sikap, persepsi, tingkat persetujuan, atau kepuasan responden terhadap sebuah subjek. Melalui metode ini, pengguna yang telah mengoperasikan aplikasi akan diminta untuk merespons serangkaian pernyataan tertulis dengan memilih salah satu dari beberapa tingkatan persetujuan (contoh: Sangat Tidak Setuju, Tidak Setuju, Netral, Setuju, Sangat Setuju). Hasil dari pengisian kuesioner ini kemudian diakumulasikan dan dihitung menggunakan persentase untuk menentukan skor akhir tingkat penerimaan pengguna (*User Acceptance Rate*) terhadap sistem yang dibangun.

BAB III

METODE PENELITIAN

3.1 Gambaran Umum Objek

Objek penelitian dalam skripsi ini adalah organisasi kemahasiswaan Himpunan Mahasiswa Teknik Perangkat Lunak (HIMAPL) pada Fakultas Komputer Universitas Universal. HIMAPL merupakan organisasi mahasiswa yang bergerak di bidang akademik, pengembangan keilmuan, dan kegiatan kemahasiswaan yang berfokus pada bidang komputer dan teknologi informasi. Organisasi ini berfungsi sebagai wadah pengembangan kompetensi mahasiswa dalam bidang keilmuan serta pembentukan karakter melalui berbagai kegiatan internal maupun eksternal.

Struktur organisasi HIMAPL terdiri dari pengurus inti, beberapa divisi atau bidang kerja, serta anggota aktif. Setiap divisi memiliki tugas dan tanggung jawab yang berbeda, seperti bidang akademik, hubungan masyarakat, kewirausahaan, dan kegiatan sosial. Kegiatan HIMAPL umumnya meliputi rapat rutin, seminar, pelatihan, kegiatan sosial, serta program kerja tahunan yang melibatkan mahasiswa secara luas.

Dalam menjalankan aktivitas organisasi, HIMAPL memerlukan sistem komunikasi dan pengelolaan *Data* yang terstruktur agar seluruh anggota dapat memperoleh informasi secara merata, tepat waktu, dan terdokumentasi dengan baik. Saat ini, proses penyampaian informasi dan *penDataan* masih dilakukan secara manual menggunakan media komunikasi umum seperti aplikasi pesan instan dan dokumen terpisah, sehingga berpotensi menimbulkan kendala dalam hal

pengelolaan kegiatan, pencatatan keanggotaan, serta pencarian dokumentasi kegiatan yang telah dilaksanakan.

Kondisi tersebut menjadikan HIMAPL sebagai objek penelitian yang relevan untuk dikaji dalam perancangan sistem informasi berbasis *mobile*. Organisasi ini membutuhkan sistem yang mampu mendukung aktivitas administrasi, komunikasi internal, dokumentasi, serta pengelolaan kegiatan organisasi secara terintegrasi. Melalui penelitian ini, diharapkan dapat dirancang sebuah sistem informasi yang sesuai dengan kebutuhan operasional organisasi mahasiswa di lingkungan Fakultas Komputer Universitas Universal.

3.2 Metode Penelitian

Penelitian ini menggunakan metode kualitatif, karena fokus penelitian adalah menggali kebutuhan pengguna secara mendalam, memahami permasalahan yang terjadi dalam proses penyebaran informasi di HIMAPL, serta merancang solusi sistem informasi yang sesuai dengan kondisi nyata di lapangan. Metode kualitatif dipilih karena mampu memberikan pemahaman yang komprehensif mengenai pengalaman, persepsi, dan kebutuhan pengguna yang tidak dapat dijelaskan hanya melalui *Data* numerik. Hasil analisis kualitatif kemudian menjadi dasar dalam merancang fitur aplikasi yang relevan dan tepat sasaran.

Metode penelitian yang digunakan dalam pengembangan sistem informasi pada penelitian ini adalah metode *Agile Sprint*. *Agile Sprint* merupakan pendekatan pengembangan perangkat lunak yang bersifat iteratif dan inkremental, di mana pengembangan sistem dilakukan dalam siklus waktu singkat yang disebut *Sprint*.

Setiap *Sprint* menghasilkan bagian sistem yang dapat diuji dan dikembangkan lebih lanjut berdasarkan hasil evaluasi dan umpan balik pengguna.



Gambar 3.1 *Agile Sprint*

Sumber: <https://www.wrike.com/scrum-guide/scrum-Sprints/>

Pendekatan ini dipilih karena mampu mengakomodasi perubahan kebutuhan pengguna, mempercepat proses pengembangan, serta memungkinkan perbaikan berkelanjutan selama proses penelitian berlangsung. *Agile Sprint* juga menekankan kolaborasi antara peneliti dan pengguna, sehingga sistem yang dikembangkan lebih sesuai dengan kebutuhan sebenarnya. Tahapan penelitian yang dilakukan sebagai berikut:

3.2.1 *Product Backlog*

Pada tahap ini, dilakukan identifikasi dan penyusunan daftar fitur dan kebutuhan yang diinginkan dalam aplikasi HIMAPL. Daftar ini mencakup semua fitur yang diinginkan oleh pengurus dan anggota HIMAPL, yang

mencakup pengelolaan *Data* anggota, pengumuman, manajemen kegiatan, serta dokumentasi kegiatan. Setiap fitur diberi prioritas berdasarkan urgensi dan relevansinya terhadap tujuan aplikasi.

3.2.2 *Sprint Backlog*

Setelah menentukan *Product Backlog*, langkah selanjutnya adalah memilih item-item dari *Backlog* yang akan dikerjakan dalam satu *Sprint*. Setiap item akan dijabarkan lebih rinci menjadi tugas-tugas yang lebih kecil (task) yang dapat diselesaikan dalam waktu singkat, biasanya satu sampai dua minggu. *Sprint Backlog* mencakup pekerjaan teknis yang harus diselesaikan untuk mencapai fitur yang diinginkan.

3.2.3 *The Sprint*

Tahap *Sprint* mencakup beberapa fase berikut:

- a. *Planning*: Di awal *Sprint*, dilakukan perencanaan untuk menentukan tugas-tugas yang akan dikerjakan berdasarkan *Sprint Backlog* yang telah disusun. Tim pengembang dan pengguna (pengurus HIMAPL) akan sepakat mengenai tujuan *Sprint* ini.
- b. *Implementation*: Fase implementasi atau pengembangan dimulai dengan menulis kode, membangun antarmuka pengguna, dan menyelesaikan tugas-tugas yang telah ditentukan di *Sprint Backlog*.
- c. *Daily scrum*: Setiap hari, tim pengembang melakukan pertemuan singkat (*daily scrum*) untuk membahas kemajuan, hambatan yang dihadapi, dan solusi yang diperlukan untuk menyelesaikan tugas.

- d. *Review*: Setelah *Sprint* selesai, dilakukan evaluasi terhadap hasil *Sprint*, baik dari segi fungsionalitas fitur yang dikembangkan maupun pencapaian tujuan *Sprint*. Pengguna (pengurus HIMAPL) memberikan umpan balik yang digunakan untuk penyesuaian di *Sprint* berikutnya.
- e. *Retrospective*: Setelah evaluasi, tim pengembang melakukan refleksi terhadap proses yang telah dilalui selama *Sprint* untuk meningkatkan kualitas dan efisiensi kerja di *Sprint* berikutnya.

3.2.4 Completed Product

Setelah seluruh *Sprint* selesai dan fitur yang ditargetkan dalam *Sprint* telah berhasil dikembangkan dan diuji, maka produk akhir yang telah tercapai dalam *Sprint* ini dinyatakan sebagai *Completed Product*. Produk ini adalah versi aplikasi HIMAPL yang dapat digunakan atau diuji lebih lanjut oleh pengguna. Aplikasi ini terus diperbaiki dan ditingkatkan pada setiap iterasi *Sprint* berikutnya berdasarkan umpan balik dari pengguna.

3.3 Pengumpulan Data

Pengumpulan *Data* dalam penelitian ini dilakukan dengan cara sebagai berikut:

1. Wawancara: dilakukan kepada pengurus HIMAPL untuk menggali kebutuhan sistem, kendala yang sering dialami, proses penyebaran informasi, serta alur kerja dalam pengelolaan kegiatan dan administrasi organisasi. Wawancara bersifat semi-terstruktur agar *Data* yang diperoleh lebih fleksibel dan mendalam.

2. Observasi: dilakukan terhadap media komunikasi yang saat ini digunakan oleh HIMAPL, seperti *WhatsApp* dan dokumen internal lain yang berkaitan dengan *Data* anggota dan kegiatan organisasi. Observasi ini bertujuan untuk memahami secara langsung bagaimana informasi dikelola dan disebarkan.
3. Studi Dokumentasi: Peneliti mempelajari dokumen-dokumen terkait, seperti struktur organisasi, daftar anggota, dokumentasi kegiatan sebelumnya, serta aturan internal HIMAPL. Dokumen ini digunakan untuk mendapatkan gambaran mengenai *Data* dan informasi yang harus disediakan dalam aplikasi.

3.4 Pengolahan *Data*

Data yang terkumpul akan dianalisis menggunakan pendekatan kualitatif untuk merancang aplikasi Sistem Informasi HIMAPL yang sesuai dengan kebutuhan pengguna di Fakultas Komputer Universitas Universal. *Data* yang diperoleh dari wawancara, observasi, dan studi dokumentasi akan dianalisis untuk memahami:

1. Permasalahan utama yang dihadapi dalam penyebaran informasi dan pengelolaan administrasi HIMAPL, seperti pesan yang tertimbun di *WhatsApp*, kesulitan mencari dokumentasi kegiatan, serta pendataan anggota yang masih dilakukan secara manual.
2. Fitur-fitur penting yang diperlukan dalam aplikasi, termasuk pengumuman, notifikasi, pengelolaan kegiatan, *Data* anggota, dokumentasi kegiatan, agenda, serta pembagian tugas.

3. Prioritas kebutuhan pengguna, baik dari sisi pengurus maupun anggota HIMAPL, seperti kemudahan akses, kecepatan penyampaian informasi, serta kerAPIan pendataan organisasi.

Hasil analisis dari ketiga aspek tersebut akan digunakan sebagai dasar dalam menyusun kebutuhan sistem, merancang antarmuka pengguna, serta menentukan alur kerja aplikasi. *Data* yang telah dianalisis kemudian menjadi landasan utama dalam proses pengembangan sistem agar aplikasi yang dibangun dapat mengatasi permasalahan yang telah diidentifikasi dan meningkatkan efektivitas operasional HIMAPL. Tahap pengolahan data adalah sebagai berikut.

1. Analisis Kebutuhan (Requirements Analysis)

Tahap analisis kebutuhan dilakukan untuk menerjemahkan hasil pengolahan data menjadi spesifikasi kebutuhan sistem yang jelas, terukur, dan dapat diimplementasikan. Kebutuhan ini disusun berdasarkan temuan permasalahan, kebutuhan fitur, serta prioritas pengguna yang sebelumnya telah diidentifikasi melalui wawancara, observasi, dan studi dokumentasi. Dalam penelitian ini, kebutuhan sistem dirumuskan ke dalam dua kategori berikut:

- a. Kebutuhan fungsional, yaitu kebutuhan yang berkaitan dengan fungsi utama aplikasi, misalnya pengelolaan data anggota, pengumuman, notifikasi, manajemen kegiatan, dokumentasi kegiatan, agenda, serta pembagian tugas.

- b. Kebutuhan non-fungsional, yaitu kebutuhan yang berkaitan dengan kualitas sistem, seperti kemudahan akses, kecepatan penyampaian informasi, kerAPIan pendataan, dan aspek kenyamanan penggunaan aplikasi.

Hasil analisis kebutuhan ini kemudian menjadi acuan dalam penyusunan daftar fitur pada *Backlog* pengembangan, sehingga setiap kebutuhan dapat dipetakan menjadi item pekerjaan yang diprioritaskan sesuai urgensi dan relevansinya terhadap tujuan aplikasi.

2. Analisis Hak Akses Pengguna

Analisis hak akses pengguna dilakukan untuk menentukan batasan wewenang setiap peran (*role*) dalam aplikasi, agar pengelolaan informasi dan administrasi organisasi berjalan terkontrol, aman, serta sesuai struktur organisasi. Penentuan hak akses ini juga mengacu pada kebutuhan fitur dan alur kerja yang telah diperoleh dari proses analisis, terutama pada pengelolaan informasi, dokumentasi kegiatan, serta pendataan anggota. Pada aplikasi Sistem Informasi HIMAPL, hak akses dibagi menjadi tiga peran utama berikut:

a. Anggota Organisasi

- Mengakses informasi dan pengumuman organisasi secara terpusat.
- Melihat agenda/kegiatan, notifikasi, serta dokumentasi kegiatan yang dipublikasikan.
- Menambahkan bukti penyelesaian tugas

- Memperbarui status tugas menjadi selesai
- Mengelola data profil dasar anggota (misalnya memperbarui informasi pribadi yang diizinkan).

b. Ketua Organisasi

- Mengelola data anggota (menambah/memperbarui data, validasi keanggotaan).
- Membuat dan mempublikasikan pengumuman serta mengatur notifikasi.
- Mengelola kegiatan (membuat agenda, mengatur detail kegiatan) dan melakukan pembagian tugas kepada anggota/pengurus.
- Meminta persetujuan proposal kegiatan kepada dosen pembina
- Mengelola dokumentasi kegiatan agar tersimpan rapi dan mudah ditelusuri.

c. Dosen Pembina

- Memantau informasi dan aktivitas organisasi melalui akses baca (monitoring) terhadap pengumuman, agenda, dan dokumentasi.
- Mengelola data anggota (menambah/memperbarui data, validasi keanggotaan).
- Membuat dan mempublikasikan pengumuman serta mengatur notifikasi.
- Menyetujui atau menolak proposal kegiatan dari ketua organisasi

- Mengakses ringkasan/rekap informasi yang dibutuhkan untuk pembinaan (misalnya riwayat kegiatan dan dokumentasi yang telah dipublikasikan).

Dengan analisis hak akses ini, setiap fitur yang dikembangkan dapat diatur penggunaannya berdasarkan peran, sehingga alur kerja aplikasi menjadi lebih tertib dan selaras dengan kebutuhan operasional organisasi.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Analisi Kebutuhan Sistem

Analisis kebutuhan dilakukan untuk mengidentifikasi kebutuhan pengguna terhadap aplikasi Sistem Informasi HIMAPL. Kebutuhan sistem diperoleh berdasarkan hasil observasi terhadap media komunikasi organisasi, studi dokumentasi, serta hasil wawancara dengan pengurus HIMAPL dan dosen pembina di Fakultas Komputer Universitas Universal.

Berdasarkan hasil analisis, ditemukan bahwa proses penyebaran informasi dan pengelolaan administrasi organisasi masih melibatkan berbagai media yang terpisah, seperti penggunaan *WhatsApp* untuk penyampaian pesan, *Google Drive* untuk penyimpanan berkas yang tidak terstruktur, serta *Microsoft Excel* untuk pendataan anggota secara manual. Kondisi tersebut menyebabkan berbagai kendala, antara lain tertimbunnya pesan penting, ketidakmerataan penyebaran informasi, sulitnya melacak riwayat kegiatan untuk keperluan arsip, serta inefisiensi dalam pembaruan data anggota.

Untuk mengatasi permasalahan tersebut, dikembangkan aplikasi Sistem Informasi HIMAPL berbasis *mobile* yang mampu mengintegrasikan seluruh operasional organisasi dalam satu platform terpusat. Sistem ini memusatkan berbagai fungsionalitas utama mulai dari pengelolaan data anggota, penyebaran pengumuman dan notifikasi, manajemen kegiatan, pembagian tugas, hingga

dokumentasi secara sistematis agar dapat meningkatkan efisiensi tata kelola HIMAPL.

4.1.1 Analisis Kebutuhan Fungsional

Kebutuhan fungsional adalah spesifikasi layanan, fitur, dan fungsi yang wajib disediakan oleh sebuah sistem

Tabel 4.1 Tabel Kebutuhan Fungsional

Kode	Kebutuhan Fungsional
KF-01	Semua pengguna dapat melakukan proses <i>register</i> , <i>login</i> , dan <i>logout</i> pada sistem.
KF-02	Semua pengguna dapat mengelola data profil secara mandiri.
KF-03	Semua pengguna dapat menentukan <i>role</i> (peran) sebagai anggota, ketua, ketua divisi atau dosen.
KF-04	Semua pengguna dapat melihat detail <i>event</i> , melihat status <i>event</i> (Tersedia, Berlangsung, Riwayat), serta melakukan registrasi ke suatu <i>event</i> .
KF-05	Semua pengguna dapat melihat daftar notifikasi yang ada pada sistem.

KF-06	Semua pengguna dapat melihat struktur organisasi HIMAPL.
KF-07	Semua pengguna dapat melihat daftar tugas yang telah diterima.
KF-08	Semua pengguna dapat mengunggah dan melihat foto dokumentasi <i>event</i> .
KF-09	Semua pengguna dapat melihat pembaruan kabar terbaru terkait organisasi.
KF-10	Ketua dapat mengelola <i>event</i> organisasi.
KF-11	Ketua dapat membuat divisi baru dan mengangkat anggota menjadi ketua divisi.
KF-12	Ketua dapat menentukan dan mendelegasikan tugas pada <i>event</i> kepada semua anggota.
KF-13	Anggota dapat memilih divisi yang ingin diikuti.
KF-14	Ketua divisi dapat menentukan tugas spesifik pada <i>event</i> kepada anggota divisinya masing-masing.
KF-15	Ketua dan ketua divisi berhak mengelola foto dokumentasi <i>event</i> yang diunggah.

KF-16	Ketua dan ketua divisi berhak mempublikasikan kabar terbaru organisasi.
KF-17	Dosen pembina dapat melakukan semua fungsionalitas yang dapat dilakukan oleh pengguna lainnya (hak akses menyeluruh).

4.1.2 Analisis Kebutuhan Non-Fungsional

Tabel 4.2 Tabel Kebutuhan Non-Fungsional

Kode	Karakteristik	Kebutuhan
KNF-01	<i>Usability</i>	Sistem harus memiliki antarmuka yang intuitif dan bersih (<i>clean UI</i>), serta menggunakan pedoman warna yang selaras dengan tema warna Universitas Universal agar memberi kesan yang cocok.
KNF-02	<i>Security</i>	Sistem harus memastikan keamanan data akun dengan menyimpan kata sandi pengguna dalam bentuk terenkripsi di dalam basis data.
KNF-03	<i>Security</i>	Sistem harus menyediakan fitur manajemen sesi (<i>session management</i>) yang memungkinkan perangkat pengguna dikenali (<i>trust this device</i>) selama 30 hari untuk kemudahan akses berulang.

KNF-04	Reliability	Sistem harus dapat menjaga ketersediaan layanan untuk diakses pengguna, dengan jadwal pemeliharaan rutin (<i>maintenance</i>) yang dibatasi pada pukul 02.00 dini hari untuk meminimalisir gangguan operasional organisasi.
KNF-05	Portability	Sistem harus dirancang dan dipastikan dapat diinstal serta berjalan dengan baik secara spesifik pada perangkat <i>mobile</i> dengan sistem operasi <i>Android</i> .
KNF-06	Performance Efficiency	Sistem harus mampu memproses fungsi dasar dan merespons interaksi pengguna di perangkat <i>Android</i> dengan waktu respons yang memadai.

4.2 Product Backlog

Fase inisiasi pada metodologi Agile dimulai dengan mentransformasikan daftar analisis kebutuhan fungsional sistem ke dalam bentuk daftar pekerjaan terstruktur (*Product Backlog*) yang dinarasikan menggunakan format *User Stories*. Untuk menjamin proses pengembangan aplikasi *mobile* HIMAPL berjalan secara sistematis dan bertahap, seluruh item pekerjaan di dalam *Product Backlog* dikelompokkan ke dalam tiga tingkat prioritas pengerjaan (High, Medium, dan Low Priority) berdasarkan tingkat ketergantungan antar-fitur (feature dependency) dan urgensi kebutuhan operasional organisasi.

Tabel 4.3 Tabel *Product Backlog*

ID Backlog	Pernyataan User Story	Tingkat Prioritas	Pengguna (Aktor)	Justifikasi Urgensi Implementasi
PB-01	Sebagai pengguna, saya ingin melakukan proses <i>register</i> , <i>login</i> , dan <i>logout</i> agar dapat mengakses fungsionalitas sistem secara aman.	<i>High Priority</i>	Semua Aktor	Fondasi utama sistem pertahanan keamanan data yang harus diselesaikan pertama kali sebelum modul transaksional lain dibangun.
PB-02	Sebagai pengguna, saya ingin menentukan peran (<i>role</i>) pendaftaran sebagai anggota, ketua, atau dosen agar mendapatkan konfigurasi hak akses yang sesuai.	<i>High Priority</i>	Semua Aktor	Menjadi parameter utama sistem dalam membatasi visibilitas menu dan otorisasi fitur sejak akun pertama kali dibuat.

PB-03	Sebagai Ketua, saya ingin membuat divisi organisasi dan memproses pengangkatan ketua divisi agar struktur kepengurusan aktif terbentuk di dalam sistem.	<i>High Priority</i>	Ketua	Prasyarat utama fungsionalitas manajemen kerja, karena pendelegasian tugas memerlukan entitas divisi yang valid.
PB-04	Sebagai Ketua, saya ingin melakukan pengelolaan data <i>event</i> (tambah, ubah, hapus) agar agenda kegiatan organisasi dapat terjadwal terpusat.	<i>High Priority</i>	Ketua	Komponen transaksional induk organisasi yang menjadi objek utama dari pendelegasian tugas dan dokumentasi kegiatan.

PB-05	Sebagai Ketua atau Ketua Divisi, saya ingin mendelegasikan tugas spesifik pada <i>event</i> kepada anggota agar pembagian kerja organisasi menjadi transparan.	<i>High Priority</i>	Ketua, Ketua Divisi	Fitur operasional krusial untuk mengatasi masalah ketidakteraturan koordinasi kerja pengurus selama pelaksanaan kegiatan.
PB-06	Sebagai pengguna, saya ingin mengakses daftar tugas individual yang diterima agar mengetahui instruksi kerja kerja secara <i>real-time</i> .	<i>High Priority</i>	Semua Aktor	Berfungsi sebagai lembar kendali bagi pengurus untuk memantau tanggung jawab tugas yang harus diselesaikan.
PB-07	Sebagai Ketua atau Ketua Divisi, saya ingin mempublikasikan	<i>High Priority</i>	Ketua, Ketua Divisi	Solusi langsung untuk mengatasi kelemahan sistem berjalan, di mana pengumuman organisasi

	kabar terbaru organisasi agar pengumuman penting dapat diakses secara terpusat.			sering kali tertimbun di <i>WhatsApp</i>.
PB-08	Sebagai pengguna, saya ingin membaca halaman kabar terbaru, melihat visualisasi struktur organisasi, serta mengakses detail jadwal <i>event</i>.	<i>High Priority</i>	Semua Aktor	Modul diseminasi informasi dasar yang wajib tersedia agar aplikasi dapat berfungsi sebagai pusat informasi HIMAPL.
PB-09	Sebagai pengguna, saya ingin melakukan registrasi kepesertaan pada <i>event</i> aktif serta mengakses daftar	<i>Medium Priority</i>	Semua Aktor	Fitur interaktif yang meningkatkan partisipasi anggota, diimplementasikan setelah fungsionalitas manajemen <i>event</i> induk dinyatakan stabil.

	notifikasi pembaruan sistem.			
PB-10	Sebagai pengguna, saya ingin mengunggah foto dokumentasi kegiatan sebagai bukti digital penyelesaian tugas <i>event</i> .	<i>Medium Priority</i>	Semua Aktor	Berperan sebagai pengayaan data transaksional, dibangun setelah alur pendelegasian dan penyelesaian tugas fungsional berjalan valid.
PB-11	Sebagai Ketua atau Ketua Divisi, saya ingin mengelola galeri dokumentasi yang masuk untuk divalidasi sebelum dapat dilihat oleh publik.	<i>Medium Priority</i>	Ketua, Ketua Divisi	Instrumen kontrol administratif untuk mengurasi kesesuaian berkas foto dokumentasi yang diunggah oleh pengurus.
PB-12	Sebagai pengguna, saya ingin mengelola data	<i>Medium Priority</i>	Semua Aktor	Fitur utilitas pendukung keandalan data personal yang dikerjakan secara

	profil pribadi secara mandiri melalui menu pengaturan akun pengguna.			paralel setelah modul utama selesai.
PB-13	Sebagai Anggota, saya ingin memilih preferensi divisi organisasi secara mandiri di dalam sistem aplikasi.	<i>Low Priority</i>	Anggota	Fitur opsional pencatatan minat anggota yang memiliki tingkat dependensi rendah terhadap fungsi utama aplikasi.

4.3 Sprint Backlog

Setelah daftar pekerjaan besar di dalam *Product Backlog* berhasil diidentifikasi dan diprioritaskan berdasarkan tingkat ketergantungan antar-fitur, langkah selanjutnya dalam kerangka kerja Agile adalah menyusun Sprint *Backlog*. Pada tahap ini, item-item pekerjaan yang berkategori prioritas tinggi (High Priority) dipilih dan dipecah kembali menjadi serangkaian tugas teknis yang lebih kecil dan spesifik (technical tasks) untuk dieksekusi dalam satu siklus iterasi waktu (Sprint).

Berdasarkan hasil kesepakatan lini masa pengembangan, durasi untuk Sprint 1 ditetapkan selama 2 minggu (14 hari kalender). Iterasi pertama ini difokuskan secara penuh pada pembangunan arsitektur dasar aplikasi, mencakup modul autentikasi multi-peran, manajemen struktur divisi organisasi, serta fungsionalitas pendelegasian tugas operasional kegiatan.

4.3.1 Ruang lingkup Sprint 1

Tabel 4.4 Tabel *Sprint Backlog 1*

ID <i>Backlog</i>	User Story terkait	Komponen Tugas Teknis (Technical Tasks)	Target Estimasi	Penanggung Jawab
SB-01	PB-01 & PB-02: Autentikasi dan Manajemen Peran Akun	1. Perancangan skema basis data untuk tabel <i>Users</i> dan <i>roles</i> . 2. Pengembangan antarmuka registrasi dan <i>login</i> berbasis <i>Ionic</i> . 3. Implementasi logika enkripsi kata sandi dan token sesi di sisi <i>backend PHP</i> .	5 Hari	Developer

		4. Pengujian validasi otorisasi masuk sistem.		
SB-02	PB-03: Manajemen Divisi Organisasi oleh Ketua	1. Pembuatan tabel relasional pembentukan divisi organisasi. 2. Pengembangan formulir input divisi baru dan menu pengangkatan ketua divisi oleh Ketua. 3. Implementasi <i>middleware</i> untuk pembatasan hak akses menu divisi.	5 Hari	Developer
SB-03	PB-12: Kemandirian Pengelolaan Profil Pengguna	1. Pengembangan halaman pengaturan profil pada antarmuka <i>mobile</i> . 2. Pengkodean fungsi pembaruan data personal dan unggah foto profil.	4 Hari	Developer

		3. Pengujian integrasi pembaruan data profil ke <i>database</i> .		
--	--	---	--	--

4.3.2 Ruang lingkup *Sprint 2*

Tabel 4.5 Tabel *Sprint Backlog 2*

ID <i>Backlog</i>	User Story terkait	Komponen Tugas Teknis (Technical Tasks)	Target Estimasi	Penanggung Jawab
SB-04	PB-04, PB-09 & PB-11: Manajemen <i>Event</i> dan Partisipasi	1. Perancangan tabel data <i>events</i> dan relasi partisipan. 2. Pembuatan halaman <i>CRUD event</i> (Ketua) dan tombol registrasi <i>event</i> (Anggota). 3. Logika pengondisian status acara secara otomatis.	4 Hari	Developer

SB-05	PB-05 & PB-06: Pendelegasian Tugas Organisasi	1. Pembuatan tabel transaksional <i>tasks</i> untuk delegasi tugas. 2. Pengembangan halaman <i>input</i> tugas (Ketua/Ketua Divisi). 3. Pembuatan halaman filter daftar tugas personal (Anggota).	4 Hari	Developer
SB-06	PB-07 & PB-08: Publikasi Berita (<i>News</i>) dan Struktur	1. Perancangan skema tabel <i>news</i> untuk berita visual. 2. Pembuatan modul publikasi berita baru (Ketua/Ketua Divisi). 3. Pengembangan halaman penayangan utama <i>News</i> dan halaman struktur organisasi.	3 Hari	Developer

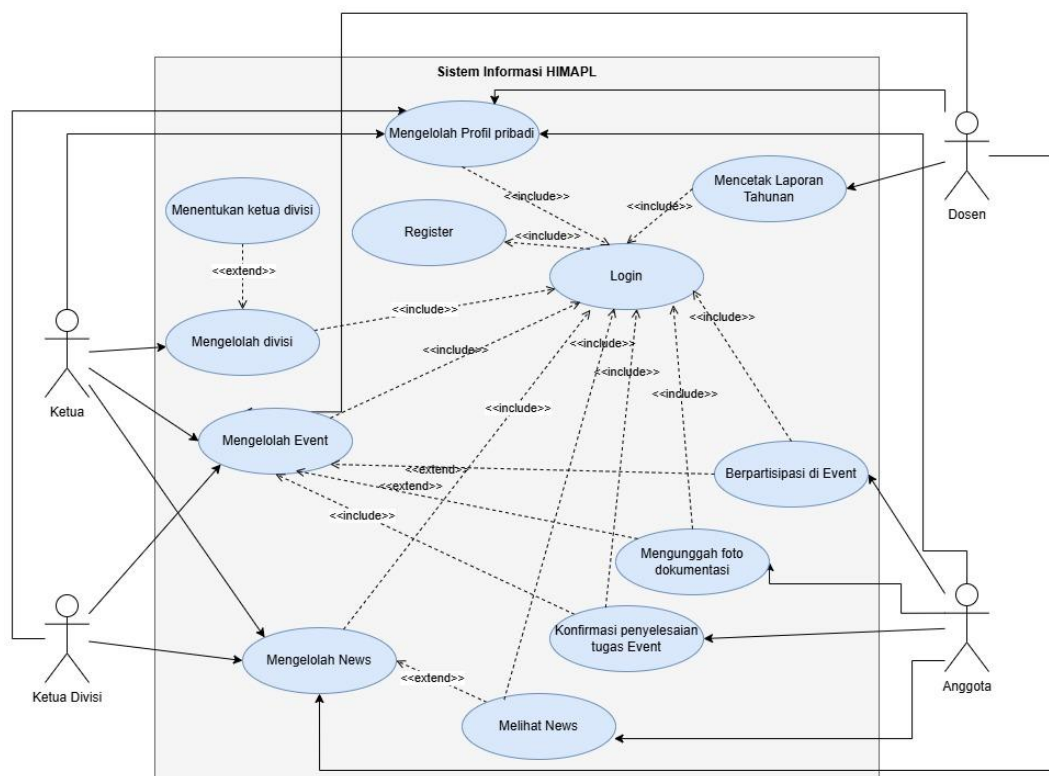
SB-07	PB-10 & PB-13: Dokumentasi Kegiatan dan Notifikasi	1. Pembuatan modul unggah foto dokumentasi (Anggota). 2. Pembuatan halaman validasi foto oleh Ketua Divisi. 3. Implementasi sistem <i>trigger</i> dan daftar riwayat notifikasi.	3 Hari	Developer
-------	---	--	--------	-----------

4.4 Analisis dan Perancangan Sistem

Pemodelan arsitektur aplikasi Sistem Informasi HIMAPL berbasis *mobile* dijabarkan menggunakan Unified Modeling Language (UML) guna memvisualisasikan alur kerja dan interaksi sistem secara terstruktur. Tahap perancangan ini bertindak sebagai jembatan (blueprint) untuk mentransformasikan deskripsi kebutuhan fungsional menjadi representasi visual berorientasi objek yang terstandarisasi sebelum memasuki tahap pengkodean program.

4.4.1 Use Case Diagram

Use Case Diagram digunakan untuk memvisualisasikan batas interaksi fungsional serta hak akses yang dimiliki oleh setiap aktor pengguna terhadap modul-modul inti di dalam ekosistem aplikasi Sistem Informasi HIMAPL. Sistem ini mengelola empat peran pengguna utama dengan hak akses yang spesifik, yaitu Anggota, Ketua, Ketua Divisi, dan Dosen Pembina.



Gambar 4.1 Use Case Diagram

Deskripsi diagram:

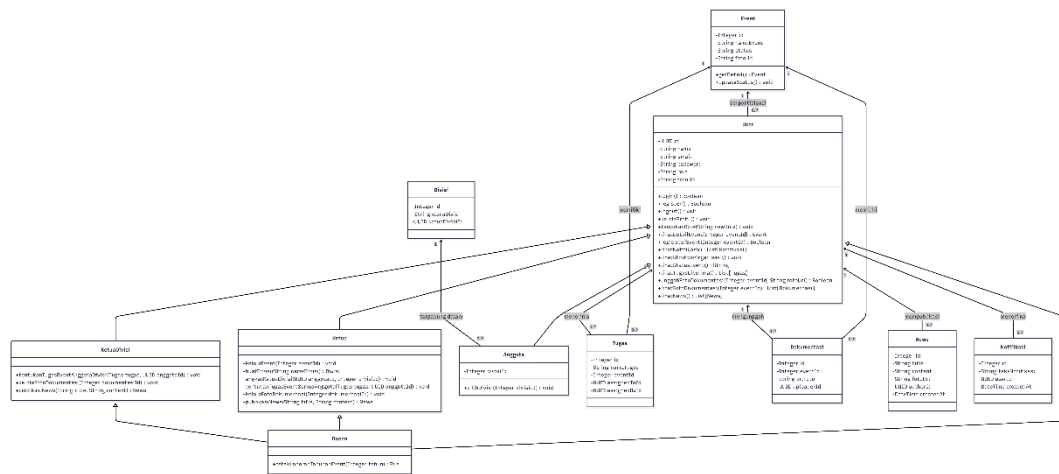
- Aktor Semua Pengguna (Aktor Induk): Memiliki otorisasi dasar untuk melakukan registrasi, login, logout, mengelola data profil pribadi, melihat

daftar notifikasi, membaca kabar terbaru, serta memantau struktur organisasi global HIMAPL.

- Aktor Anggota: Memiliki hak akses khusus untuk melakukan registrasi kepesertaan pada suatu *event* aktif, memilih divisi organisasi secara mandiri, serta melihat daftar instruksi tugas individual yang didelegasikan kepadanya.
- Aktor Ketua Divisi: Memiliki wewenang manajerial tingkat menengah untuk mendelegasikan tugas spesifik terkait *event* kepada anggota yang berada di bawah naungan divisinya, mempublikasikan kabar terbaru, serta mengelola validasi foto dokumentasi kegiatan yang diunggah anggota divisi.
- Aktor Ketua: Bertindak sebagai administrator utama organisasi dengan hak akses manajerial tingkat tinggi untuk mengelola data *event* (tambah, ubah, hapus), membuat divisi baru, mengangkat anggota menjadi ketua divisi, menentukan delegasi tugas kepada seluruh pengurus, serta mempublikasikan pengumuman organisasi secara terpusat.
- Aktor Dosen Pembina: Memiliki hak akses menyeluruh (multirole super-*User*) yang memungkinkan dosen untuk memantau seluruh aktivitas organisasi dan mengeksekusi semua fungsionalitas yang tersedia bagi aktor Ketua maupun pengguna lainnya demi kelancaran proses pembinaan.

4.4.2 *Class Diagram*

Class Diagram memetakan struktur statis sistem dengan menunjukkan hubungan relasional antar-kelas, atribut data, operasi, serta skema pemodelan basis data yang membentuk logika dasar aplikasi Sistem Informasi HIMAPL.



Gambar 4.2 Class Diagram

Deskripsi hubungan antar-entitas:

- Kelas *User* memiliki relasi pewarisan spesialisasi (*Generalization*) yang menurunkan karakteristik data dasar akun ke entitas peran yang lebih spesifik, yaitu kelas *Anggota*, *Ketua*, dan *Dosen*.
- Kelas *Ketua* memiliki relasi satu-ke-banyak (*One-to-Many*) dengan kelas *Division*, yang menunjukkan bahwa seorang *Ketua* dapat membentuk dan mengelola banyak divisi di dalam organisasi.
- Kelas *Division* terhubung dengan kelas *Anggota* untuk merekam komposisi kepengurusan, di mana satu divisi menaungi banyak anggota aktif.
- Kelas *Event* bertindak sebagai entitas transaksi induk yang memiliki relasi *One-to-Many* terhadap kelas *Task* (pendelegasian tugas kegiatan) dan kelas *Documentation* (galeri foto pembuktian kegiatan). Hal ini memastikan setiap tugas yang dibuat dan foto yang diunggah akan selalu terikat pada ID *event* yang valid.

4.5 The Sprint

Fase *The Sprint* merupakan tahap eksekusi dari draf perencanaan yang telah disusun di dalam *Sprint Backlog*. Seluruh siklus pengembangan perangkat lunak pada aplikasi Sistem Informasi HIMAPL berbasis *mobile* ini diselesaikan dalam beberapa tahapan iterasi yang adaptif dan terukur. Penentuan jumlah iterasi didasarkan pada kompleksitas fungsionalitas sistem serta batas lini masa pengerjaan yang telah dijadwalkan.

Untuk memastikan kualitas hasil pengembangan dan kendali integrasi sistem yang baik, setiap siklus iterasi dijalankan dengan memenuhi lima pilar tahapan utama kerangka kerja *Agile*, yaitu fase perencanaan (*planning*), fase pengembangan dan perancangan (*implementation*), sinkronisasi harian (*daily scrum*), demonstrasi hasil (*review*), serta evaluasi proses internal (*retrospective*). Pada bagian ini, seluruh dokumentasi proses dan hasil dari eksekusi siklus *Sprint* akan dijabarkan secara mendalam.

4.5.1 Sprint 1

Siklus *Sprint* 1 merupakan tahapan eksekusi perdana yang difokuskan pada pembangunan fondasi dasar arsitektur aplikasi Sistem Informasi HIMAPL berbasis *mobile*. Berdasarkan hasil analisis dependensi pada *Product Backlog*, iterasi pertama ini dijadwalkan berlangsung dengan durasi total selama 14 hari kalender. Fokus utama dari iterasi ini adalah menyelesaikan seluruh item pekerjaan yang berkategori prioritas tinggi (High Priority), mencakup modul

otentikasi multi-peran, manajemen divisi, manajemen *event* utama, serta delegasi tugas operasional.

4.5.1.1 *Planning*

Fase *Planning* dilaksanakan pada awal siklus melalui pertemuan formal antara peneliti selaku pengembang (*developer*) dan Ketua HIMAPL selaku representasi pengguna (*User*). Pertemuan ini bertujuan menyepakati *Sprint* Goal, memvalidasi rincian tugas teknis (*technical tasks*), serta menentukan estimasi waktu pengerjaan yang realistis agar target produk tercapai di akhir iterasi.

Distribusi lini masa pelaksanaan aktivitas teknis selama *Sprint* 1 disusun secara sistematis sebagaimana dirinci pada Tabel berikut.

Tabel 4.6 Tabel *Planning Sprint 1*

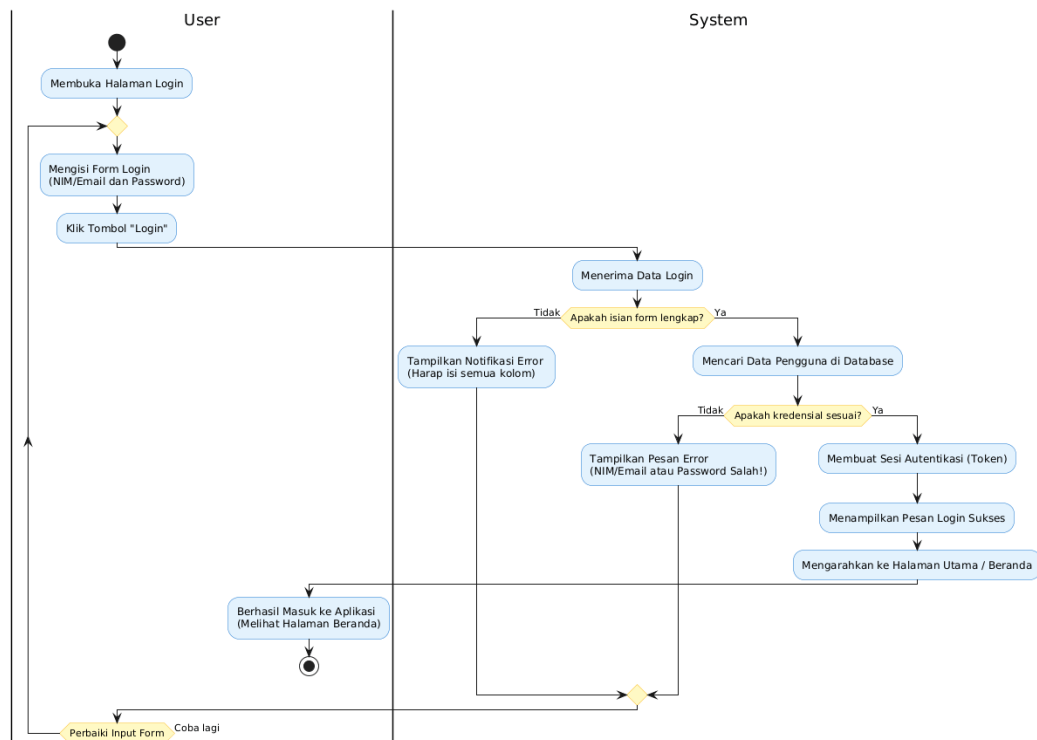
Lini Masa (Hari Ke-)	Target Komponen Fungsi Sistem	Deskripsi Implementasi Teknis
1 – 5	SB-01: Autentikasi dan Manajemen Peran Akun	Perancangan skema tabel <i>Users</i> dan <i>roles</i> pada basis data, pembuatan antarmuka registrasi dan <i>login</i> berbasis <i>Ionic</i> , serta pengkodean fungsi enkripsi kata sandi.

6 – 10	SB-02: Manajemen Divisi Organisasi	Pembuatan tabel relasional pembentukan divisi, implementasi formulir pembuatan divisi, serta menu otorisasi untuk pengangkatan ketua divisi oleh Ketua.
11 – 14	SB-03: Kemandirian Kelola Profil Pengguna	Pengembangan halaman pengaturan profil pada antarmuka <i>mobile</i> , pengkodean fungsi pembaruan data personal, serta fitur unggah foto profil ke server.

4.5.1.2 Implementation

a. Activity Diagram

1) Login



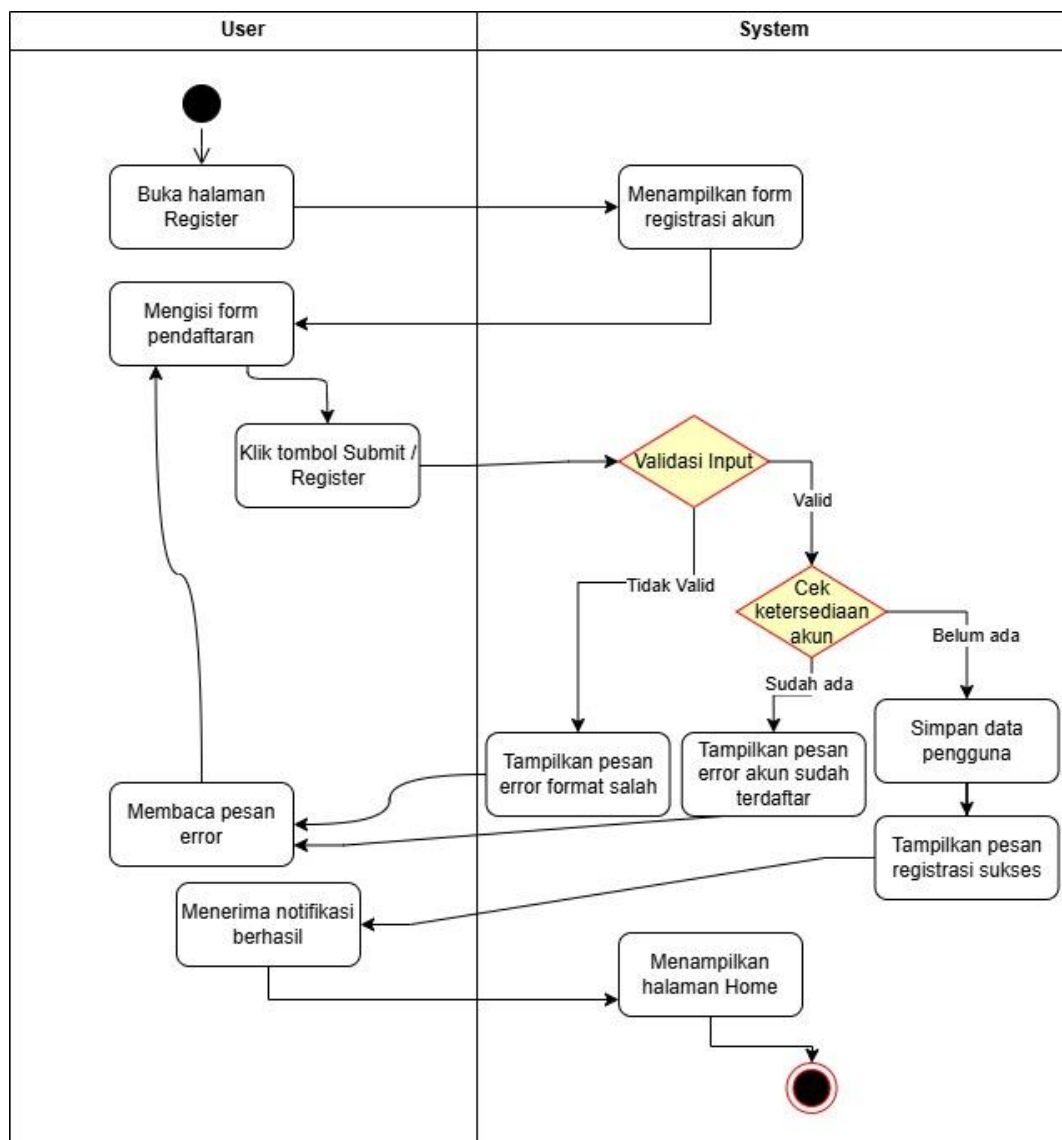
Gambar 4.3 Activity Diagram Login

Gambar 4.3 memvisualisasikan alur *Activity Diagram* dari proses *login* pada aplikasi Sistem Informasi HIMAPL. Diagram ini menjelaskan interaksi antara pengguna (*User*) dan sistem. Alur dimulai ketika pengguna membuka halaman *login* dan menginputkan data kredensial berupa email/NIM dan kata sandi (*password*), lalu menekan tombol "*Login*".

Sistem kemudian menerima data tersebut dan melakukan validasi kelengkapan isian form secara lokal. Apabila data yang dimasukkan belum lengkap, sistem akan memunculkan pesan peringatan (*error*) dan meminta pengguna untuk melengkapi data. Jika data sudah lengkap, sistem akan melanjutkan proses dengan mencari kecocokan data pengguna di dalam database. Apabila kredensial yang dimasukkan salah atau tidak ditemukan, sistem akan menampilkan pesan *error* format/kredensial salah. Namun, jika

kredensial terotentikasi dengan benar, sistem akan membuat sesi otorisasi berupa *token*, menampilkan notifikasi *login* berhasil, dan secara otomatis mengarahkan pengguna untuk masuk ke Halaman Utama (Beranda/*Home*).

2) Register



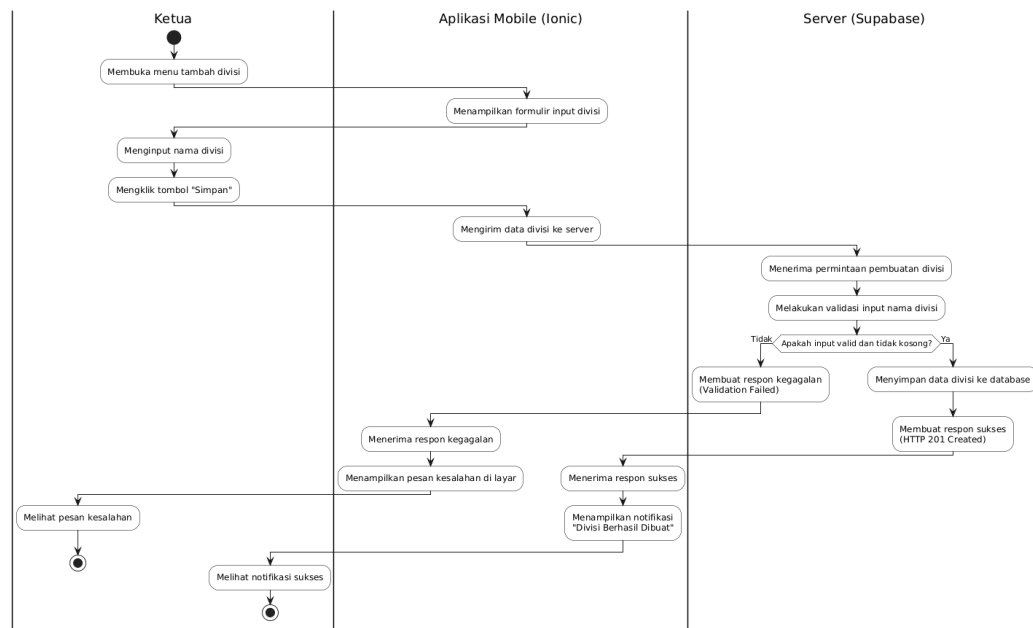
Gambar 4.4 Activity Diagram Register

Gambar 4.4 memaparkan *Activity Diagram* dari proses pendaftaran akun (*register*) bagi pengguna baru. Proses dimulai saat pengguna membuka

halaman *Register* dan mengisi formulir pendaftaran yang telah disediakan. Setelah seluruh data diisi, pengguna menekan tombol "*Submit / Register*".

Sistem kemudian akan merespons dengan memvalidasi format inputan yang dikirimkan. Apabila format data tidak *valid*, sistem akan menampilkan pesan *error* terkait *format* yang salah dan meminta pengguna untuk memperbaikinya. Jika *input valid*, sistem akan melakukan pengecekan ketersediaan akun di dalam database. Apabila email atau NIM telah terdaftar, sistem akan memunculkan pesan bahwa akun sudah ada. Sebaliknya, jika data tersebut belum pernah terdaftar, sistem akan memproses penyimpanan data pengguna baru ke dalam database. Setelah proses penyimpanan berhasil, sistem menampilkan notifikasi pendaftaran sukses kepada pengguna dan secara otomatis mengarahkannya ke halaman *Home* (Beranda).

3) Membuat Divisi



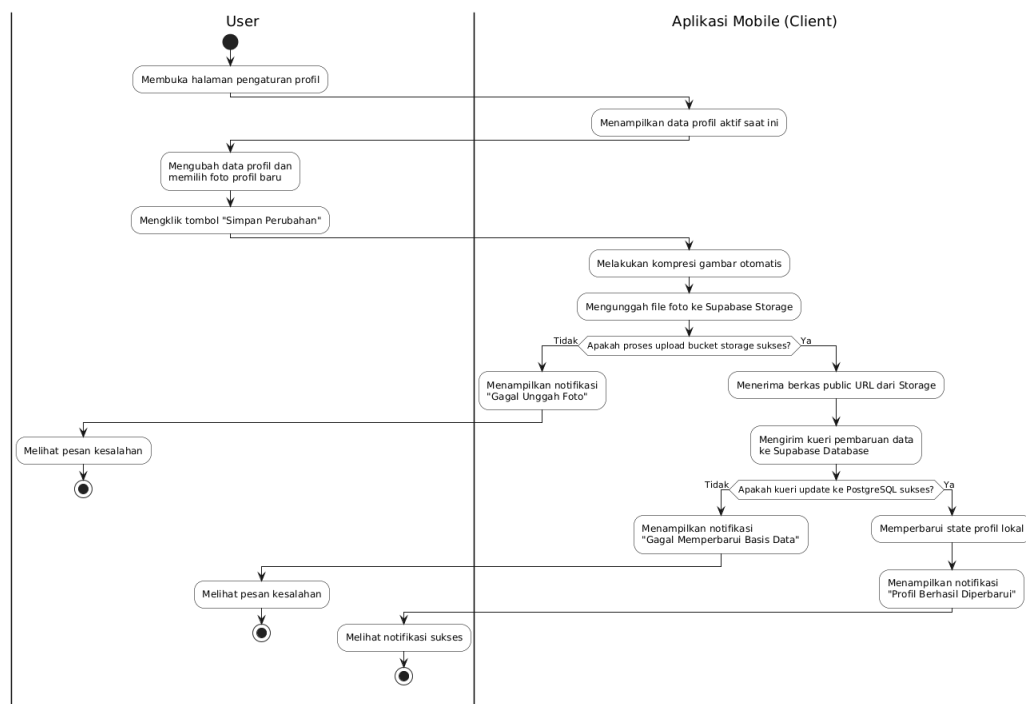
Gambar 4.5 Activity Diagram Membuat Divisi

Gambar 4.5 memperlihatkan *Activity Diagram* tentang proses penambahan divisi baru yang hanya dapat diakses oleh pengguna dengan hak akses (*role*) Ketua. Proses ini melibatkan interaksi antara aktor Ketua, Aplikasi Mobile (*Ionic*), dan Server (*Supabase*). Alur dimulai ketika Ketua membuka menu tambah divisi, kemudian aplikasi akan menampilkan formulir *input* divisi. Ketua lalu memasukkan nama divisi dan menekan tombol "Simpan".

Setelah tombol ditekan, aplikasi mobile akan mengirimkan permintaan penambahan data tersebut ke *server Supabase*. *Server* kemudian akan mengevaluasi permintaan dengan melakukan validasi kelengkapan *input*. Jika *input* kosong atau tidak *valid* (misalnya terjadi duplikasi), *server* akan mengembalikan respons kegagalan, dan aplikasi akan menampilkan pesan kesalahan kepada Ketua. Sebaliknya, jika data divalidasi dengan benar

(*input valid*), *server* akan memproses penyimpanan data divisi tersebut ke dalam *database* dan mengembalikan respons sukses (*HTTP 201 Created*). Aplikasi *mobile* kemudian menangkap respons tersebut dan memunculkan notifikasi "Divisi Berhasil Dibuat" yang dapat dilihat langsung oleh Ketua.

4) Kelola Profil



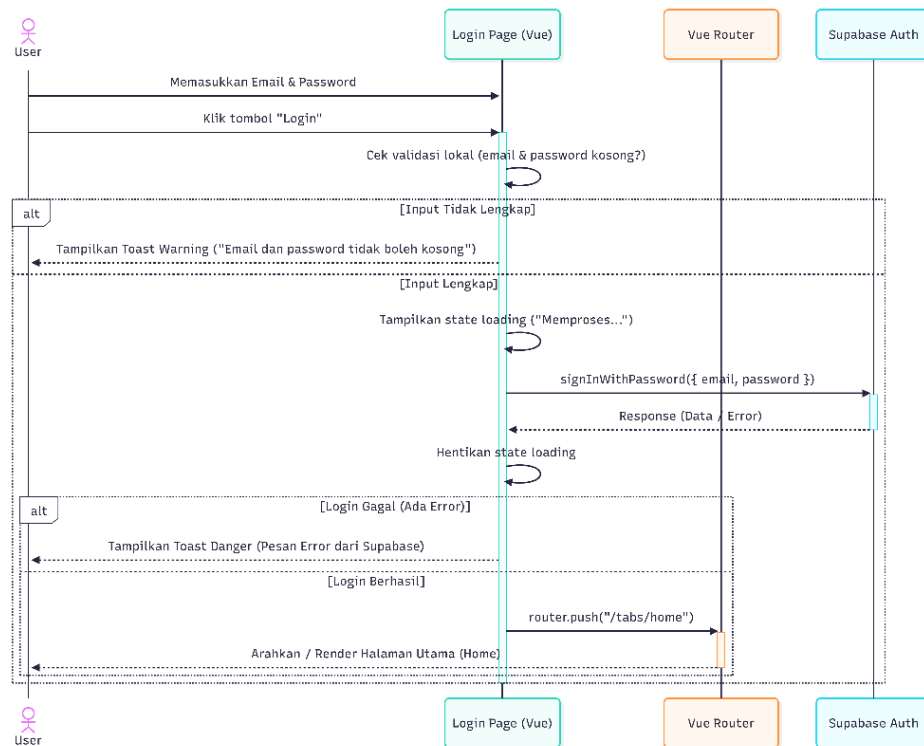
Gambar 4.6 Activity Diagram Kelola Profil

Gambar 4.6 menggambarkan *Activity Diagram* dari proses pengelolaan data profil pengguna. Alur dimulai ketika pengguna (*User*) membuka halaman pengaturan profil, dan aplikasi *mobile* merespons dengan menampilkan informasi profil yang sedang aktif. Pengguna kemudian dapat melakukan perubahan pada data profil, termasuk memilih foto profil baru dari perangkat mereka, dan menekan tombol "Simpan Perubahan".

Setelah tombol ditekan, aplikasi *mobile* akan melakukan proses kompresi gambar secara otomatis agar ukuran file lebih optimal, kemudian mencoba mengunggah foto tersebut ke *Supabase* Storage. Jika proses unggah gagal, aplikasi akan memunculkan pesan peringatan "Gagal Unggah Foto" kepada pengguna. Namun, jika proses unggah berhasil, aplikasi akan menerima *URL* publik dari gambar yang tersimpan dan melanjutkan dengan mengirimkan *query* pembaruan data (termasuk *URL* foto baru) ke *database PostgreSQL* di *Supabase*. Sistem kembali melakukan pengecekan; jika update *database* gagal, pengguna akan menerima notifikasi "Gagal Memperbarui Basis Data". Jika berhasil, aplikasi akan memperbarui state profil secara lokal agar perubahan langsung terlihat, lalu menampilkan notifikasi "Profil Berhasil Diperbarui" sebagai penanda bahwa seluruh proses telah sukses.

b. *Sequence Diagram*

1) *Login*

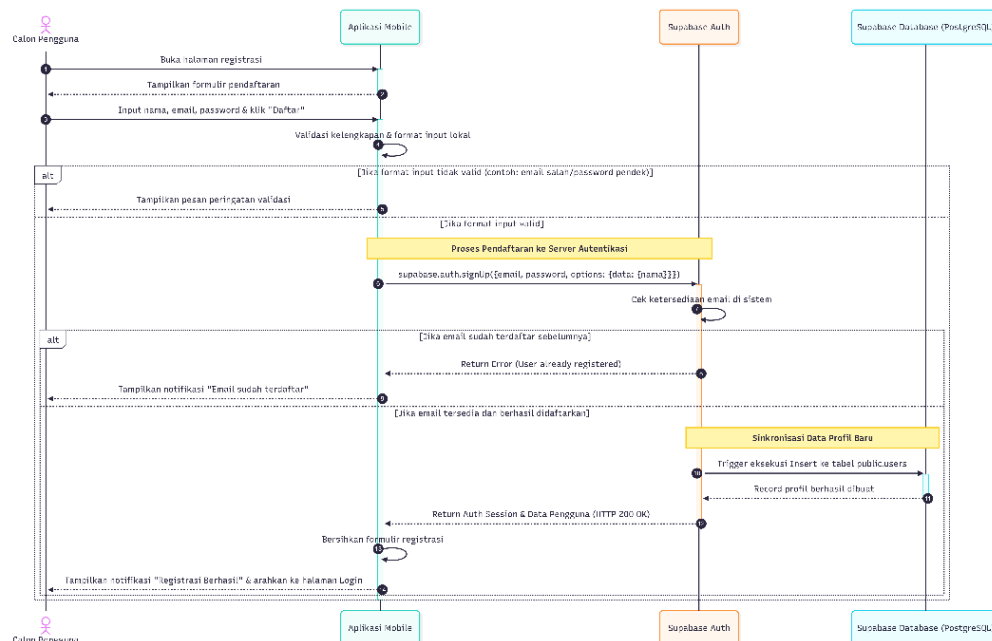


Gambar 4.7 *Sequence Diagram Login*

Gambar 4.7 memvisualisasikan *Sequence Diagram* untuk proses *login* pada aplikasi, yang melibatkan interaksi antara *User*, halaman *Login* (berbasis *Vue*), *Vue Router*, dan layanan *Supabase Auth*. Alur dimulai ketika *User* memasukkan alamat email dan kata sandi pada halaman *Login*, lalu menekan tombol "*Login*". Sistem pertama-tama akan melakukan pengecekan validasi lokal pada sisi *client* (*Vue*) untuk memastikan bahwa kolom *email* dan kata sandi tidak dikosongkan. Jika terdapat *input* yang kosong, sistem mengeksekusi kondisi alternatif pertama (*Input Tidak Lengkap*) dengan memunculkan pesan peringatan (*Toast Warning*) berupa "*Email dan password tidak boleh kosong*" kepada pengguna.

Sebaliknya, jika *input* sudah lengkap, sistem akan menampilkan *state loading* ("Memproses...") di layar dan memanggil fungsi *signInWithPassword* untuk mengirimkan kredensial tersebut ke *server Supabase Auth*. Setelah menerima respons dari *Supabase*, sistem menghentikan *state loading*. Di sini terdapat percabangan kondisi alternatif kedua berdasarkan respons *server*: jika autentikasi gagal, halaman login akan menangkap respons *error* dari *Supabase* dan menampilkannya kepada *User* dalam bentuk *Toast Danger*. Namun, jika autentikasi berhasil, sistem akan menginstruksikan *Vue Router* untuk melakukan navigasi (*router.push("/tabs/home")*), yang kemudian akan mengarahkan pengguna untuk masuk dan melihat tampilan Halaman Utama (*Home*).

2) Register



Gambar 4.8 Sequence Diagram Register

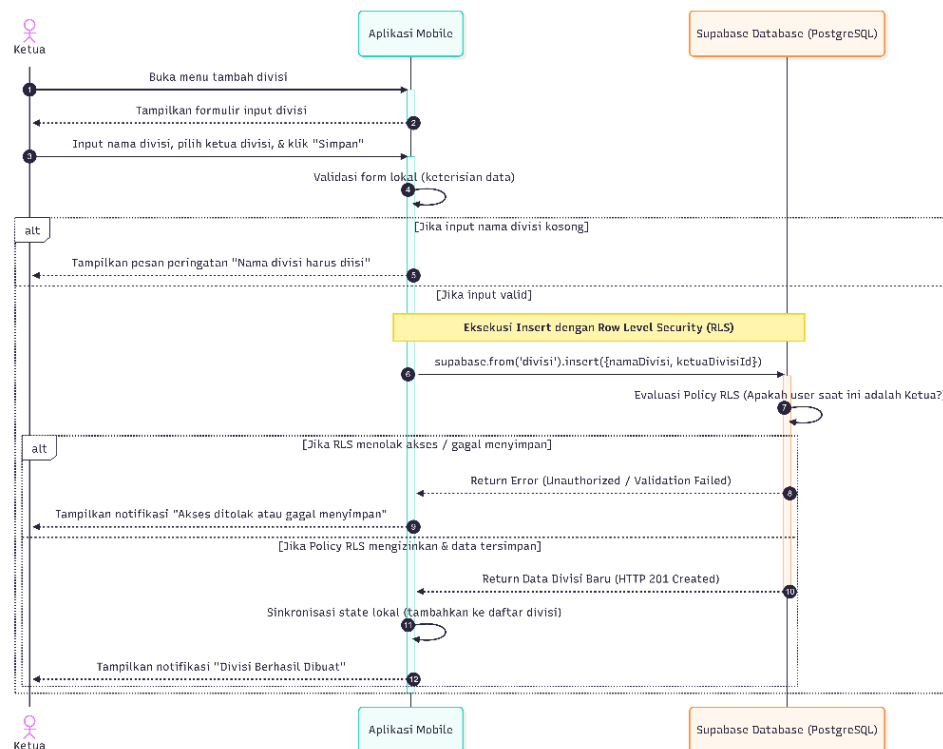
Gambar 4.8 menggambarkan *Sequence Diagram* untuk proses pendaftaran akun (*Register*). Proses ini melibatkan interaksi antara Calon Pengguna, Aplikasi *Mobile*, layanan *Supabase Auth*, dan *Supabase Database (PostgreSQL)*. Alur diawali saat calon pengguna membuka halaman registrasi, dan aplikasi memunculkan formulir pendaftaran. Setelah calon pengguna mengisi nama, *email*, dan kata sandi lalu mengeklik "Daftar", aplikasi *mobile* akan melakukan validasi lokal untuk memeriksa kelengkapan dan format isian. Terdapat percabangan kondisi (*alt*) pertama: jika format *input* tidak *valid* (misalnya penulisan *email* salah atau kata sandi terlalu pendek), aplikasi langsung menampilkan pesan peringatan validasi di layar tanpa mengirimkan permintaan ke *server*.

Jika format *input* divalidasi dengan benar, aplikasi akan memproses pendaftaran dengan mengirimkan perintah *supabase.auth.signUp* beserta parameter *email*, *password*, dan data (nama) ke *server Supabase Auth*. *Supabase Auth* kemudian mengecek ketersediaan *email* di dalam sistem. Pada percabangan kondisi (*alt*) kedua, jika *email* tersebut sudah pernah didaftarkan sebelumnya, *Supabase Auth* akan mengembalikan pesan *error* ("User already registered"), dan aplikasi akan menampilkan notifikasi "Email sudah terdaftar" kepada pengguna.

Sebaliknya, jika *email* tersedia dan pendaftaran berhasil, *Supabase Auth* akan memicu sinkronisasi data profil baru melalui *trigger* internal untuk melakukan operasi *insert* (penambahan data) ke tabel *users* di *Supabase*

Database (PostgreSQL). Setelah *record* profil berhasil dibuat di *database*, *Supabase Auth* mengembalikan sesi autentikasi dan data pengguna dengan status *HTTP 200 OK* ke aplikasi *mobile*. Aplikasi merespons dengan membersihkan formulir registrasi dan menampilkan notifikasi "Registrasi Berhasil" sekaligus mengarahkan calon pengguna kembali ke halaman *Login*.

3) Membuat divisi



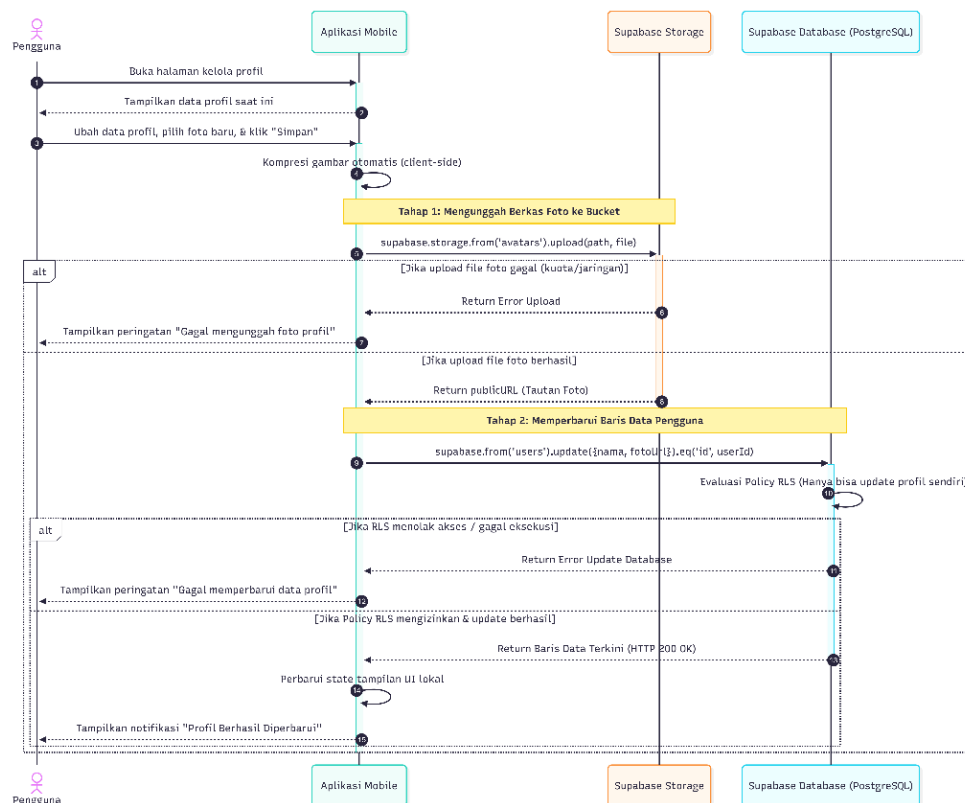
Gambar 4.9 *Sequence Diagram* Membuat Divisi

Gambar 4.9 menjelaskan *Sequence Diagram* untuk fitur penambahan divisi baru dalam organisasi. Interaksi pada diagram ini melibatkan aktor Ketua, Aplikasi *Mobile*, dan sistem *Supabase Database (PostgreSQL)*. Proses ini dimulai saat Ketua membuka menu tambah divisi, dan aplikasi

memunculkan formulir isian divisi. Setelah Ketua menginputkan nama divisi beserta memilih nama ketua divisi yang ditunjuk, kemudian menekan tombol "Simpan", aplikasi *mobile* melakukan validasi kelengkapan form secara lokal. Pada percabangan kondisi (*alt*) pertama, jika *input* nama divisi kosong, aplikasi seketika memunculkan pesan peringatan "Nama divisi harus diisi" tanpa meneruskan permintaan ke basis data.

Jika *input* tervalidasi dengan benar, aplikasi menjalankan perintah *insert* ke *database* (*supabase.from('divisi').insert(...)*). Pada tahap ini, *Supabase Database* akan melakukan evaluasi *Policy* dari *Row Level Security (RLS)* untuk memverifikasi apakah pengguna yang mengirimkan perintah saat ini benar-benar memiliki otoritas sebagai Ketua. Pada percabangan kondisi (*alt*) kedua, apabila *policy RLS* menolak akses tersebut atau terjadi kegagalan validasi *database*, *Supabase* akan mengembalikan respons *error* (*Unauthorized* atau *Validation Failed*), dan aplikasi *mobile* akan memunculkan notifikasi "Akses ditolak atau gagal menyimpan" kepada pengguna. Namun, jika *policy RLS* mengizinkan tindakan tersebut dan data berhasil disimpan, *Supabase Database* akan mengembalikan data divisi yang baru saja dibuat dengan status *HTTP 201 Created*. Aplikasi kemudian melakukan sinkronisasi *state* lokal dengan menambahkan data baru tersebut ke daftar divisi di antarmuka, dan ditutup dengan menampilkan notifikasi "Divisi Berhasil Dibuat".

4) Kelola Profil



Gambar 4.10 *Sequence Diagram* Kelola Profil

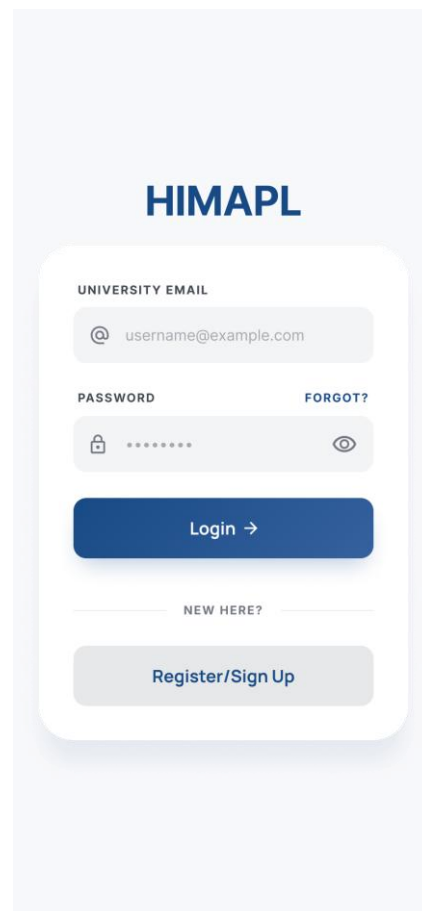
Gambar 4.10 memvisualisasikan *Sequence Diagram* untuk fitur kelola profil, yang melibatkan Pengguna, Aplikasi *Mobile*, *Supabase Storage*, dan *Supabase Database (PostgreSQL)*. Alur dimulai ketika pengguna membuka halaman kelola profil, dan aplikasi menampilkan informasi profil yang sedang digunakan saat ini. Pengguna lalu melakukan perubahan data serta memilih gambar baru untuk foto profil, kemudian mengeklik tombol "Simpan". Aplikasi *mobile* akan langsung melakukan kompresi gambar secara otomatis di sisi *client* sebelum masuk ke eksekusi tahap pertama: mengirimkan permintaan pengunggahan berkas foto ke *bucket Supabase Storage* (*supabase.storage.from('avatars').upload*).

Terdapat percabangan kondisi (*alt*) pertama di sini. Jika unggahan file gagal dikarenakan isu jaringan atau kuota, *Supabase Storage* akan mengembalikan respons *error*, dan aplikasi akan memunculkan peringatan "Gagal mengunggah foto profil" kepada pengguna. Namun jika unggahan berhasil, *Supabase Storage* akan merespons dengan mengembalikan *public URL* (tautan foto) dari gambar tersebut ke aplikasi.

Aplikasi lalu berlanjut ke eksekusi tahap kedua: mengirim permintaan pembaruan ke baris data pengguna (*supabase.from('users').update*) di *Supabase Database (PostgreSQL)*, sambil menyertakan *URL* foto profil baru. Saat itu juga, *database* akan mengevaluasi *policy Row Level Security (RLS)* untuk memastikan bahwa akun yang mengeksekusi query tersebut hanya memperbarui data profil miliknya sendiri. Pada kondisi (*alt*) kedua, bila *RLS* menolak akses tersebut atau eksekusi *database* gagal, akan muncul peringatan "Gagal memperbarui data profil". Jika *RLS* mengizinkan tindakan tersebut dan update berhasil, *Supabase Database* akan membalas dengan baris data terkini (*HTTP 200 OK*). Aplikasi *mobile* kemudian merespons dengan menyinkronkan *state* tampilan antarmuka secara lokal dan menampilkan notifikasi "Profil Berhasil Diperbarui".

c. Desain (*Figma*)

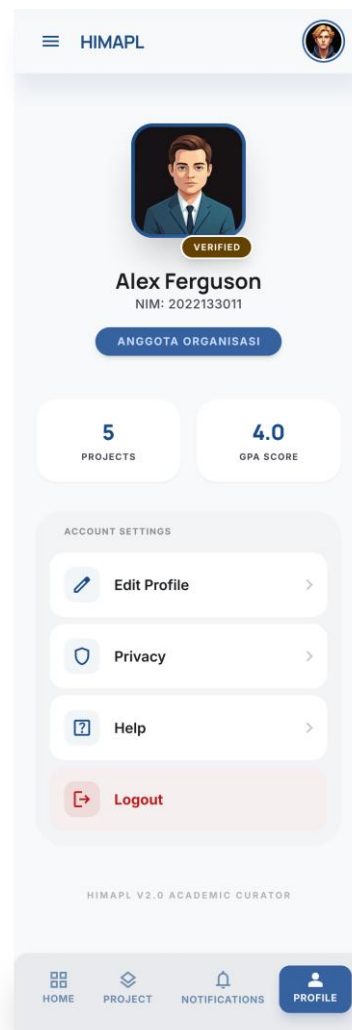
1) *Login & Register*



Gambar 4.11 Desain Halaman *Login*

Gambar 4.11 menampilkan rancangan *visual* atau *wireframe* antarmuka pengguna untuk halaman *Login* yang disusun menggunakan *platform Figma*. Desain ini mengadopsi konsep desain yang bersih (*clean UI*), di mana susunan elemen difokuskan di bagian tengah layar untuk memudahkan navigasi. Komponen utamanya meliputi form *input* untuk *Email* dan kata sandi (*Password*), disertai fasilitas ikon visibilitas sandi. Antarmuka ini juga mengakomodasi tombol aksi utama (*primary action*) untuk *Login*, tombol alternatif untuk *Sign in with Google*, dan tautan *Register/Sign Up* di bagian bawah bagi pengguna yang belum memiliki akun.

2) Halaman Profil



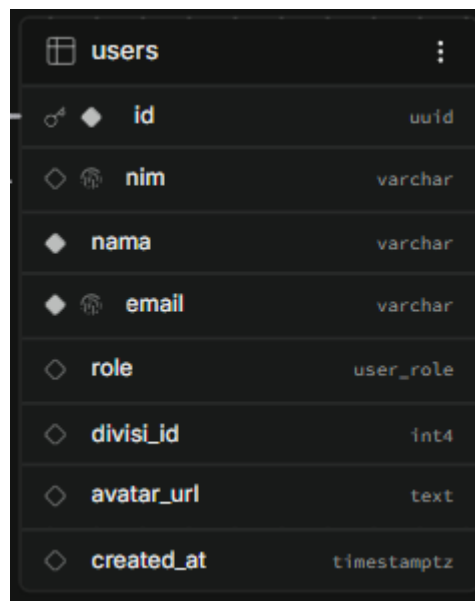
Gambar 4.12 Desain Halaman Profil

Gambar 4.12 menunjukkan *wireframe* antarmuka untuk Halaman Profil. Bagian atas halaman secara langsung menyorot identitas utama dengan menampilkan avatar (*foto profil*), nama lengkap, NIM, serta label peran (*badge role*) seperti "Anggota Organisasi". Terdapat juga blok *card* berisi elemen metrik pencapaian (misalnya skor GPA dan proyek). Untuk pengaturan akun, digunakan tata letak berbasis daftar (*list menu*) yang mudah diakses dengan sentuhan jari, memuat menu untuk *Edit Profile*, *Privacy*, *Help*, serta fitur penutup sesi (*Logout*). Di area paling bawah,

disematkan *bottom navigation bar* untuk menghubungkan pengguna secara cepat ke modul-modul lain di aplikasi.

d. Desain Skema Basis Data (*Supabase*)

1) *Database Users*

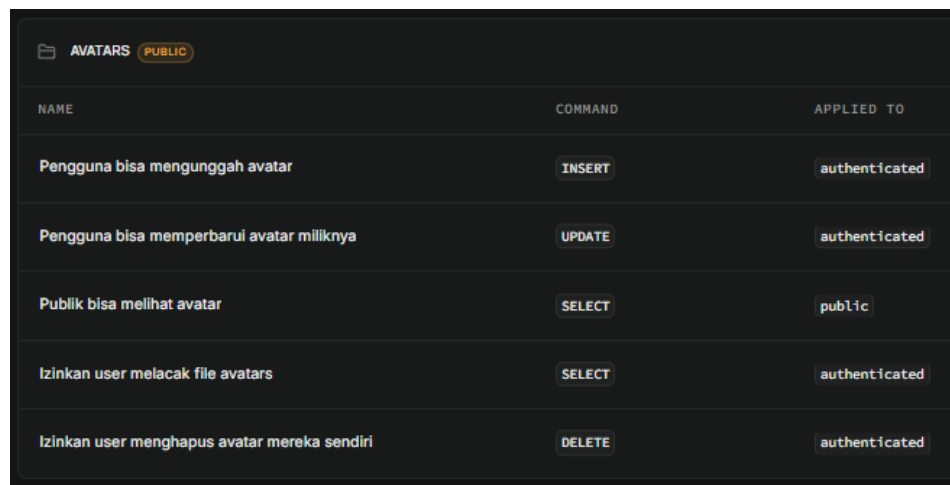


Column	Type	Constraints
id	uuid	Primary Key
nim	varchar	
nama	varchar	
email	varchar	
role	user_role	
divisi_id	int4	
avatar_url	text	
created_at	timestampz	

Gambar 4.13 *Database Tabel Users*

Gambar 4.13 menampilkan rancangan struktur skema untuk tabel *users* di dalam *PostgreSQL Supabase*. Tabel ini berfungsi sebagai pilar utama untuk menyimpan identitas akun dan profil pengguna aplikasi. Kolom-kolom di dalamnya meliputi *id* (bertipe *UUID* yang menjadi kunci utama dan saling berelasi dengan tabel *Auth*), *nim*, *nama*, *email*, penentuan tingkat akses melalui *role*, *divisi_id* untuk merepresentasikan letak anggota dalam struktur organisasi, serta *avatar_url* untuk menyimpan tautan lokasi *file* foto profil pengguna.

2) Penyimpanan Foto Profil

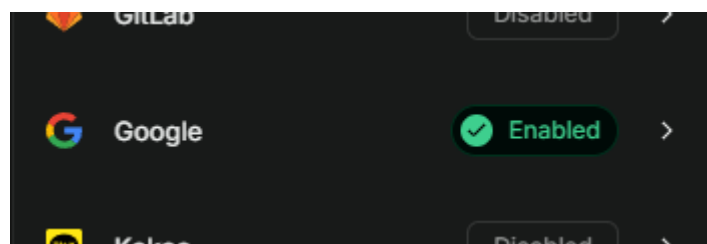


NAME	COMMAND	APPLIED TO
Pegguna bisa mengunggah avatar	INSERT	authenticated
Pegguna bisa memperbarui avatar miliknya	UPDATE	authenticated
Publik bisa melihat avatar	SELECT	public
Izinkan user melacak file avatars	SELECT	authenticated
Izinkan user menghapus avatar mereka sendiri	DELETE	authenticated

Gambar 4.14 *storage bucket avatars*

Gambar 4.14 mendokumentasikan pengaturan manajemen *bucket storage* di *Supabase* bernama *avatars*. *Bucket* ini difungsikan secara khusus sebagai wadah penyimpanan *file* gambar foto profil pengguna. Tampilan ini juga memperlihatkan konfigurasi aturan keamanan berbasis kebijakan (*Policy*) untuk mengontrol izin *file*. Pada pengaturannya, tindakan mengubah (*UPDATE*), menambah (*INSERT*), atau menghapus (*DELETE*) *file* hanya boleh dieksekusi oleh pengguna pemilik akun yang telah masuk (*authenticated*), sementara hak untuk membaca data (*SELECT*) terbuka bagi publik (*public*) agar foto pengguna lain dapat dilihat dari aplikasi.

3) Autentikasi Akun Google



Gambar 4.15 Autentikasi Akun Google

Gambar 4.15 menunjukkan panel pengaturan aktivasi Authentication Provider dari modul *Supabase Auth*, secara khusus untuk layanan integrasi akun *Google (OAuth)*. Pengaktifan status *Enabled* pada pengaturan ini menandakan bahwa sistem memiliki infrastruktur *Single Sign-On (SSO)*. Mekanisme ini mempermudah pengguna untuk mendaftar dan *login* ke aplikasi secara cepat, terenkripsi, dan aman tanpa keharusan untuk membuat dan mengingat kata sandi baru.

4) Database Divisi



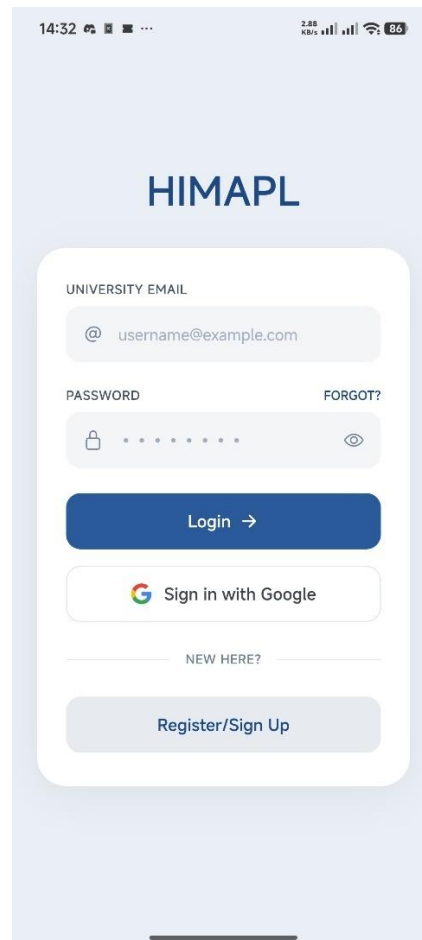
Column	Type
id	int4
nama_divisi	varchar
ketua_divisi_id	uuid
created_at	timestampz

Gambar 4.16 Database Tabel Divisi

Gambar 4.16 menyajikan susunan kolom untuk tabel *divisi* dalam *database* relasional. Tabel ini dirancang untuk mendigitalisasi pencatatan struktur hierarki internal organisasi. Atribut penyusunnya meliputi *id* bertipe integer yang bersifat increment, *nama_divisi* (keterangan identitas divisi), *ketua_divisi_id* (bertipe *UUID* yang memuat relasi *Foreign Key* ke tabel *users* guna mengetahui identitas akun ketua yang mengepalai divisi tersebut), dan *created_at* sebagai pencatat stempel waktu (*timestamp*) pembuatan rekaman.

e. Hasil Implementasi

1) Halaman *Login*

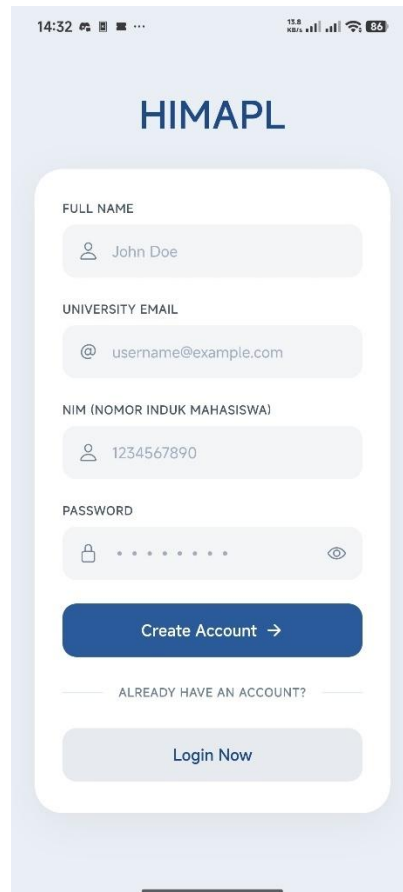


Gambar 4.17 Halaman *Login*

Gambar 4.17 memperlihatkan antarmuka halaman *Login* yang telah di-build menjadi aplikasi Android. Secara *visual*, tata letak implementasi ini dipertahankan agar sangat identik dengan desain *wireframe Figma* (Gambar 4.11), dengan penambahan polesan detail pada tombol *Sign in with Google*. Dari sisi fungsionalitas dan *database*, halaman ini terhubung langsung dengan modul *Supabase Auth*. Tombol *login* reguler akan mengirimkan kueri validasi alamat *email* dan *password*, sementara tombol integrasi *Google* menggunakan layanan *OAuth* dari *Supabase*. Setelah autentikasi

divalidasi oleh *server*, sistem akan memeriksa tabel *users* untuk memuat *state* pengguna berdasarkan hak aksesnya.

2) Halaman *Register*

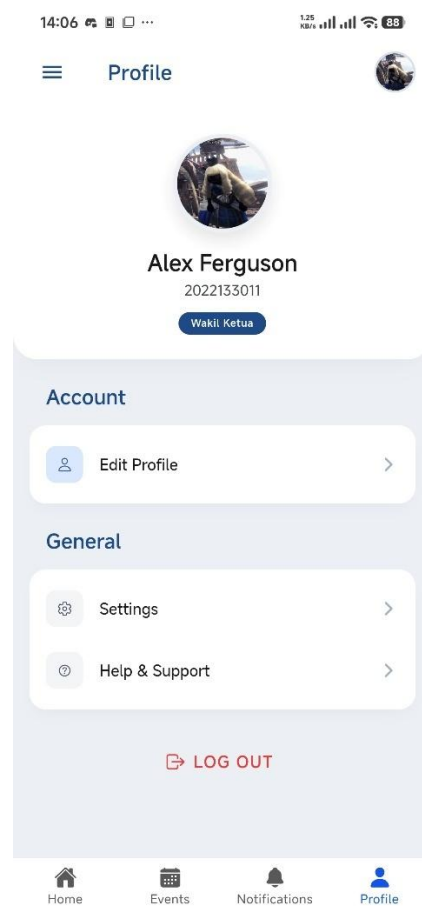


Gambar 4.18 Halaman *Register*

Gambar 4.18 menampilkan halaman *Register* untuk pendaftaran pengguna baru. Walaupun pada rancangan *Figma* sebelumnya antarmuka ini digabungkan secara konseptual dengan halaman *Login*, pada hasil implementasinya pengembang memisahkannya dan menambahkan kolom *input* spesifik yang tidak ada di *wireframe*, yaitu kolom "*FULL NAME*" dan "*NIM (NOMOR INDUK MAHASISWA)*". Penambahan ini disesuaikan

dengan kebutuhan relasi *database*, karena setiap *input* pendaftaran yang lolos akan memicu *Database Trigger* untuk menyisipkan (*insert*) nilai kolom nama dan NIM secara sinkron ke dalam struktur tabel *users* di *PostgreSQL*.

3) Halaman Profil

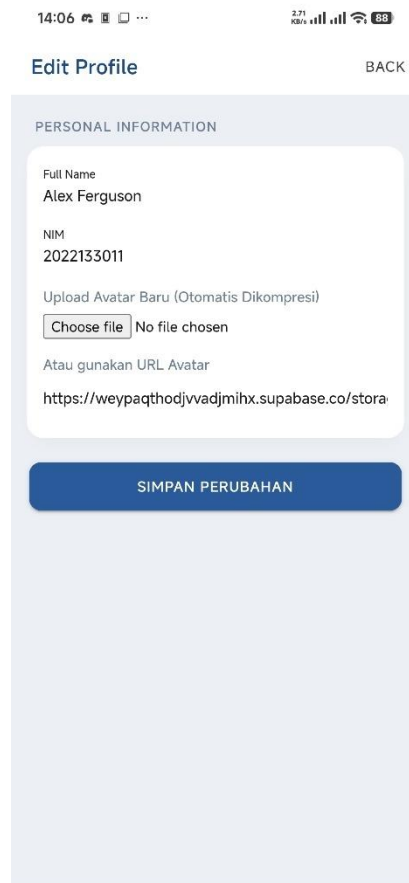


Gambar 4.19 Halaman Profil

Gambar 4.19 menunjukkan hasil implementasi Halaman Profil. Terdapat beberapa perubahan desain yang signifikan jika dibandingkan dengan *wireframe Figma* (Gambar 4.12). Pada hasil implementasi, kartu (card) metrik pencapaian seperti *Projects* dan *GPA Score* dihilangkan

karena data tersebut tidak relevan dengan tabel *users* pada sistem saat ini. Selain itu, badge *role* di bawah nama pengguna (yang di *Figma* tertulis "Anggota Organisasi") diubah sifatnya menjadi teks dinamis (contoh: "Wakil Ketua"). Hubungannya dengan *database*, antarmuka ini mengeksekusi kueri *SELECT* ke tabel *users* untuk menarik data nama, nim, dan kolom *role*, serta memuat gambar langsung dari *bucket avatars* menggunakan nilai *URL* pada kolom *avatar_url*.

4) Kelola Profil



14:06 88% ... 2.71 KB/s

Edit Profile BACK

PERSONAL INFORMATION

Full Name
Alex Ferguson

NIM
2022133011

Upload Avatar Baru (Otomatis Dikompresi)

Choose file No file chosen

Atau gunakan URL Avatar

<https://weypaqthodjvvdjmihx.supabase.co/stora>

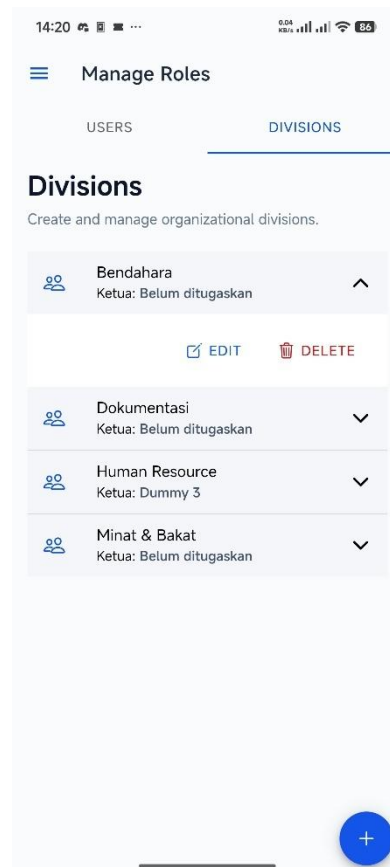
SIMPAN PERUBAHAN

Gambar 4.20 Halaman Kelola Profil

Gambar 4.20 menampilkan halaman *Edit Profil* yang secara khusus dikembangkan dalam fase coding, tanpa merujuk pada desain *wireframe Figma* sebelumnya. Halaman ini bertindak sebagai jembatan *Input/Output* untuk memperbarui profil. Kaitannya dengan sistem *backend*, halaman ini memuat logika untuk mengompresi gambar dan mengeksekusi perintah upload berkas secara terenkripsi ke *bucket Supabase Storage*. Setelah *upload* berhasil, halaman ini mengirimkan kueri *UPDATE* yang akan memodifikasi nilai nama dan *avatar_url* pada tabel *users*.

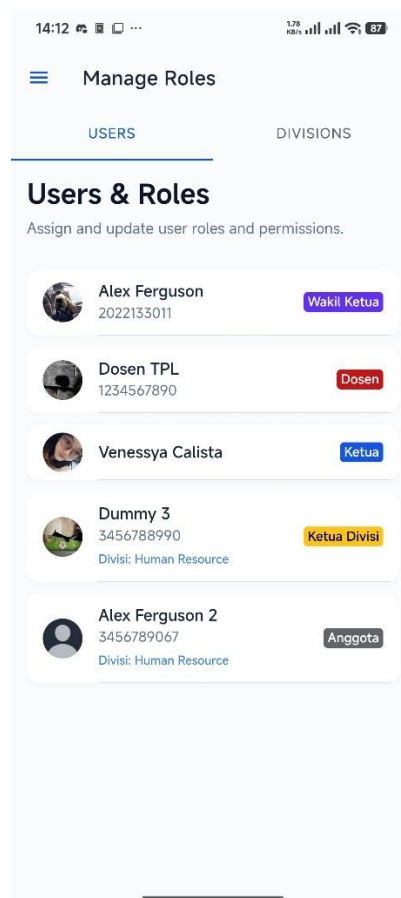
5) Kelola Peran dan Divisi

Halaman *role management* merupakan kumpulan antarmuka administratif tingkat lanjut yang dibangun murni menggunakan pustaka komponen *framework Ionic*, menyesuaikan dengan kebutuhan operasional sistem tanpa draf *wireframe Figma* sebelumnya.



Gambar 4.21 Halaman Kelola Divisi

Antarmuka ini membaca (*SELECT*) dan memanipulasi (*INSERT/UPDATE/DELETE*) tabel divisi di *Supabase*. Sistem menghubungkan *ID* Ketua Divisi yang tercatat (*ketua_divisi_id*) untuk menampilkannya bersama nama divisinya.



Gambar 4.22 Halaman Peran

Halaman ini berfungsi sebagai tabel data (data grid) *administrator* yang secara dinamis mengambil seluruh rekam data dari tabel *users*, lalu memetakan label peran (*role*) dan keanggotaan divisi setiap akun dalam bentuk daftar.

14:55 0.00 KB/s 84%

Edit Role CLOSE

Alex Ferguson
2022133011
Current Role: Anggota

Role
Anggota

Division
None

SAVE CHANGES

Gambar 4.23 Halaman Kelola Peran

Halaman edit *role* merupakan *form* turunan untuk menyunting data pengguna spesifik. Saat tombol "*Save Changes*" ditekan, aplikasi akan mengeksekusi kueri *UPDATE* ke tabel *users* untuk memodifikasi hak akses pada kolom *role* (menjadi Anggota/Ketua Divisi/Ketua) dan memperbarui relasi kepengurusan pada kolom *divisi_id*.

4.5.1.3 Review

Tabel 4.7 Tabel Review Sprint 1

ID Backlog	Fitur	Skenario Validasi (Acceptance Criteria)	Hasil Pengujian	Status
SB-01	Autentikasi dan Peran Akun	1. Pengguna dapat mendaftarkan akun baru melalui <i>Supabase Auth</i>. 2. Sistem berhasil menolak <i>login</i> dengan kredensial yang salah. 3. Pengguna diarahkan ke halaman <i>dashboard</i> yang sesuai dengan hak akses (<i>role</i>) masing-masing.	Sistem merespons seluruh proses autentikasi dan pembatasan halaman dengan tepat waktu dan akurat.	Diterima
SB-02	Manajemen Divisi Organisasi	1. Ketua dapat menambahkan data divisi baru ke <i>Supabase Database</i>.	Integrasi fungsi pembuatan divisi dan pembatasan keamanan <i>RLS</i>	Diterima

		2. Ketua dapat menetapkan status anggota menjadi Ketua Divisi. 3. Kebijakan <i>Row Level Security (RLS)</i> berhasil memblokir anggota biasa yang mencoba membuat divisi.	berjalan stabil tanpa celah (<i>bypass</i>).	
SB-03	Kelola Profil Pengguna	1. Pengguna dapat mengubah data <i>string</i> (nama/<i>password</i>) secara mandiri. 2. Aplikasi memproses kompresi gambar sebelum mengunggahnya ke <i>Supabase Storage</i>. 3. Pembaruan data profil langsung	Proses unggah berkas foto dan sinkronisasi data relasional berjalan lancar dengan waktu muat yang optimal.	Diterima

		direfleksikan pada antarmuka aplikasi (<i>state</i> lokal).		
--	--	--	--	--

Berdasarkan hasil demonstrasi dan pengujian fungsional secara langsung, seluruh item pekerjaan teknis (SB-01, SB-02, dan SB-03) dinyatakan Diterima (Accepted) oleh representasi pengguna. Dengan demikian, fondasi keamanan, struktur hierarki divisi, dan manajemen profil pengguna telah *valid* dan memenuhi standar kelayakan untuk diintegrasikan dengan modul lanjutan.

4.5.1.4 *Retrospective*

Fase *Sprint Retrospective* merupakan tahapan akhir dari iterasi *Sprint* 1 yang difokuskan pada evaluasi internal proses pengembangan perangkat lunak. Evaluasi ini tidak mengulas fungsionalitas sistem, melainkan menganalisis efektivitas alur kerja, penggunaan teknologi, dan manajemen waktu selama iterasi berlangsung. Tujuan utamanya adalah mengidentifikasi aspek kerja yang sudah optimal untuk dipertahankan, serta menemukan ruang perbaikan untuk diterapkan pada iterasi berikutnya (*Sprint* 2).

Hasil refleksi proses pengembangan pada *Sprint* 1 diuraikan ke dalam tiga matriks evaluasi utama: apa yang berjalan dengan baik (What Went Well), hambatan teknis yang dihadapi (Needs Improvement), dan rencana perbaikan kerja (Action Plan). Analisis tersebut didokumentasikan secara rinci pada tabel berikut.

Tabel 4.8 Tabel *Rotrospective Sprint 1*

Kategori Evaluasi	Hasil Refleksi Pengembang
Yang Berjalan Baik <i>(What Went Well)</i>	1. Penggunaan arsitektur <i>Supabase (BaaS)</i> terbukti mempercepat proses peluncuran layanan <i>backend</i>, sehingga pengembang tidak perlu membangun sistem autentikasi dari awal. 2. Integrasi fungsi kompresi gambar pada sisi <i>frontend (Ionic)</i> berhasil menghemat pemakaian kuota <i>bandwidth</i> penyimpanan di <i>Supabase Storage</i>.
Yang Perlu Ditingkatkan <i>(Needs Improvement)</i>	1. Konfigurasi aturan keamanan <i>Row Level Security (RLS)</i> pada <i>PostgreSQL</i> membutuhkan waktu penyesuaian (<i>debugging</i>) yang lebih lama dari estimasi awal akibat penolakan akses yang berlapis. 2. Proses penyesuaian tata letak (<i>UI/UX</i>) antarmuka <i>mobile</i> sering mengalami perubahan berulang di tengah fase <i>coding</i>, sehingga memperlambat kecepatan penyelesaian komponen.

Rencana Tindak Lanjut <i>(Action Plan)</i>	1. Untuk <i>Sprint 2</i> : Pengembang akan memetakan draf aturan <i>RLS</i> secara spesifik di atas kertas sebelum menuliskan kueri kebijakannya ke dalam sistem <i>Supabase</i> . 2. Untuk <i>Sprint 2</i> : Menerapkan pembekuan desain <i>visual (UI/UX freeze)</i> sebelum fase implementasi dimulai agar fokus pengkodean tidak terdistraksi oleh perubahan tata letak yang minor.
---	--

Melalui evaluasi komprehensif pada fase *retrospective* ini, siklus pengembangan *Sprint 1* secara resmi dinyatakan selesai. Pengalaman dan strategi tindak lanjut yang dirumuskan akan langsung diterapkan sebagai landasan pengerjaan operasional pada *Sprint 2*.

4.5.2 *Sprint 2*

Siklus *Sprint 2* merupakan iterasi lanjutan sekaligus iterasi final dalam tahap pengembangan utama (development phase) aplikasi Sistem Informasi HIMAPL berbasis *mobile*. Berdasarkan matriks prioritas pada *Sprint Backlog*, iterasi kedua ini dijadwalkan berlangsung selama 14 hari kalender. Fokus pengerjaan bergeser dari fondasi arsitektur dasar menuju fungsionalitas operasional organisasi, yang mencakup manajemen *event* dan partisipasi, pendelegasian

tugas pengurus, publikasi berita (News), hingga sistem dokumentasi dan notifikasi.

4.5.2.1 *Planning*

Fase perencanaan (*Planning*) pada *Sprint 2* dilaksanakan dengan meninjau kembali hasil evaluasi (*retrospective*) dari siklus sebelumnya. Pengembang menyepakati *Sprint Goal* untuk menyelesaikan seluruh sisa modul fungsional agar aplikasi siap diimplementasikan secara utuh. Berbekal evaluasi sebelumnya, perancangan kebijakan keamanan (*Row Level Security/RLS*) pada basis data *Supabase* akan dipetakan lebih awal sebelum tahap pengkodean dimulai guna mengoptimalkan waktu pengerjaan.

Distribusi lini masa dan rincian tugas teknis selama 14 hari siklus *Sprint 2* disusun secara sistematis sebagaimana dirinci pada tabel berikut.

Tabel 4.9 Tabel *Planning Sprint 2*

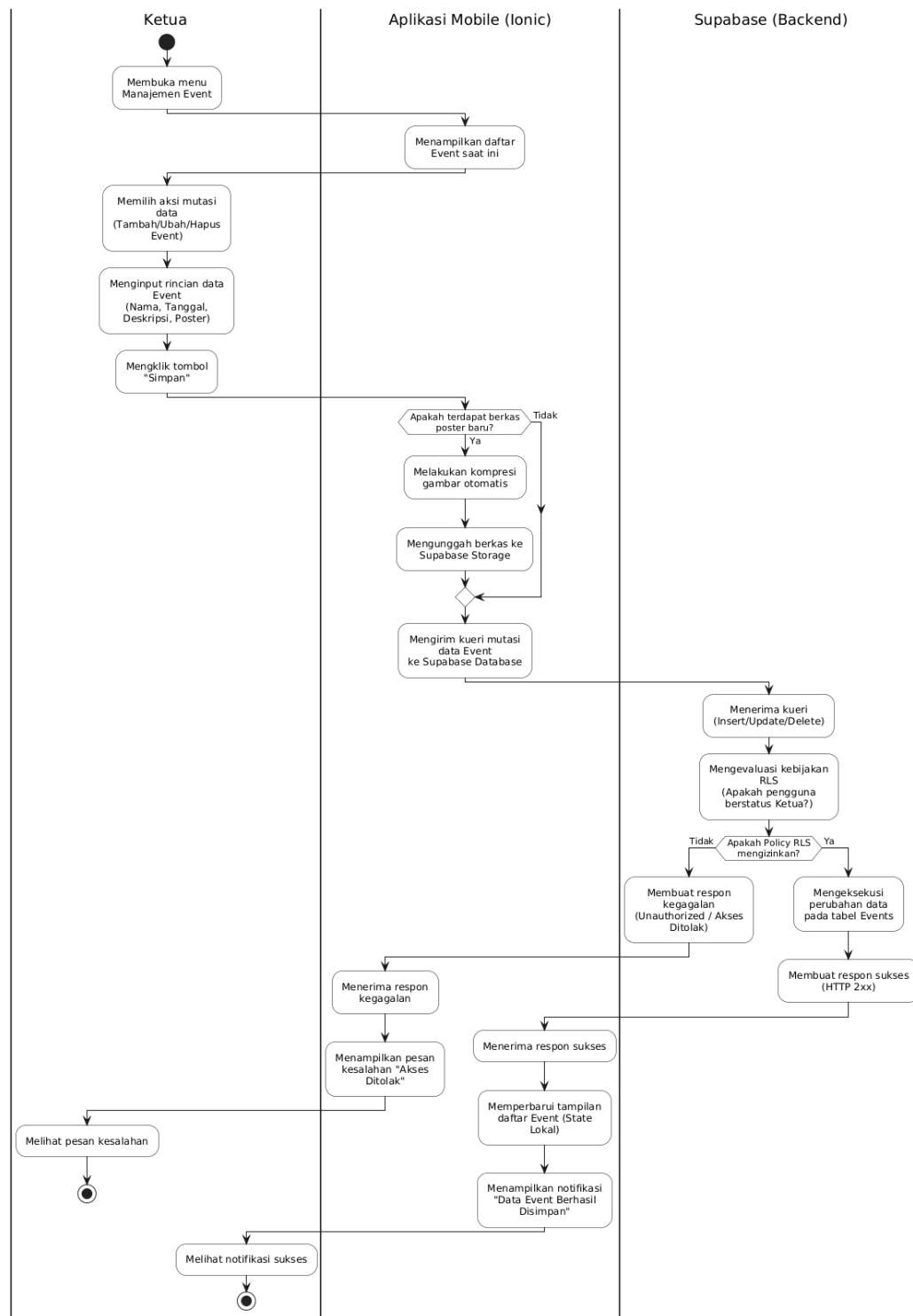
Lini Masa (Hari Ke-)	Target Komponen Fungsi Sistem	Deskripsi Implementasi Teknis
1 – 4	SB-04: Manajemen <i>Event</i> dan Partisipasi	Perancangan struktur relasional tabel <i>events</i> di <i>Supabase</i> , pembuatan halaman <i>CRUD event</i> bagi Ketua, penyediaan tombol registrasi <i>event</i> bagi anggota, serta pengkodean logika pengubahan status acara secara dinamis.

5 – 8	SB-05: Pendelegasian Tugas Organisasi	Pembuatan tabel <i>tasks</i> , pengembangan formulir pendelegasian tugas oleh Ketua/Ketua Divisi, serta perancangan antarmuka daftar tugas personal yang difilter berdasarkan <i>ID</i> pengguna aktif.
9 – 11	SB-06: Publikasi Berita (<i>News</i>) dan Struktur	Pembuatan modul publikasi berita <i>visual</i> yang terintegrasi dengan <i>Supabase Storage</i> , pengembangan halaman <i>feed</i> penayangan <i>News</i> , serta halaman interaktif untuk struktur organisasi.
12 – 14	SB-07: Dokumentasi Kegiatan dan Notifikasi	Pembuatan modul unggah foto pasca-kegiatan, halaman validasi dokumentasi oleh Ketua Divisi, serta implementasi <i>database trigger</i> di <i>PostgreSQL</i> untuk menghasilkan riwayat notifikasi secara otomatis.

4.5.2.2 Implementation

a) Activity Diagram

1) Kelola Event

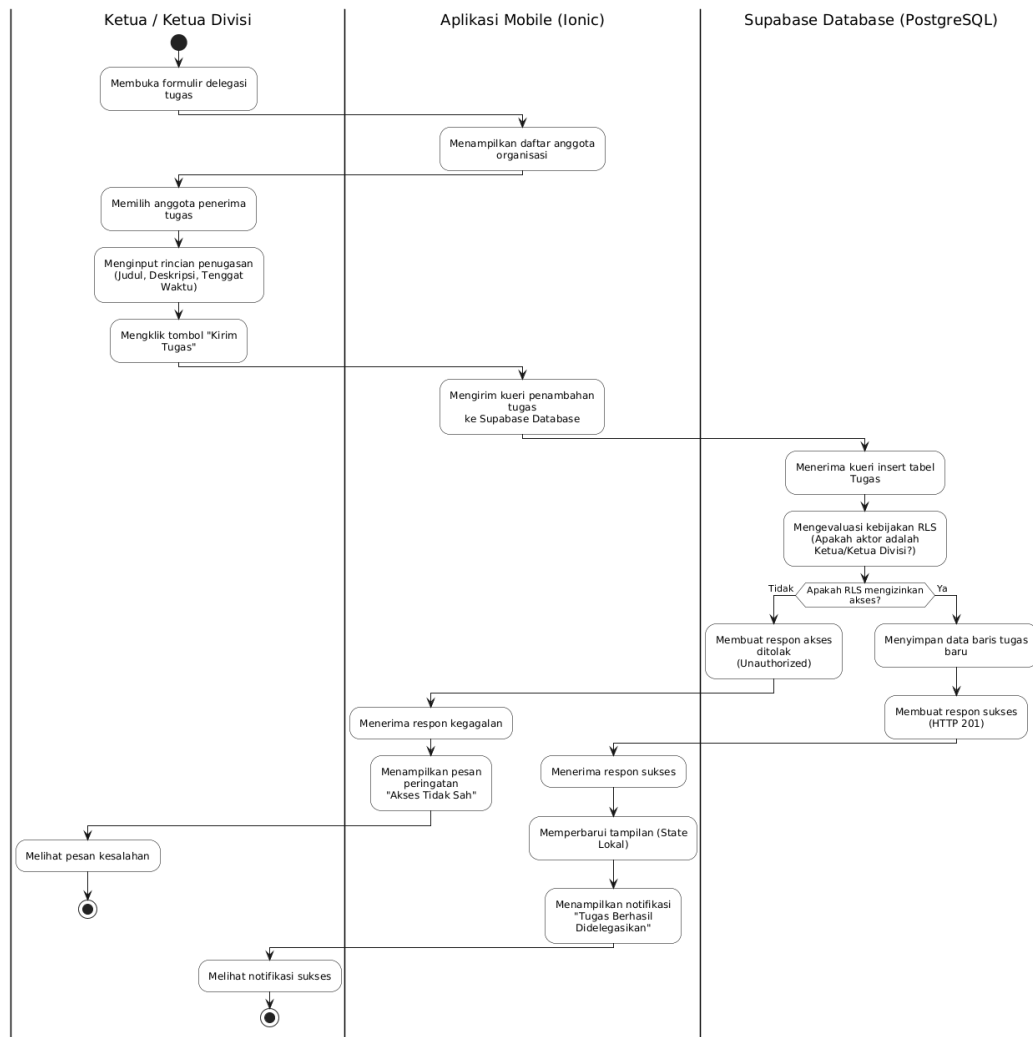


Gambar 4.24 Activity Diagram Kelola Event

Gambar 4.24 menggambarkan Activity Diagram untuk pengelolaan acara (Event). Alur ini dikhususkan bagi pengguna dengan peran Ketua.

Proses dimulai saat Ketua membuka menu Manajemen *Event* dan memilih aksi mutasi data (Tambah, Ubah, atau Hapus). Ketua kemudian mengisi rincian *event* termasuk nama, tanggal, deskripsi, dan poster acara. Jika terdapat gambar poster, aplikasi akan melakukan kompresi dan mengunggahnya ke *Storage*. Setelah *URL* poster didapatkan, aplikasi mengirimkan kueri mutasi ke *Supabase Database*. *Server* kemudian mengevaluasi kebijakan *RLS*, jika Ketua berwenang, maka data akan disimpan dan aplikasi menampilkan notifikasi "*Event Berhasil Disimpan*". Jika tidak berwenang, sistem akan menolak kueri dan menampilkan pesan kegagalan.

2) Delegasi Tugas

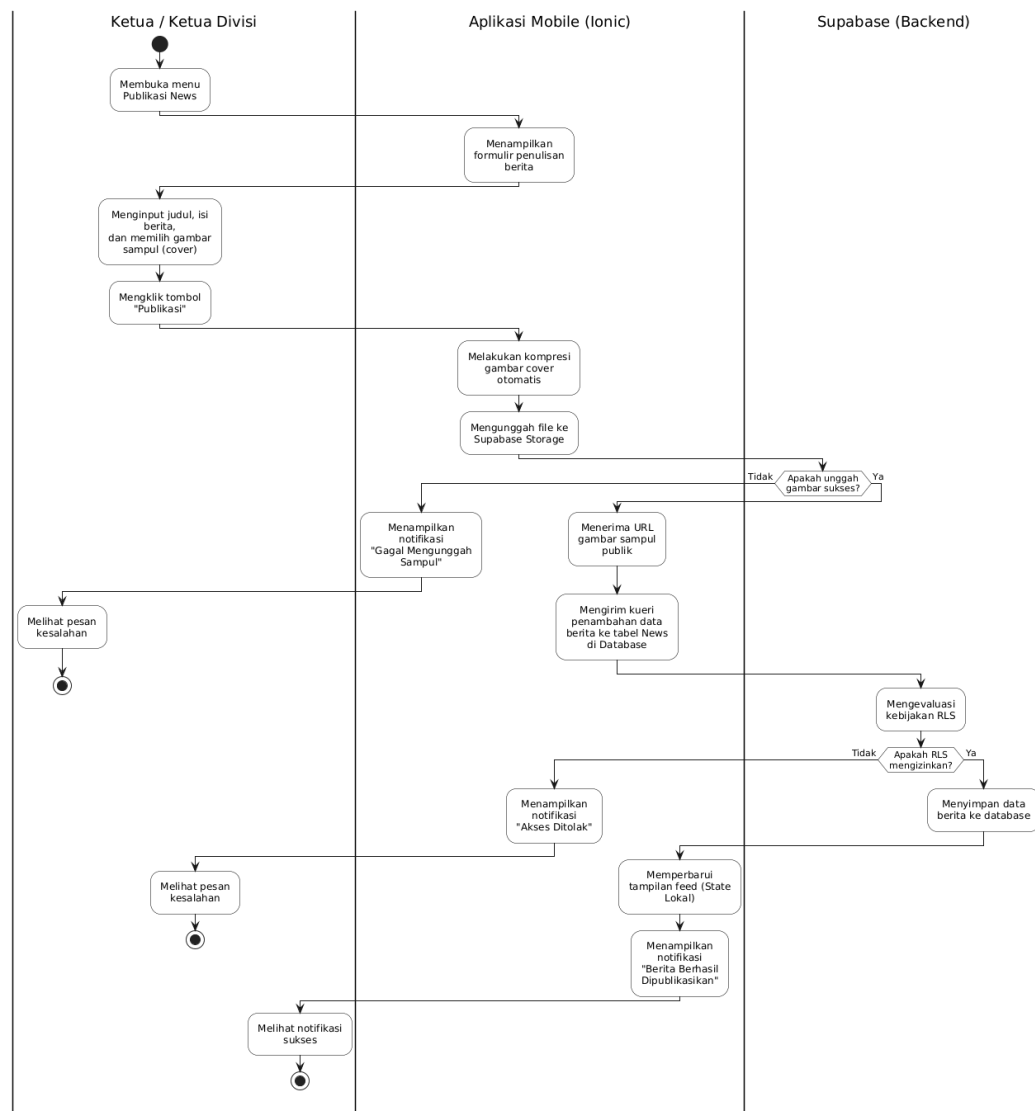


Gambar 4.25 Activity Diagram Delegasi Tugas

Gambar 4.25 menunjukkan alur *Activity Diagram* untuk fitur Delegasi Tugas yang dapat dilakukan oleh Ketua atau Ketua Divisi. Proses diawali dengan membuka formulir delegasi, di mana sistem akan menampilkan daftar anggota aktif. Pembuat tugas kemudian memilih anggota penerima tugas, menginputkan rincian penugasan, tenggat waktu, dan mengeklik "Kirim Tugas". Kueri penambahan data akan dikirimkan ke *database*. Setelah evaluasi kebijakan otorisasi (*RLS*) di sisi *database* berhasil dilewati, data baris tugas baru akan direkam, lalu sistem mengembalikan pesan

sukses sehingga aplikasi *mobile* dapat memperbarui daftar tugas dan memunculkan notifikasi keberhasilan delegasi.

3) Kelola News

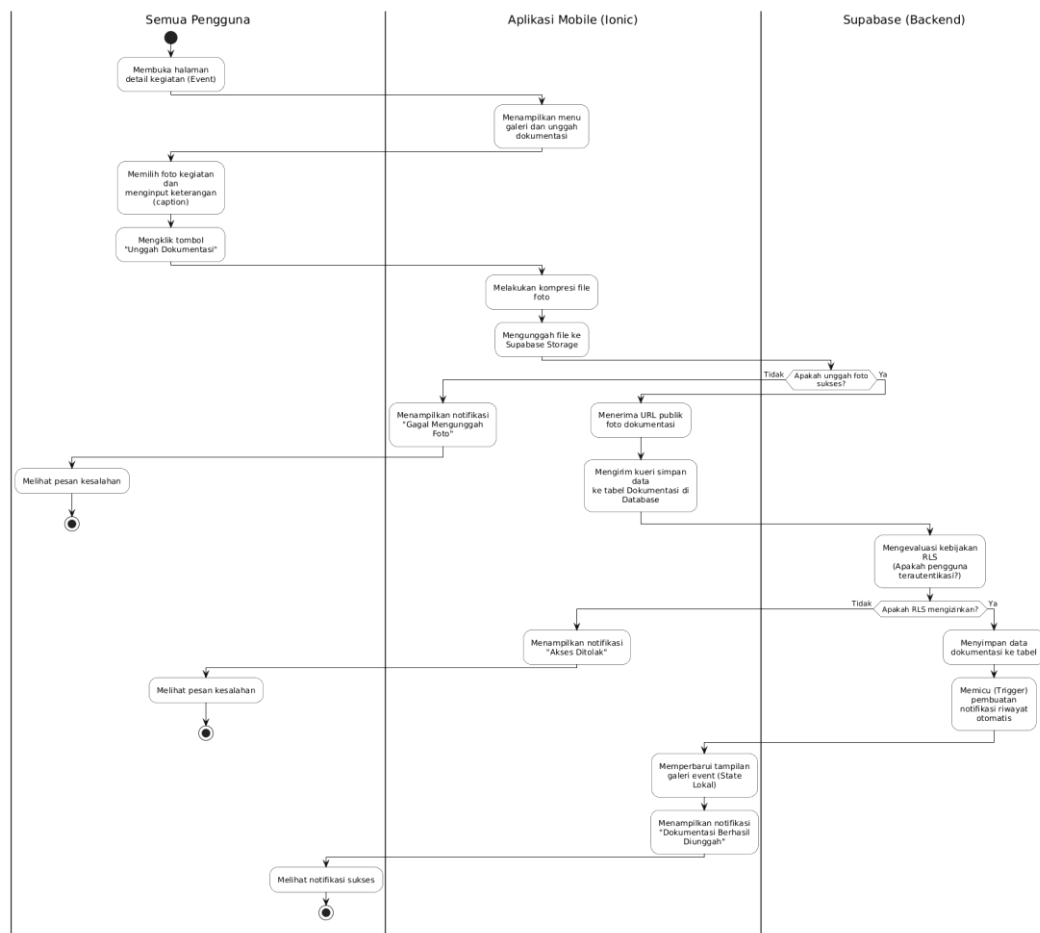


Gambar 4.26 Activity Diagram Kelola News

Gambar 4.26 menguraikan alur *Activity Diagram* pada publikasi berita (News). Mirip dengan pengelolaan *event*, alur ini diawali oleh Ketua/Ketua Divisi yang menyusun judul, isi berita, dan memilih gambar sampul (cover).

Sistem secara otomatis mengompresi gambar dan mengunggahnya ke *Supabase Storage*. Setelah link gambar publik diperoleh, aplikasi menggabungkan data teks dan link tersebut menjadi satu kueri *insert* ke tabel News di *database*. Pasca evaluasi *RLS* oleh sistem, data berita tersimpan dan *state* antarmuka aplikasi diperbarui agar berita langsung muncul pada feed pengguna.

4) Dokumentasi kegiatan *Event*

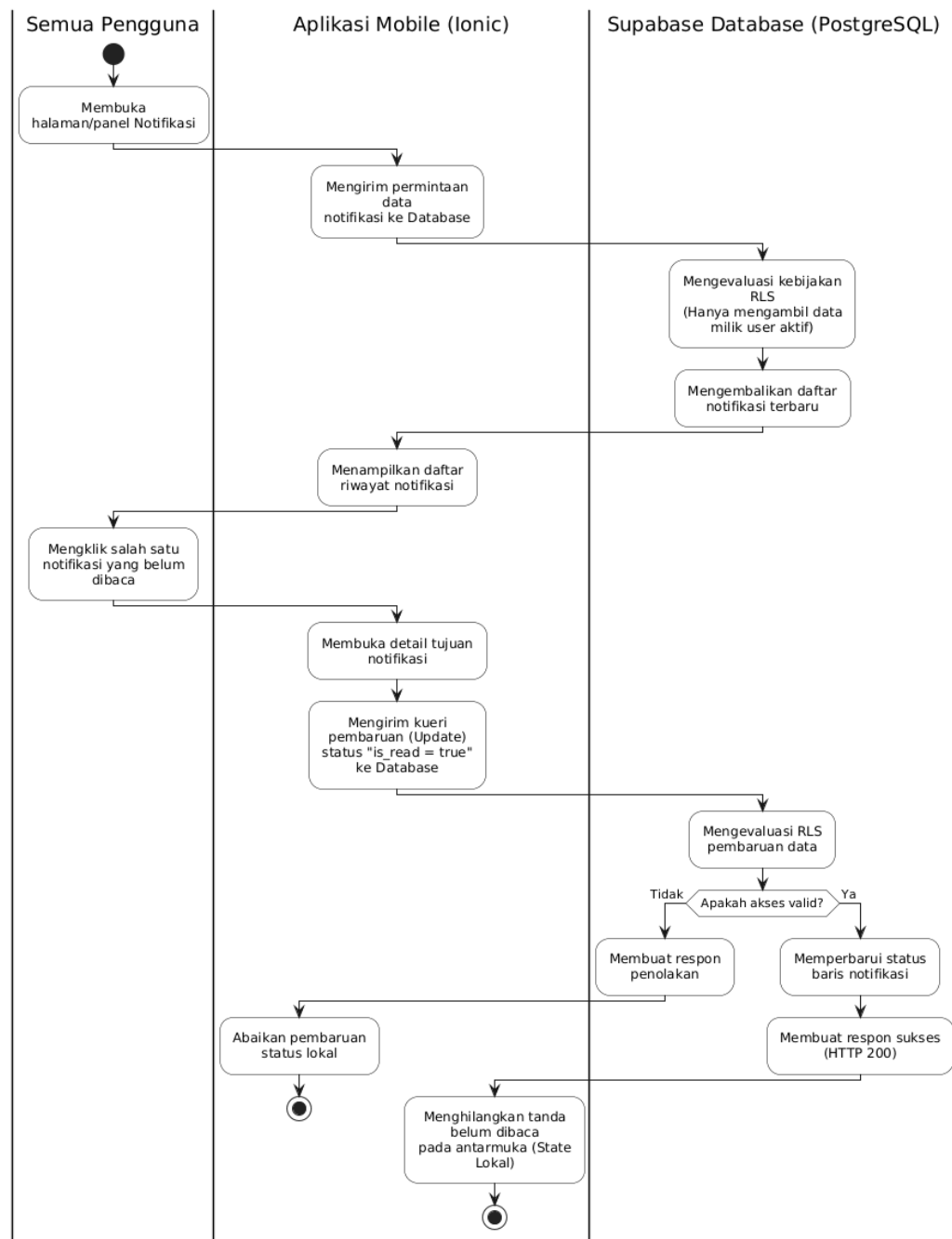


Gambar 4.27 Activity Diagram Dokumentasi *Event*

Gambar 4.27 menjelaskan Activity Diagram pengunggahan foto dokumentasi pasca-acara yang dapat dilakukan oleh Semua Pengguna yang

hadir. Pengguna membuka detail *event* terkait, memilih foto dokumentasi, menyisipkan caption, lalu menekan tombol "Unggah". Foto dikompresi dan dikirim ke *Supabase Storage*. *URL* foto yang dihasilkan dikirim melalui kueri simpan ke tabel Dokumentasi di basis data. Setelah evaluasi keamanan *RLS* berhasil, data dokumentasi disimpan. Poin krusial pada alur ini adalah penyimpanan data dokumentasi tersebut akan memicu eksekusi *Database Trigger* di *backend* untuk secara otomatis meracik dan mengirimkan riwayat notifikasi baru kepada seluruh pihak yang relevan

5) Kelola Notifikasi



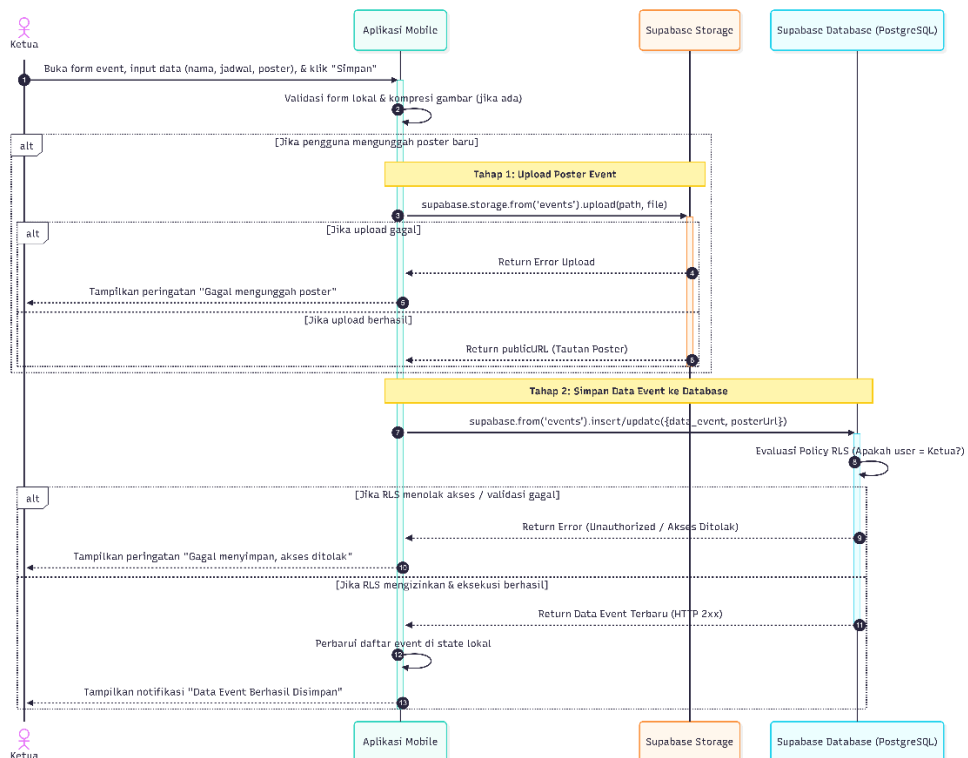
Gambar 4.28 Activity Diagram Kelola Notifikasi

Gambar 4.28 memvisualisasikan alur pengelolaan Notifikasi untuk semua level pengguna. Saat pengguna membuka panel notifikasi, aplikasi meminta data ke basis data. *Server* menerapkan evaluasi *RLS* agar kueri hanya mengambil (*SELECT*) notifikasi milik pengguna yang sedang aktif

(terautentikasi) guna menjaga privasi. Sistem lalu mengembalikan daftar notifikasi. Jika pengguna mengeklik salah satu item notifikasi untuk membacanya, aplikasi akan mengirimkan kueri pembaruan status (*is_read* = *true*) ke basis data. Setelah berhasil diproses *server*, sistem merespons dengan menghilangkan tanda belum dibaca di antarmuka lokal pengguna.

b) Sequence Diagram

1) Kelola Event

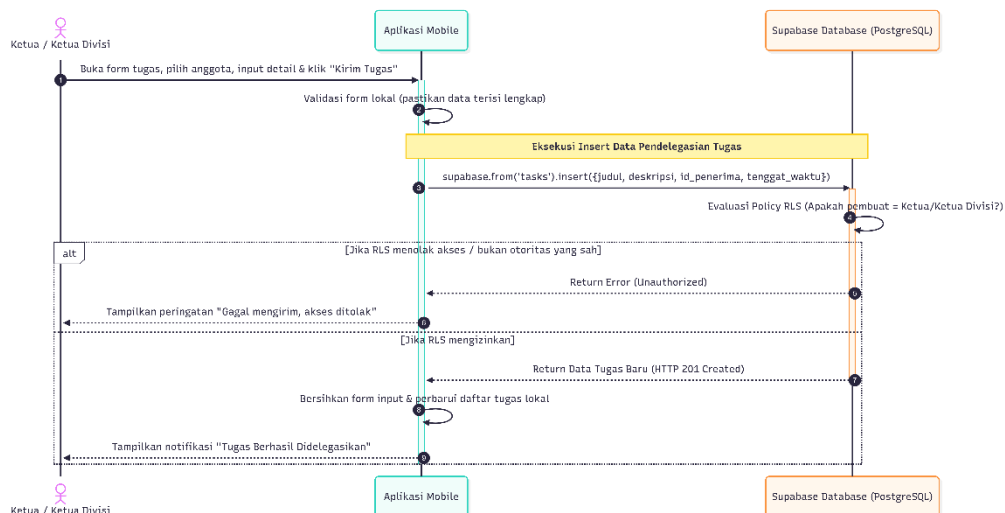


Gambar 4.29 Sequence Diagram Kelola Event

Gambar 4.29 menjabarkan *Sequence Diagram* pada proses pengelolaan acara. Alur teknis ini melibatkan peran Ketua, aplikasi *mobile*, *Supabase Storage*, dan *Supabase Database*. Setelah form divalidasi dan gambar dikompresi di sisi aplikasi, proses pertama adalah pengunggahan poster

event ke *Supabase Storage*. Bila berhasil, *link* poster publik akan disisipkan bersama parameter *event* lainnya dalam kueri *insert* ke *database*. Lapisan keamanan *Row Level Security (RLS)* di *server* akan memverifikasi sesi akses pengguna. Bila pengguna tervalidasi sebagai Ketua, data akan direkam (*HTTP 201 Created*) dan aplikasi akan menyinkronkan tampilan *state* lokal secara real-time.

2) Delegasi Tugas

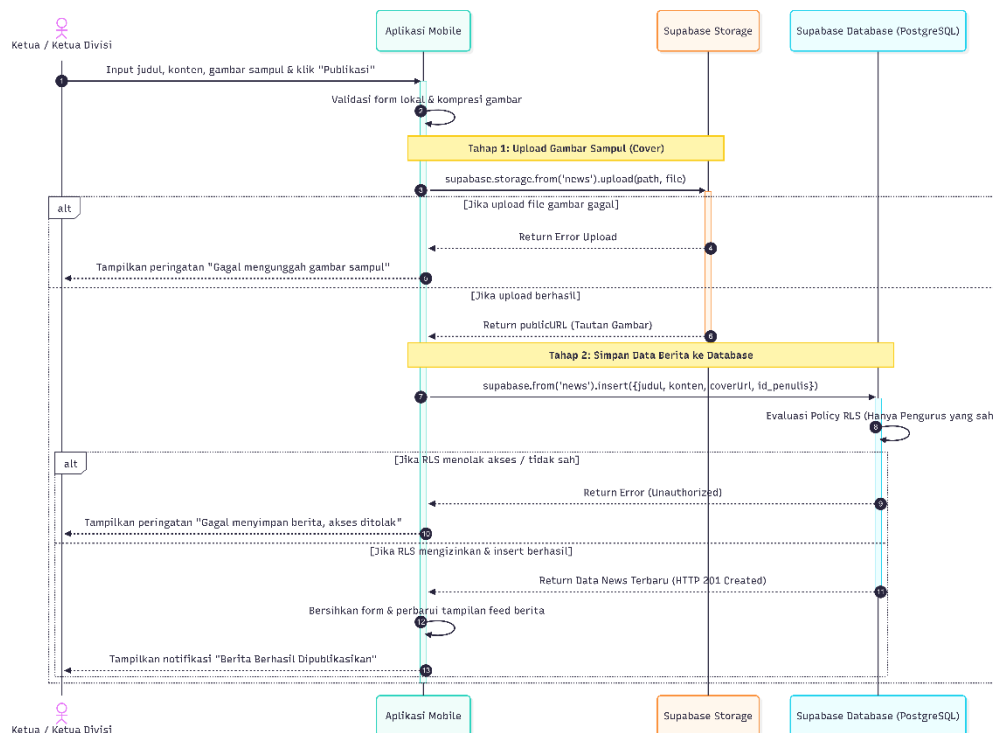


Gambar 4.30 *Sequence Diagram* Delegasi Tugas

Gambar 4.30 memvisualisasikan *Sequence Diagram* pendistribusian tugas. Tanpa melibatkan layanan *Storage*, interaksi langsung difokuskan pada pengiriman kueri *insert* ke *database* setelah Ketua/Ketua Divisi memilih anggota dan mengisi rincian tugas. *Supabase Database* akan mengeksekusi *Policy RLS* untuk memastikan kueri dikirim oleh pembuat yang memiliki wewenang (Ketua/Ketua Divisi). Apabila berhasil, *Supabase*

merespons dengan status sukses dan aplikasi *mobile* langsung membersihkan *input* serta menambahkan baris tugas baru pada antarmuka.

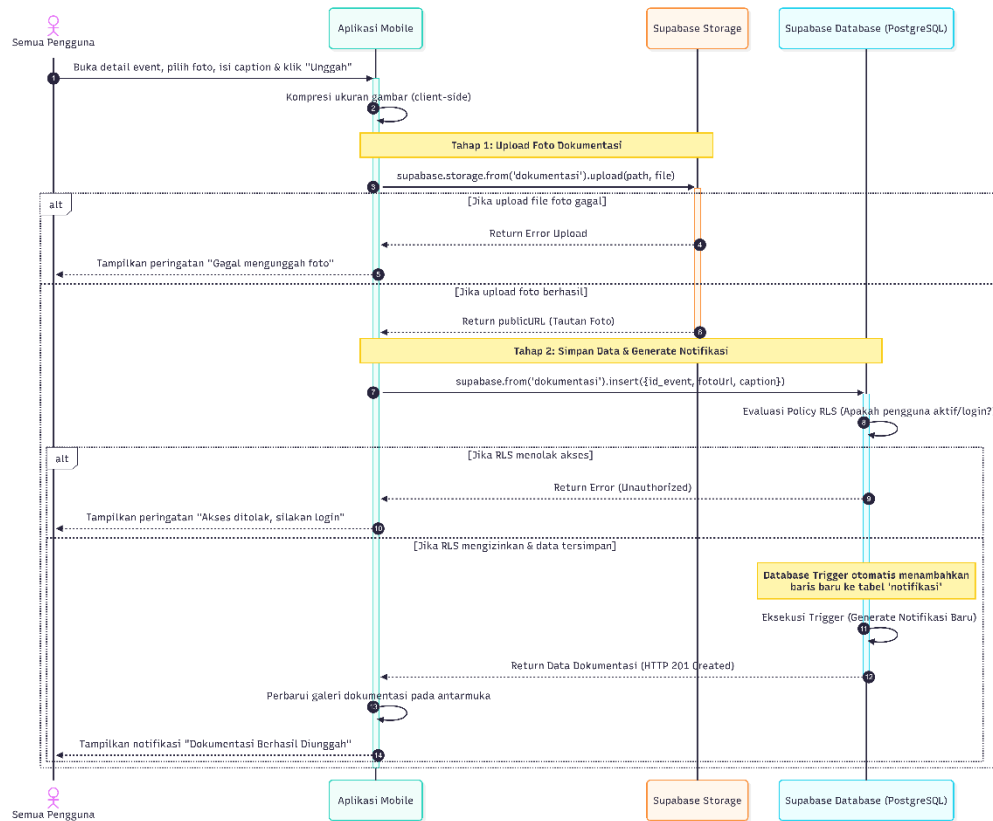
3) Kelola News



Gambar 4.31 *Sequence Diagram* Kelola News

Gambar 4.31 menunjukkan *Sequence Diagram* siklus publikasi berita. Skema yang digunakan hampir identik dengan kelola *Event*, di mana alur dibagi menjadi dua tahap transaksi *server*: pengunggahan gambar sampul (*cover*) ke *Supabase Storage*, dan penyisipan data teks berita ke *Supabase Database*. Mekanisme perlindungan data menggunakan *RLS* juga tetap dieksekusi oleh *server* untuk menangkal potensi modifikasi data dari luar pihak yang berwenang (misalnya anggota biasa yang mencoba meretas masuk).

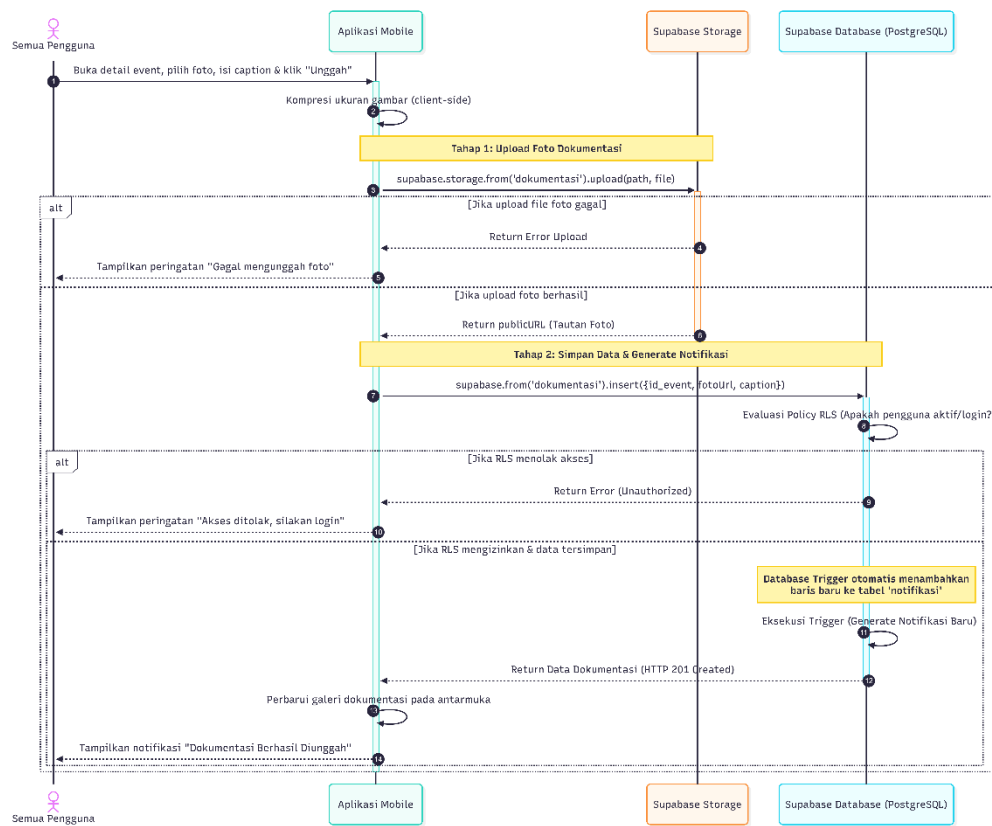
4) Dokumentasi kegiatan *Event*



Gambar 4.32 *Sequence Diagram* Dokumentasi *Event*

Gambar 4.32 merepresentasikan *Sequence Diagram* proses pengarsipan foto dokumentasi yang bersifat terbuka bagi Semua Pengguna. Alur dimulai dengan kompresi *client-side*, lalu *upload* ke *Supabase Storage*. Kueri penyisipan *URL* foto dikirim ke *Supabase Database*. Di tahap ini, *Policy RLS* yang dievaluasi lebih longgar, yakni hanya mewajibkan pengguna dalam kondisi *login* (*authenticated*) tanpa memandang jabatannya.

5) Kelola Notifikasi

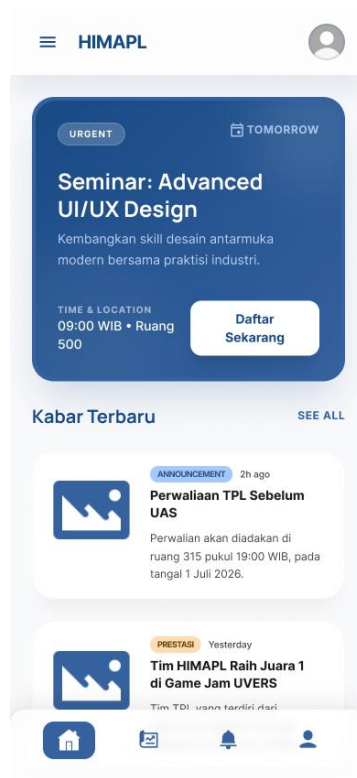


Gambar 4.33 *Sequence Diagram* Kelola notifikasi

Gambar 4.33 secara khusus menyoroti kelanjutan *Sequence Diagram* dari fungsionalitas notifikasi yang bekerja secara terintegrasi dengan modul Dokumentasi. Diagram ini menegaskan bagaimana arsitektur *backend* mengambil alih peran otomatisasi. Ketika kueri dokumentasi berhasil tersimpan, *database* mengeksekusi sebuah fungsi *Database Trigger*. Fungsi *Trigger* ini berjalan independen di lapisan *server* untuk meracik dan melakukan penambahan baris baru (*insert*) ke tabel notifikasi, sehingga aplikasi *mobile* tidak perlu mengirimkan dua permintaan *API* terpisah untuk menciptakan riwayat pemberitahuan.

c. Desain (*Figma*)

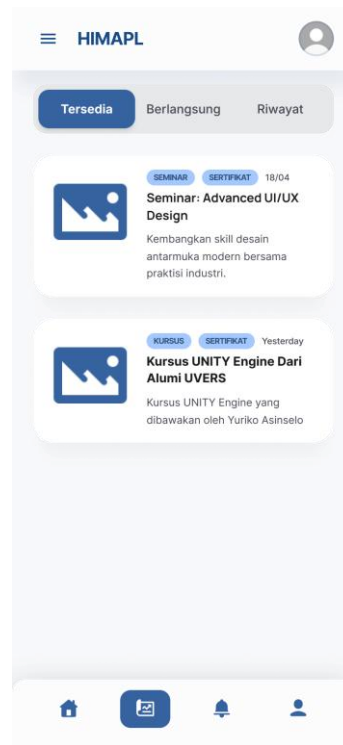
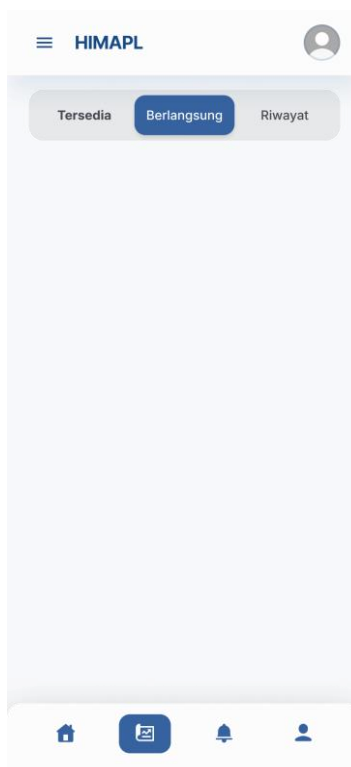
1) Halaman Beranda

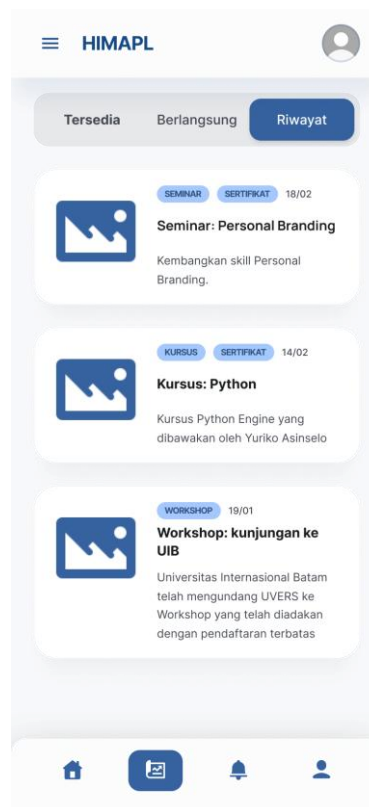


Gambar 4.34 Desain halaman beranda

Gambar 4.34 menampilkan rancangan dashboard antarmuka utama (*Home*). Elemen prioritas tertinggi diposisikan pada area atas layar (*hero section*), seperti kartu sorotan kegiatan mendesak (*urgent*). Pada bagian bawahnya, disusun daftar berita atau informasi (Kabar Terbaru) dalam pola daftar memanjang secara vertikal, yang didukung dengan tata letak minimalis dan *clean UI*.

2) Halaman *Event*

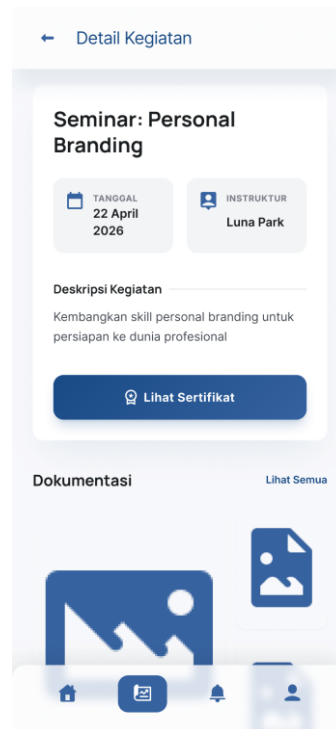
Gambar 4.35 Desain halaman *event sub-tab1*Gambar 4.36 Desain halaman *event sub-tab2*



Gambar 4.37 Desain halaman *event sub-tab2*

Gambar 4.35 hingga Gambar 4.37 memperlihatkan *wireframe* halaman navigasi *Event*. Untuk mengatasi penumpukan informasi, desain ini mengandalkan pendekatan *tab* bar horizontal yang memfilter daftar kegiatan secara dinamis ke dalam *sub-kategori* Tersedia, Berlangsung, dan Riwayat yang diubah ke bahasa Inggris menjadi *Available*, *Upcoming*, dan *Completed*. Masing-masing kegiatan diwakili oleh kartu *visual* dengan penempatan khusus untuk label kategori dan tanggal.

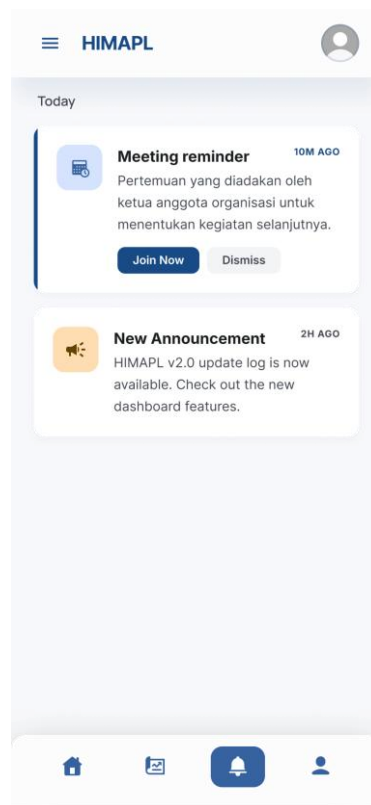
3) Template Halaman Detail



Gambar 4.38 Desain *template* halaman detail

Gambar 4.38 adalah cetak biru (blueprint) antarmuka template untuk menayangkan detail informasi dari *event* atau news. Antarmuka dirancang hierarkis: diawali gambar *header* berukuran besar, judul tebal, deretan label atribut utama (seperti tempat dan waktu), paragraf deskripsi informatif, serta area penempatan galeri arsip dokumentasi di sisi bawah layar.

4) Halaman Notifikasi

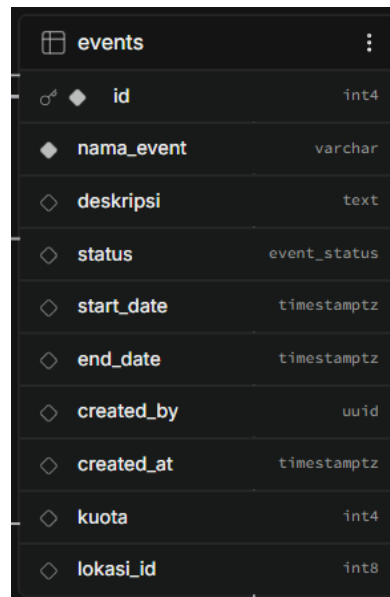


Gambar 4.39 Desain halaman notifikasi

Gambar 4.39 menggambarkan antarmuka berbasis daftar (*list view*) untuk menampilkan rentetan pemberitahuan. Tiap kotak riwayat notifikasi dirancang dengan *layout* teks tebal untuk judul dan teks tipis pendukung di bawahnya, dilengkapi label jejak waktu (*timestamp*) untuk menginformasikan kapan pesan tersebut tiba secara kronologis.

d. Desain Skema Basis Data (*Supabase*)

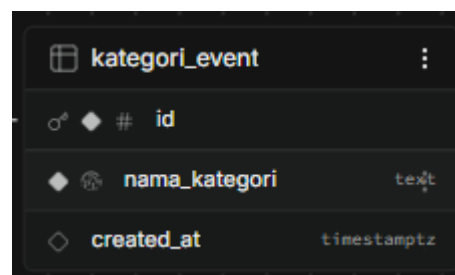
1) *Database Events*



events		
id	int4	Primary Key
nama_event	varchar	
deskripsi	text	
status	event_status	
start_date	timestampz	
end_date	timestampz	
created_by	uuid	
created_at	timestampz	
kuota	int4	
lokasi_id	int8	

Gambar 4.40 *events database*

Gambar 4.40 menampilkan skema tabel *events* yang berperan sebagai entitas induk (*master table*) untuk manajemen acara. Tabel sentral ini menampung seluruh atribut inti dari sebuah kegiatan, mencakup kolom *id* unik bertipe *integer*, *nama_event*, *deskripsi*, indikator status acara (*status*), parameter waktu pelaksanaan (*start_date* dan *end_date*), relasi *ID* pembuat kegiatan (*created_by*), serta batasan jumlah peserta (*kuota*).

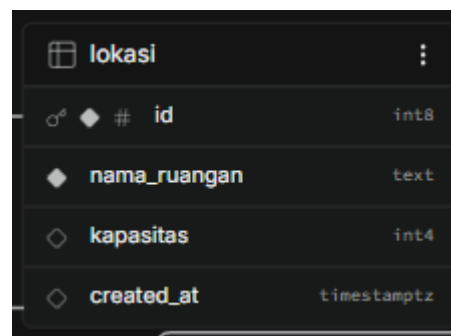


kategori_event		
id	text	Primary Key
nama_kategori	text	
created_at	timestampz	

Gambar 4.41 *kategori_event database*

Gambar 4.41 memperlihatkan struktur tabel referensi *kategori_event*. Tabel ini dirancang secara terpisah dan minimalis, memuat kolom *id* dan *nama_kategori*. Tujuannya adalah untuk membakukan label jenis acara

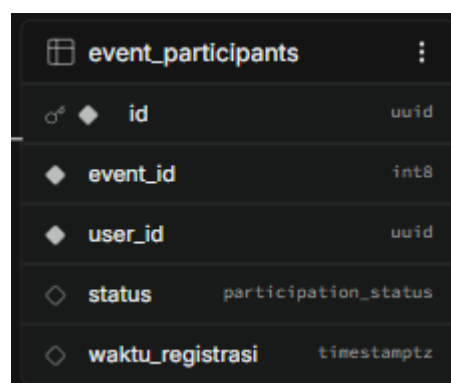
(seperti: Seminar, Sertifikat, *Project*, *Workshop*, Rapat) sehingga mencegah terjadinya kesalahan ketik ganda (typo redundansi) dan mempermudah algoritma aplikasi saat pengguna melakukan penyaringan (filtering) data *event*.



lokasi	
# id	int8
nama_ruangan	text
kapasitas	int4
created_at	timestampz

Gambar 4.42 lokasi *database*

Gambar 4.42 menunjukkan tabel lokasi yang mendata ketersediaan ruang—baik fisik di kampus maupun ruang virtual. Tabel ini mendeskripsikan atribut nama_ruangan beserta daya tampung maksimalnya pada kolom kapasitas. Data pada tabel ini berelasi dengan tabel *events* melalui rujukan kunci tamu (*Foreign Key*) pada kolom lokasi_id.



event_participants	
# id	uuid
event_id	int8
user_id	uuid
status	participation_status
waktu_registrasi	timestampz

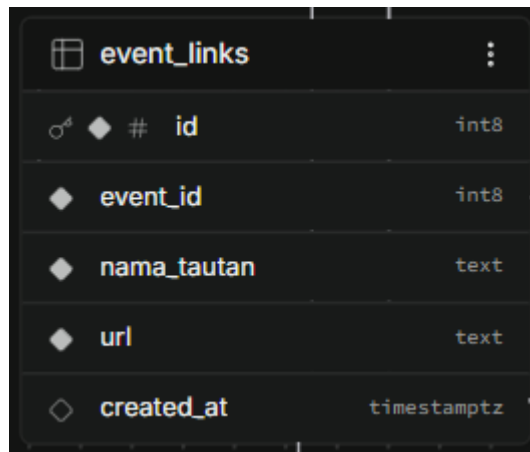
Gambar 4.43 event_participants *database*

Gambar 4.43 menggambarkan tabel transaksional *event_participants* yang secara spesifik difungsikan untuk merekam lalu lintas pendaftaran anggota ke sebuah acara. Struktur tabel ini mengikat kolom *event_id* dan *user_id*, sehingga sistem mengetahui anggota mana yang mendaftar pada acara apa. Dilengkapi juga dengan kolom status partisipasi (misalnya: *registered*, *attended*) serta *timestamp* pendaftaran (*waktu_registrasi*).

event_kategori_mapping	
id	int8
event_id	int8
kategori_id	int8
	int8

Gambar 4.44 *event_kategori_mapping database*

Gambar 4.44 mendokumentasikan tabel jembatan (junction table) bernama *event_kategori_mapping*. Tabel ini diciptakan khusus untuk meresolusi bentuk relasi *Many-to-Many*, karena pada aplikasi ini satu *event* bisa masuk ke dalam beberapa kategori sekaligus, begitu pula sebaliknya. Kolom utamanya murni berisi persilangan antara *event_id* dan *kategori_id*.

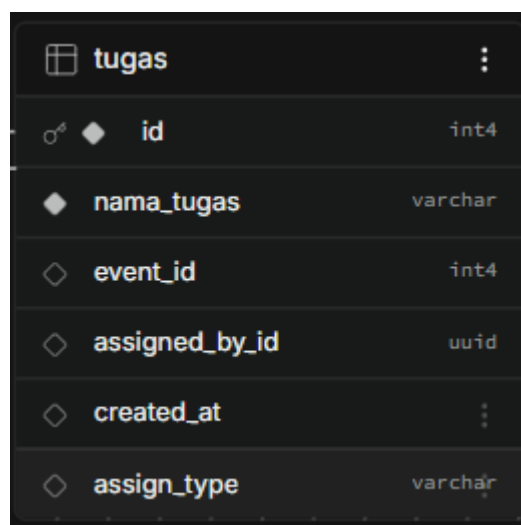


event_links	
id	int8
event_id	int8
nama_tautan	text
url	text
created_at	timestampz

Gambar 4.45 *event_links database*

Gambar 4.45 menampilkan struktur tabel *event_links*. Tabel ini dirancang untuk mengakomodasi fleksibilitas materi acara dengan menampung lampiran tautan *URL* eksternal pendukung (seperti tautan *Google Meet*, tautan *form feedback*, atau tautan sertifikat). Kolom penyusunnya meliputi *nama_tautan* sebagai label tombol yang muncul di aplikasi, tipe *url* sebagai tujuan tautan, serta *Foreign Key event_id*.

2) Database Tugas



tugas	
id	int4
nama_tugas	varchar
event_id	int4
assigned_by_id	uuid
created_at	
assign_type	varchar

Gambar 4.46 *tugas database*

Gambar 4.46 menampilkan skema untuk tabel tugas. Tabel ini berfungsi untuk menyimpan deskripsi atau instruksi dari sebuah beban pekerjaan yang diciptakan oleh pengurus. Di dalamnya terdapat kolom *id* bertipe *integer* yang bersifat unik, *nama_tugas* yang memuat deskripsi instruksi kerja, serta *event_id* bertipe *Foreign Key* yang memastikan bahwa setiap tugas selalu terikat dan relevan dengan satu acara tertentu. Selain itu, terdapat kolom *assigned_by_id* untuk mencatat identitas Ketua/Ketua Divisi yang memberikan instruksi, dan *assign_type* untuk mengklasifikasikan ruang lingkup penugasan.

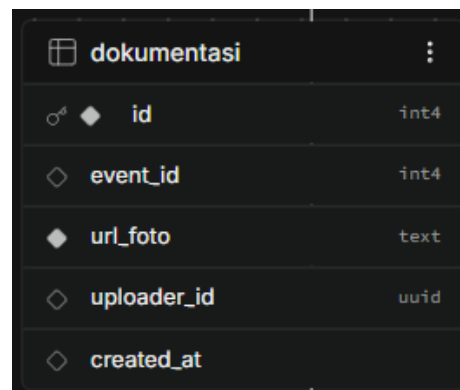
Column Name	Data Type	Constraints
id	int8	Primary Key
tugas_id	int8	
user_id	uuid	
status_selesai	bool	
created_at	timestampz	

Gambar 4.47 tugas_assignments database

Gambar 4.47 memperlihatkan struktur tabel *tugas_assignments* yang bertindak sebagai tabel jembatan relasional untuk distribusi beban kerja. Jika tabel tugas menyimpan instruksinya, tabel inilah yang mendistribusikan instruksi tersebut ke anggota. Struktur tabel ini mengalokasikan referensi kolom *tugas_id* kepada anggota spesifik melalui tautan *user_id*. Tabel ini memiliki peran penting pada antarmuka aplikasi karena dilengkapi dengan kolom boolean *status_selesai* (dengan parameter *True/False*), yang

berfungsi memvalidasi apakah tugas dari pengurus telah rampung dikerjakan oleh anggota yang bersangkutan.

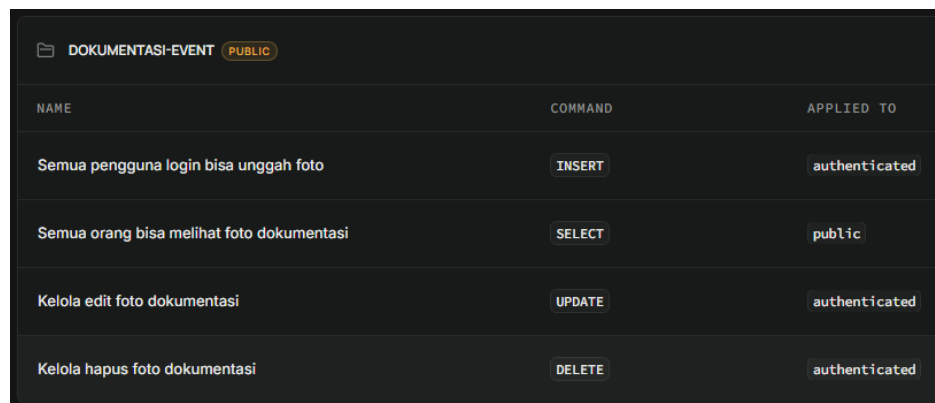
3) Database Dokumentasi



dokumentasi	
id	int4
event_id	int4
url_foto	text
uploader_id	uuid
created_at	

Gambar 4.48 dokumentasi *database*

Gambar 4.48 menampilkan skema dari tabel dokumentasi yang dirancang khusus untuk pengarsipan pasca-acara. Setiap baris data dalam tabel ini mewakili satu *file* foto dokumentasi. Tabel ini disusun dengan kolom *id* unik, dan memiliki *Foreign Key* pada kolom *event_id* yang berfungsi memastikan setiap foto secara presisi terikat pada *ID* acara tertentu. Informasi lokasi *file* gambar disimpan dalam bentuk teks *link* pada kolom *url_foto*. Selain itu, terdapat kolom *uploader_id* bertipe *UUID* yang merujuk pada tabel *users*, berguna untuk melacak identitas anggota yang mempublikasikan dokumentasi tersebut.

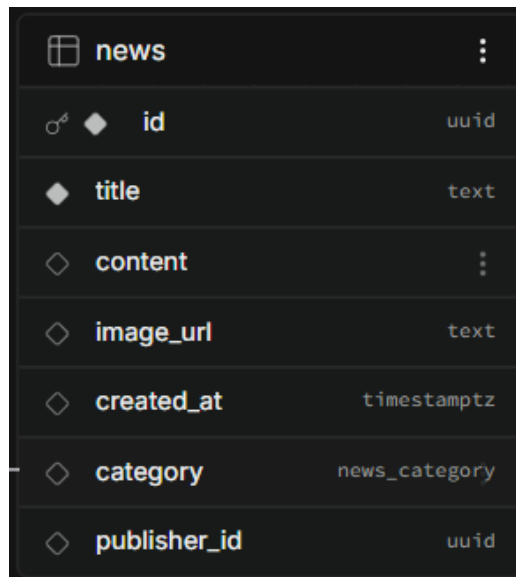


NAME	COMMAND	APPLIED TO
Semua pengguna login bisa unggah foto	INSERT	authenticated
Semua orang bisa melihat foto dokumentasi	SELECT	public
Kelola edit foto dokumentasi	UPDATE	authenticated
Kelola hapus foto dokumentasi	DELETE	authenticated

Gambar 4.49 storage bucket dokumentasi-event

Gambar 4.49 menunjukkan panel pengaturan *Storage* pada *Supabase* untuk *bucket* yang diberi nama *dokumentasi-event*. Tangkapan layar ini memperlihatkan secara jelas konfigurasi tingkat keamanan (*Policies*) terhadap aset *visual*. Bucket ini dikunci agar hak penambahan data (*INSERT*), penyuntingan (*UPDATE*), dan penghapusan (*DELETE*) hanya dapat dieksekusi oleh pihak internal, yaitu pengguna dengan status *authenticated* (telah terverifikasi masuk ke sistem). Sebaliknya, hak untuk membaca data (*SELECT*) dibiarkan *public* (terbuka) agar *file* gambar dokumentasi tersebut bebas ditampilkan ke layar antarmuka pengguna tanpa hambatan.

4) Database News

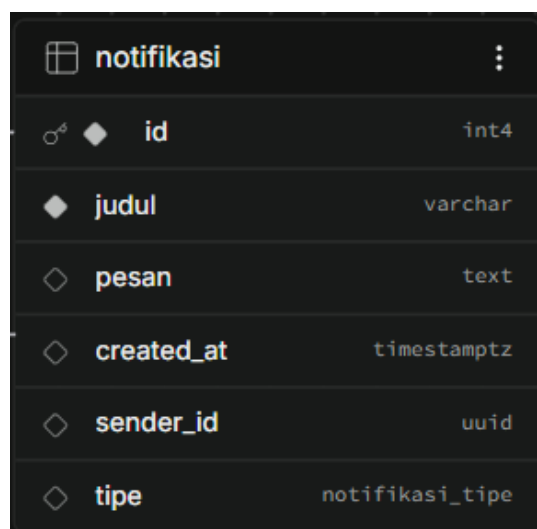


news	
id	uuid
title	text
content	text
image_url	text
created_at	timestamp
category	news_category
publisher_id	uuid

Gambar 4.50 news database

Gambar 4.50 menampilkan skema tabel *news* yang independen, mencakup kolom untuk merekam *title*, muatan teks *content*, penanda unik pembuat *publisher_id*, serta kolom penampung nama kategori dan struktur URL gambar (*image_url*).

5) Database Notifikasi



notifikasi	
id	int4
judul	varchar
pesan	text
created_at	timestamp
sender_id	uuid
tipe	notifikasi_tipe

Gambar 4.51 notifikasi database

Gambar 4.51 memperlihatkan struktur skema dari tabel induk notifikasi. Tabel sentral ini berfungsi sebagai penampung cetakan dasar (template) sebuah pemberitahuan sebelum disebar ke anggota. Kolom-kolom pembangunnya meliputi *id* bertipe *integer* unik, judul sebagai tajuk utama, teks panjang pada kolom pesan, waktu perilsan pesan (*created_at*), kolom *sender_id* untuk mendata siapa pengurus yang memicu keluarnya notifikasi tersebut, serta pengklasifikasian tipe notifikasi.

penerima_notifikasi	
id	int8
notif_id	int8
user_id	uuid
is_read	bool
created_at	timestampz

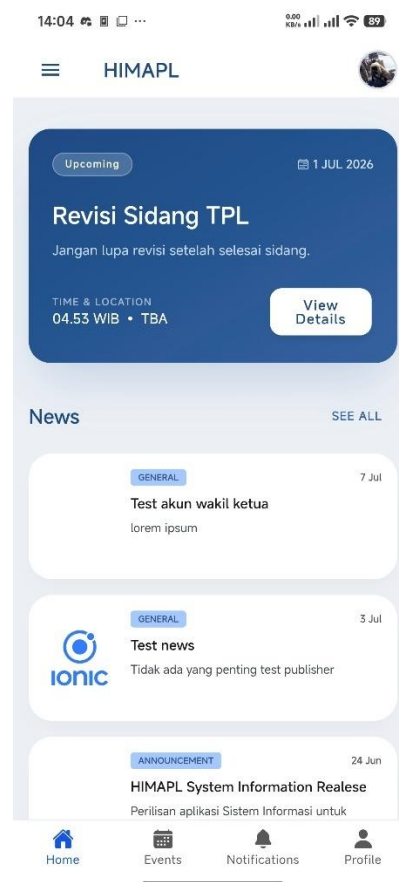
Gambar 4.52 penerima_notifikasi database

Gambar 4.52 menampilkan skema tabel relasional sekunder bernama penerima_notifikasi. Tabel ini menjadi nyawa dari efisiensi arsitektur penyiaran pesan massal (*broadcast*). Dibandingkan menggandakan kalimat pesan yang sama berulang-ulang, sistem hanya menyisipkan referensi *notif_id* dari tabel induk ke sini, kemudian menyilangkannya dengan *ID* sasaran pada kolom *user_id*. Kunci dari tabel ini terletak pada kolom *is_read* bertipe boolean (kondisi *True/False*), yang berfungsi independen mencatat

state jejak digital apakah notifikasi tersebut sudah diklik (dibaca) atau belum oleh si penerima.

e. Hasil Implementasi

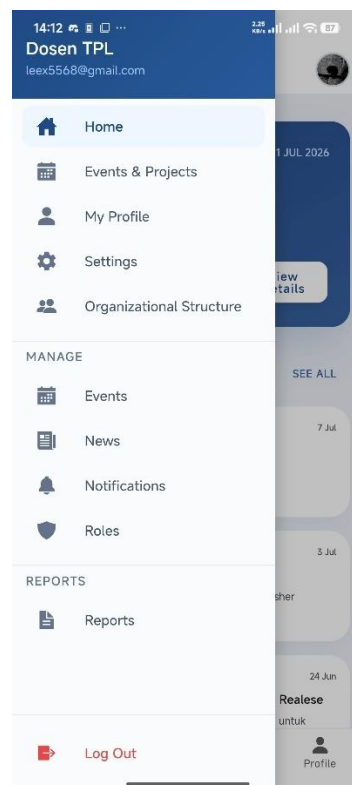
1) Halaman Beranda



Gambar 4.53 Halaman Beranda

Gambar 4.53 memperlihatkan wujud nyata dari Halaman Beranda (*Home*) pada aplikasi Android. Secara *visual*, tata letaknya berhasil direalisasikan sangat konsisten dengan rancangan *wireframe Figma* (Gambar 4.34). Perubahannya terletak pada penyajian data; aplikasi tidak lagi menampilkan teks *dummy* (contoh), melainkan *render* data operasional secara *real-time*. Pada bagian atas, kartu *Upcoming* menarik data jadwal

mendesak melalui kueri *SELECT* dengan pengurutan tanggal dari tabel *events*. Sementara itu, di area tengah, daftar News secara dinamis mengambil kumpulan berita dari tabel *news*, disertai pencantuman nama penulisnya yang didapat dari relasi antar-tabel (*Join*) ke tabel *users*.

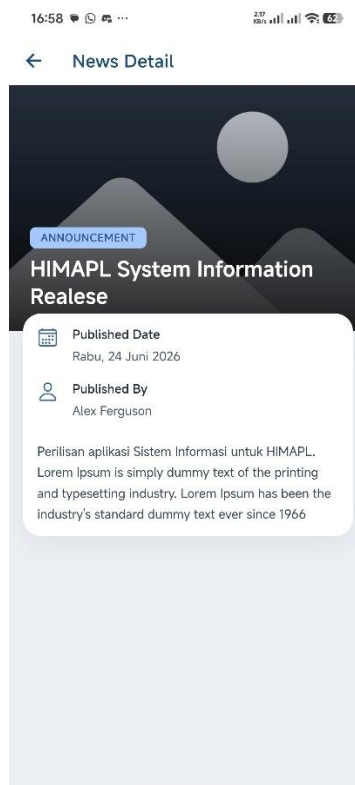


Gambar 4.54 Side Menu

Gambar 4.54 menunjukkan panel laci navigasi samping (Side Menu). Meskipun desain laci navigasi ini tidak didefinisikan secara khusus pada desain *wireframe Figma* sebelumnya, komponen ini diimplementasikan menggunakan pustaka bawaan *Ionic* guna mengakomodasi padatnya jalur menu administratif. Hubungannya dengan *database* sangat kuat: aplikasi memuat opsi menu pada laci ini berdasarkan logika *Conditional Rendering* (penyembunyian elemen). Sistem membaca atribut *role* milik pengguna

yang tersimpan di sesi lokal (berasal dari tabel *users* di *Supabase*), sehingga menu-menu sensitif di bagian "*MANAGE*" dan "*REPORTS*" hanya akan muncul di layar jika pengguna berstatus Ketua, Ketua Divisi, atau Dosen.

2) Halaman *Detail News*

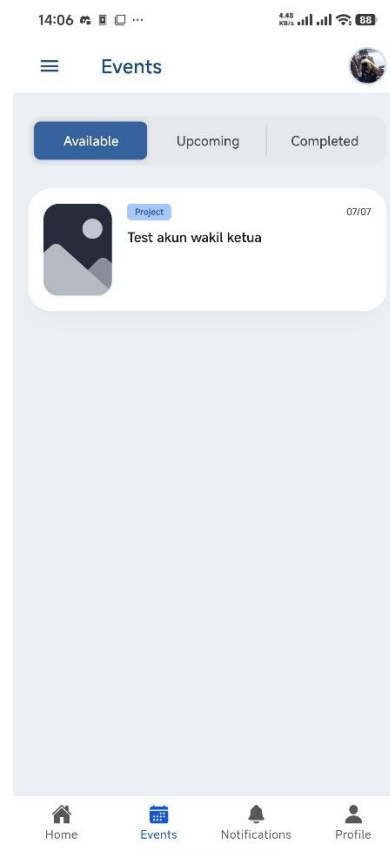


Gambar 4.55 Halaman *News Detail*

Gambar 4.55 menyajikan Halaman Detail *News*. Halaman ini dikembangkan dengan mengadopsi cetak biru (*blueprint*) dari *Template Halaman Detail* di *Figma* (Gambar 4.38). Karena template aslinya ditujukan untuk *Event*, pada implementasinya untuk *News*, pengembang melakukan penyesuaian (*layout adjustment*) dengan membuang ikon *widget* yang tidak perlu (seperti letak peta lokasi atau label sertifikat) dan mengutamakan penyajian teks artikel penuh. Secara fungsional sistem membaca *ID* berita

yang ditekan, lalu mengirim kueri spesifik ke tabel *news* untuk memanggil teks panjang (*content*) dan merender gambar secara *asynchronous* melalui tautan *URL* yang bermuara langsung di *Supabase Storage*.

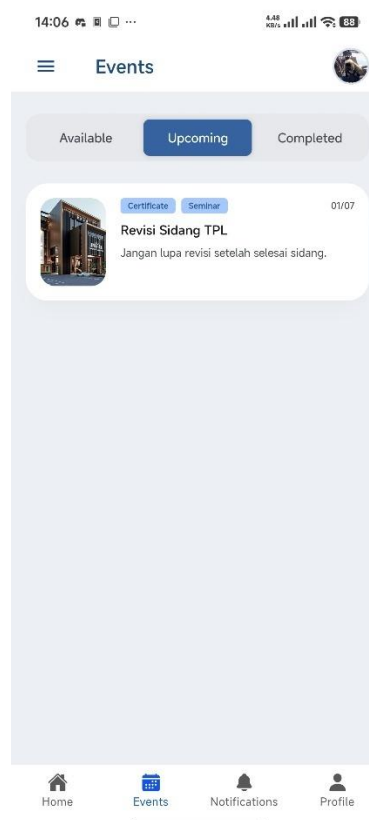
3) Halaman *Event*



Gambar 4.56 Halaman *Events* sub-tab *Available*

Gambar 4.56 menyorot antarmuka *tab Available*, yang merupakan realisasi presisi dari desain *tab* bar navigasi acara di *Figma* (Gambar 4.35). Antarmuka ini dikhususkan untuk menampilkan katalog acara yang masih membuka pendaftaran. Dari sisi pengelolaan *database*, daftar *event* pada halaman ini dihasilkan dari kueri pemilahan (*filtering*) ke tabel *events* di mana batas waktu acara (*end_date*) belum terlewati dan kuota belum penuh.

Selain itu, sistem mengeksekusi pengecekan silang (*Left Join*) ke tabel *event_participants* untuk memastikan bahwa *User ID* yang sedang aktif tidak dimunculkan *event* yang sudah ia daftar sebelumnya. Label kategori (contoh: *Project*) yang tertera ditarik dari tabel referensi *kategori_event* menggunakan tabel *mapping*.



Gambar 4.57 Halaman *Event sub-tab Upcoming*

Gambar 4.57 menampilkan visualisasi *sub-tab Upcoming*, sebuah ruang khusus yang memuat antrean jadwal kegiatan pribadi milik anggota. Tidak terdapat deviasi desain struktural dari rancangan *Figma* (Gambar 4.36). Pada area *backend*, antarmuka ini murni mengandalkan relasi tabel *event_participants*. Aplikasi meminta *server* untuk mengembalikan rincian data dari tabel *events* hanya pada acara-acara di mana *User ID* terkait telah

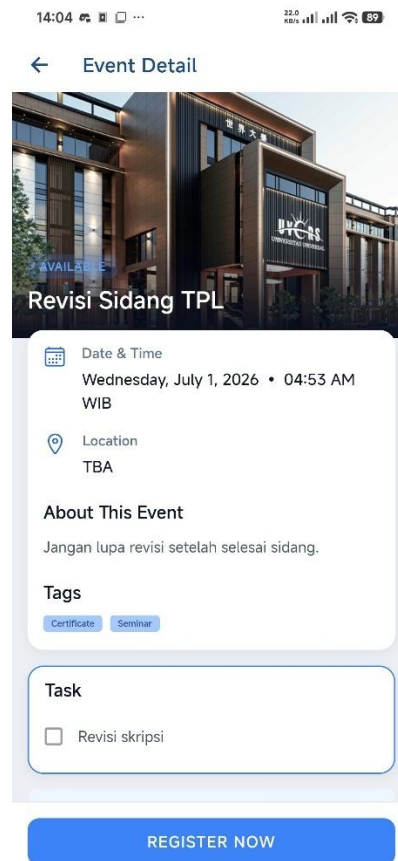
berstatus terdaftar di dalamnya, dengan kondisi tambahan bahwa waktu pelaksanaan acara (*start_date*) berada di masa depan relatif terhadap jam sistem.



Gambar 4.58 Halaman *Events sub-tab Completed*

Gambar 4.58 merupakan wujud nyata *sub-tab Completed* yang mendokumentasikan riwayat portofolio kegiatan. Terdapat sedikit modifikasi taktis dari *wireframe Figma* (Gambar 4.37); pada hasil coding ini ditambahkan teks tajuk "Attended" (Dihadiri) untuk menegaskan status riwayat pengguna. Pemrosesan *database* pada halaman ini menyeleksi arsip data dari tabel *events* yang tanggalnya telah terlampaui. Kueri tersebut kemudian mencocokkan *User ID* ke tabel *event_participants* guna memvalidasi nilai akhir pada kolom status (misalnya: *attended* atau *absent*),

sehingga aplikasi hanya merender kartu acara tempat anggota tersebut benar-benar hadir berpartisipasi.

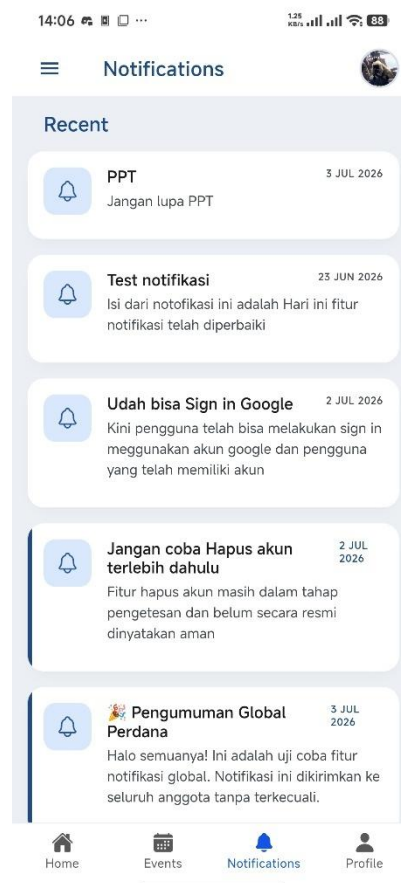


Gambar 4.59 Halaman *Event Detail*

Gambar 4.59 adalah wujud implementasi dari cetak biru "*Template Halaman Detail*" *Figma* (Gambar 4.38). Pengembang berhasil merealisasikan elemen *visual* secara utuh: mulai dari gambar *header* proporsi besar, teks *overlay* judul, rincian tanggal, hingga lokasi fisik kegiatan. Terdapat perbedaan dinamis pada layar bawah; jika di *Figma* dicontohkan kemunculan daftar "*Task*", pada tangkapan layar ini muncul tombol biru terang "*REGISTER NOW*". Hal ini menunjukkan bahwa antarmuka memiliki hubungan langsung dengan *database* untuk membaca

kondisi *real-time*. Sistem memeriksa ketersediaan kuota di tabel *events* dan melihat apakah *user_id* sudah ada di tabel *event_participants*. Jika acara masih tersedia dan belum didaftar, tombol pendaftaran akan ditampilkan. Detail lokasi dan label (*Tags*) ditarik menggunakan kueri berasal dari tabel referensi lokasi dan kategori_event.

4) Halaman Notifikasi



Gambar 4.60 Halaman Notifikasi

Gambar 4.60 menampilkan antarmuka daftar riwayat pemberitahuan (Notification List). Antarmuka ini dirancang linier dan sangat patuh terhadap *wireframe* desain berbasis daftar yang ada di *Figma* (Gambar 4.39).

Setiap kotak pesan memvisualisasikan ikon tipe pemberitahuan, teks judul yang ditebalkan, potongan singkat isi pesan (*snippet*), dan stempel waktu relatif (contoh: "3 JUL 2026"). Pada ranah pemrosesan data, halaman ini mengirimkan kueri *SELECT* spesifik ke tabel penerima_notifikasi dengan parameter filter *user_id* milik akun yang sedang *login*. Hasil kueri tersebut langsung dihubungkan (*Inner Join*) ke tabel notifikasi untuk menarik data struktur pesan utamanya. Indikator *visual* "belum dibaca" di antarmuka dikontrol murni oleh nilai boolean dari kolom *is_read*.

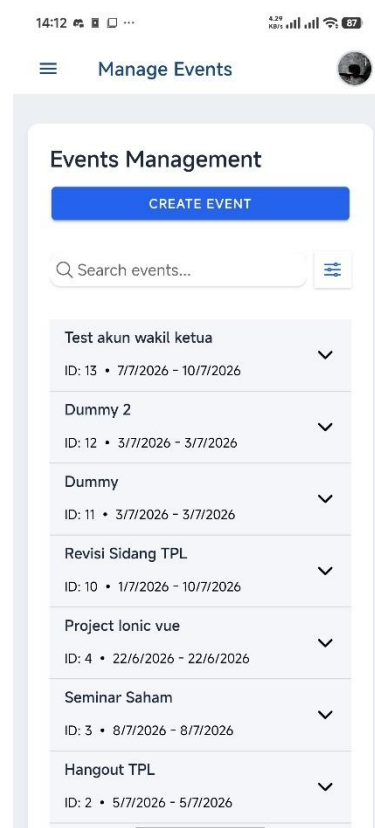


Gambar 4.61 Halaman Detail Notifikasi

Gambar 4.61 menyoroti Halaman Detail Notifikasi, sebuah antarmuka penampil teks utuh yang diinisiasi dan ditambahkan mandiri pada fase coding, guna menutupi keterbatasan *User Experience (UX)* pada desain

Figma awal apabila pengguna harus membaca teks pesan yang sangat panjang di kotak *list view* yang sempit. Hubungannya dengan basis data sangat kritikal: sesaat setelah pengguna menekan (*tap*) sebuah kartu notifikasi untuk masuk ke halaman detail ini, aplikasi *mobile* akan langsung menembakkan kueri mutasi *UPDATE* ke tabel *penerima_notifikasi*. Kueri ini akan menimpa dan mengubah nilai kolom *is_read* dari *False* menjadi *True*, yang memicu aplikasi untuk menghilangkan tanda notifikasi baru (titik merah atau ikon menyala) di menu navigasi pengguna secara permanen.

5) Halaman Kelola *Events*



Gambar 4.62 Halaman Kelola *Events*

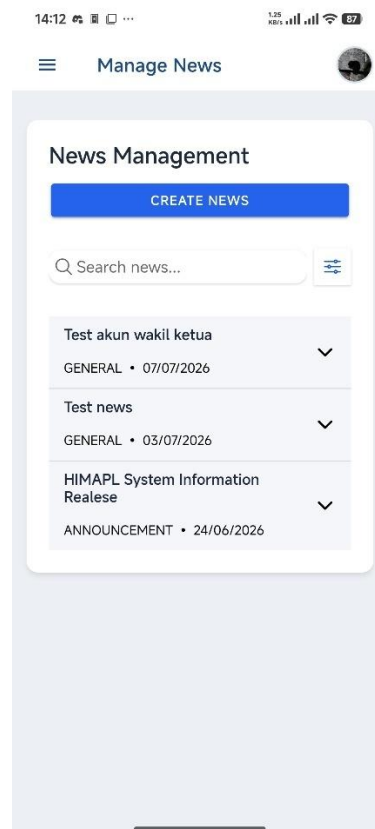
Gambar 4.62 menampilkan antarmuka manajemen data acara (Events Management) yang hak aksesnya dikhususkan bagi peran Ketua. Halaman ini adalah fitur administratif murni yang dibangun saat fase implementasi (tanpa *wireframe Figma* sebelumnya), menggunakan komponen daftar gaya *accordion* dari pustaka *Ionic*. Untuk fungsionalitasnya, antarmuka ini secara terus-menerus mengeksekusi kueri *SELECT* ke tabel *events* di *Supabase* untuk melist seluruh rekam jejak acara. Melalui elemen ini, Ketua memiliki kontrol otoritas tunggal untuk memantau *ID* kegiatan secara dinamis, serta melakukan operasi penghapusan (*DELETE*) acara langsung dari basis data.

Gambar 4.63 Halaman *Create Event*

Gambar 4.63 menunjukkan formulir interaktif (*Create Event*) untuk menambah kegiatan baru ke dalam sistem organisasi. Antarmuka ini memuat seluruh kolom isian operasional secara mendetail, yang mewakili

struktur tabel *events* (seperti *Event Name*, *Start/End Date*, *Quota*, *Location*, dsb). Setelah tombol *Submit* ditekan, formulir akan mengemas seluruh *input* dan menembakkan kueri *INSERT* ke tabel *events* di *Supabase*, sekaligus memicu proses *upload* jika terdapat penyisipan gambar poster. Kolom *input* jadwal juga didesain untuk memastikan tipe inputan sesuai dengan *format* *Timestamp* yang diwajibkan oleh *PostgreSQL*.

6) Halaman Kelola *News*



Gambar 4.64 Halaman Kelola *News*

Gambar 4.64 memperlihatkan antarmuka pengelolaan daftar publikasi berita (*News Management*). Sama halnya dengan *Kelola Event*, halaman ini menggunakan pendekatan daftar gaya lipat (*accordion list view*) dari

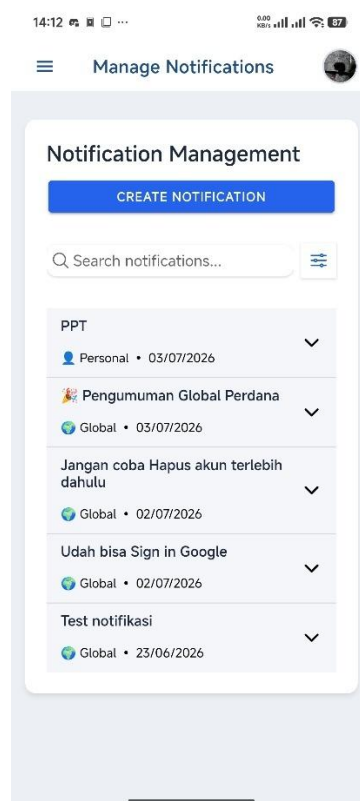
framework Ionic. Antarmuka menyeleksi dan merender baris data utama dari tabel *news* pada *database* (menampilkan tajuk berita dan jadwal rilis). Halaman ini disediakan sebagai panel kendali (*control panel*) bagi Ketua maupun Ketua Divisi untuk memantau publikasi organisasinya secara terpusat, dengan integrasi fitur mutasi untuk menyunting atau mencabut (menghapus) berita dari sistem.

Gambar 4.65 Halaman *Create News*

Gambar 4.65 menyajikan formulir pembuatan pengumuman baru (*Create News*). Antarmuka inputan ini dirancang lebih minimalis, hanya meminta *News Title*, *Category*, sumber gambar, dan teks panjang *Content*. Dari sisi basis data, aksi penyimpanan (*SAVE NEWS*) akan menginisiasi transaksi terpisah: berkas gambar akan dikompresi dan dikirim ke *Storage*

Bucket, sementara teks isi berita disuntikkan ke dalam tabel *news*. Pada detik yang sama, mesin *database Supabase* akan mengevaluasi aturan *RLS* tabel *news* untuk memverifikasi apakah akun pengguna memiliki hak administratif untuk membuat publikasi.

7) Halaman Kelola Notifikasi



Gambar 4.66 Halaman Kelola Notifikasi

Gambar 4.66 memvisualisasikan layar *Notification Management* untuk memantau rekam jejak siaran pesan manual yang pernah dibuat pengurus. Antarmuka ini menampilkan data menggunakan sistem *fetch* ke tabel induk notifikasi di *Supabase*, bukan tabel relasinya (*penerima_notifikasi*). Hasilnya, pengurus dapat melacak daftar template pesan sentral yang telah mereka siarkan (lengkap dengan klasifikasi pesan *Global* atau *personal*),

serta memiliki opsi untuk merapikan tabel notifikasi tersebut dengan menghapus data pemberitahuan usang.

16:51 55% 4G LTE

Create Notification CANCEL

Title

Assign To
Global (Everyone)

Message

SAVE NOTIFICATION

Gambar 4.67 Halaman *Create Notification*

Gambar 4.67 menampilkan formulir khusus untuk mengirim pesan notifikasi secara langsung ke perangkat anggota (*Create Notification*). Kolomnya sangat spesifik: *Title*, *audiens* target (*Assign To* - seperti *Global/Everyone* atau *ID* spesifik), dan teks *Message*. Hubungan transaksionalnya dengan *database* sangat vital pada aspek performa; saat tombol "*SAVE NOTIFICATION*" disetujui, aplikasi pertama-tama melempar data pesan untuk dicatat ke tabel induk notifikasi. Segera setelah template tersebut direkam dengan sukses, sistem menginisiasi kueri penyisipan berkelompok (*batch insert*) ke tabel penerima_notifikasi secara massal sesuai jumlah *audiens* target agar pesan dapat didistribusikan efisien.

8) Halaman Laporan



ID Event	Event Name	Participants	Start Date
3	Seminar Saham	1	08/07/2026
13	Test akun wakil ketua	0	07/07/2026
2	Hangout TPL	1	05/07/2026
12	Dummy 2	0	03/07/2026
11	Dummy	0	03/07/2026
1	Sidang TPL	1	02/07/2026

PRINT REPORT (PDF)

Gambar 4.68 Halaman Laporan

Gambar 4.68 memperlihatkan wujud luaran akhir dari pengembangan perangkat lunak ini, yaitu halaman kompilasi rekapitulasi data (*Annual Reports*). Antarmuka ini tidak menayangkan baris data mentah, melainkan hasil olah kueri komputasional dari *database*. Secara teknis, sistem menjalankan fungsi analitik agregasi *COUNT* pada tabel *event_participants* (berdasarkan filter status bernilai hadir) untuk menghitung partisipan, kemudian hasil kalkulasinya digabungkan (*Joined*) dengan nama dan tanggal acara dari tabel *events*. Halaman ini juga disempurnakan dengan kehadiran tombol "*PRINT REPORT (PDF)*" di bagian dasar, yang bertugas mem- parsing hasil agregasi *SQL* tersebut menjadi dokumen *format PDF* portabel sebagai arsip nyata pertanggungjawaban acara.

4.5.2.3 Review

Fase *Sprint Review* pada iterasi kedua dilaksanakan untuk mendemonstrasikan secara komprehensif seluruh modul operasional organisasi yang telah diselesaikan. Demonstrasi perangkat lunak ini kembali melibatkan representasi pengguna guna memvalidasi fungsionalitas manajemen *Event*, pendelegasian tugas, publikasi *News*, serta sistem dokumentasi dan notifikasi otomatis agar sesuai dengan kriteria penerimaan (Acceptance Criteria).

Fokus pengujian pada *Sprint 2* dititikberatkan pada kapabilitas aplikasi dalam menangani transaksi data relasional kompleks yang melibatkan komputasi penyimpanan aset *visual* (*Supabase Storage*) dan perlindungan privasi data antar-pengguna (*Row Level Security*). Rincian skenario validasi beserta status penerimaannya didokumentasikan pada tabel berikut.

Tabel 4.10 Tabel Review *Sprint 2*

ID Backlog	Fitur	Skenario Validasi (Acceptance Criteria)	Hasil Pengujian	Status
SB-04	Manajemen <i>Event</i> dan Partisipasi	1. Ketua dapat membuat jadwal <i>Event</i> dan mengunggah poster kegiatan.	Pembaruan data <i>Event</i> tersimpan sempurna dan sinkron dengan <i>state UI</i> aplikasi.	Diterima

		2. Sistem berhasil menolak modifikasi <i>Event</i> dari pengguna non-otoritas. 3. Status <i>Event</i> bergeser secara dinamis sesuai perbandingan tanggal sistem.		
SB-05	Pendelegasian Tugas Organisasi	1. Ketua/Ketua Divisi dapat mendelegasikan tugas kepada anggota spesifik. 2. Anggota hanya dapat melihat daftar tugas yang didelegasikan ke <i>ID</i> akun mereka sendiri.	Filter data berbasis <i>User ID</i> berjalan akurat tanpa kebocoran data antar-anggota.	Diterima

SB-06	Publikasi Berita (<i>News</i>)	1. Pengurus dapat memublikasikan artikel berita beserta gambar sampul. 2. Aplikasi memproses kompresi gambar sebelum dikirim ke <i>Supabase Storage</i>.	Publikasi berita berhasil ditampilkan pada halaman <i>feed</i> seluruh pengguna.	Diterima
SB-07	Dokumentasi & Notifikasi	1. Semua pengguna terautentikasi dapat mengunggah foto dokumentasi kegiatan. 2. <i>Database Trigger PostgreSQL</i> berhasil menghasilkan	Siklus unggah dokumentasi dan pemunculan notifikasi <i>real-time</i> berjalan stabil.	Diterima

		notifikasi otomatis. 3. Pengguna dapat mengubah status notifikasi menjadi telah dibaca (<i>is_read = true</i>).		
--	--	---	--	--

Berdasarkan hasil pengujian fungsional secara menyeluruh, seluruh item pekerjaan teknis dalam iterasi ini (SB-04 hingga SB-07) dinyatakan Diterima (Accepted). Kesuksesan integrasi komponen pada *Sprint 2* ini sekaligus menandai bahwa aplikasi Sistem Informasi HIMAPL telah memenuhi standar kelayakan operasional secara utuh dan siap untuk beralih ke tahapan penyebaran (deployment).

4.5.2.4 Retrospective

Tahapan *Sprint Retrospective* ini menjadi penutup *Sprint 2* sekaligus menandai akhir dari fase pengembangan Sistem Informasi HIMAPL. Evaluasi kali ini difokuskan pada penanganan fungsionalitas dinamis dan relasi data. Hasil refleksi kemudian dirangkum ke dalam tiga matriks utama, dengan rencana tindak lanjut (Action Plan) yang diarahkan pada persiapan rilis (deployment) dan pemeliharaan sistem.

Tabel 4.11 Tabel *Retrospective Sprint 2*

Kategori Evaluasi	Hasil Refleksi Pengembang
Yang Berjalan Baik (<i>What Went Well</i>)	1. Pemanfaatan <i>Database Trigger</i> pada <i>PostgreSQL</i> (<i>Supabase</i>) sangat efektif dalam mengotomatisasi pembuatan notifikasi tanpa membebani kode pada sisi antarmuka (<i>frontend</i>). 2. Alur kerja yang telah dievaluasi pada <i>Sprint 1</i> berhasil mempercepat proses perancangan <i>Row Level Security (RLS)</i> untuk modul tugas dan <i>Event</i>.
Yang Perlu Ditingkatkan (<i>Needs Improvement</i>)	1. Sinkronisasi perubahan data (<i>state management</i>) secara <i>real-time</i> di sisi aplikasi <i>mobile</i> terkadang mengalami sedikit jeda (<i>latensi</i>) setelah pengguna melakukan pembaruan data secara beruntun. 2. Pengujian terhadap perangkat dengan ukuran layar (<i>viewport</i>) yang sangat kecil masih menemukan beberapa komponen <i>UI</i> (seperti teks panjang pada <i>News</i>) yang terpotong secara <i>visual</i>.
Rencana Tindak Lanjut (<i>Action Plan</i>)	1. Fase Pemeliharaan: Menerapkan optimalisasi <i>cache</i> lokal pada <i>Ionic</i> untuk mengurangi beban

	kueri berulang ke <i>server</i> dan meminimalisir latensi <i>visual</i>. 2. Fase Implementasi/Rilis: Menyusun panduan penggunaan aplikasi (<i>User Manual</i>) yang komprehensif bagi pengurus HIMAPL sebelum sistem didistribusikan secara resmi.
--	--

Dengan diselesaikannya evaluasi *retrospective* ini, seluruh rangkaian siklus *Agile Scrum* (mulai dari perencanaan hingga rilis inkremen fungsional) telah dieksekusi secara tuntas. Aplikasi Sistem Informasi HIMAPL dinyatakan siap untuk memasuki tahap pengujian akhir (*User Acceptance Testing*) dan penyebaran (*deployment*) ke lingkungan produksi.

4.6 Completed Product

Hasil akhir dari pengembangan aplikasi Sistem Informasi HIMAPL berbasis *mobile* divisualisasikan secara utuh sebagai bentuk akumulasi pencapaian dari seluruh tahapan iterasi *Sprint* yang telah dilaksanakan. Produk akhir ini mencerminkan keberhasilan dua siklus pengembangan utama. Pada *Sprint* 1, implementasi berhasil mencapai pembentukan fondasi arsitektur antarmuka menggunakan *Framework Ionic* dan *backend Supabase*, yang mencakup fungsionalitas autentikasi, manajemen profil, dan pengaturan hierarki divisi. Selanjutnya, pada *Sprint* 2, sistem berhasil disempurnakan

dengan rampungnya integrasi modul operasional, yang meliputi manajemen *Event*, pendelegasian tugas, publikasi berita (*News*), serta sistem dokumentasi dan notifikasi.

Aplikasi telah di-*build* ke bentuk *APK* dalam versi *release*, sehingga jika dilakukan instalasi dalam perangkat *android* akan lebih gampang karena *Google Play Protect* bersikap lebih lunak terhadap aplikasi versi *release* yang menyebabkan proses instalasi tidak digagalkan oleh sistem keamanan



Gambar 4.69 Ikon aplikasi di *Android*

Tampilan ikon aplikasi dari *APK* yang telah diinstalasi pada perangkat *Android*

4.7 Pengujian Pada Sistem

Pengujian sistem merupakan tahapan akhir dari implementasi untuk memastikan bahwa seluruh modul aplikasi Sistem Informasi HIMAPL berjalan sesuai dengan rancangan yang diharapkan dan bebas dari kecacatan logika (*bug*). Metode pengujian yang digunakan pada penelitian ini adalah pengujian *Black-Box*

dengan teknik *Equivalence Partitioning*, yang berfokus pada fungsionalitas *input* dan output antarmuka pengguna tanpa perlu meninjau struktur kode internal.

4.7.1 Skenario dan Hasil Pengujian *Black-Box*

Pengujian dilakukan secara komprehensif terhadap modul-modul utama yang telah dikembangkan pada fase *Sprint*, meliputi modul Autentikasi, Manajemen *Event*, Delegasi Tugas, dan Dokumentasi. Rincian skenario pengujian beserta hasil observasinya diuraikan pada tabel berikut.

Tabel 4.12 Tabel Hasil Pengujian *Black-Box*

No	Fungsionalitas yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengamatan	Status
1	Autentikasi (<i>Login</i>)	Menginput <i>email</i> yang belum terdaftar atau <i>password</i> yang salah.	Sistem menolak akses dan menampilkan pesan <i>error</i> peringatan kredensial tidak <i>valid</i> .	Sistem memunculkan notifikasi <i>error</i> dan tetap berada di halaman <i>login</i> .	Valid
2	Manajemen <i>Event</i>	Mengisi <i>form</i> pembuatan <i>Event</i> baru dengan mengosongkan	Sistem mencegah penyimpanan data dan meminta pengguna	Tombol simpan dinonaktifkan atau memunculkan	Valid

		kolom tanggal kegiatan.	melengkapi kolom tanggal.	peringatan validasi <i>form</i> .	
3	Delegasi Tugas	<i>Login</i> sebagai anggota biasa, lalu mencoba mengakses halaman <i>endpoint</i> pembuatan tugas.	Akses ditolak oleh sistem dan <i>Row Level Security (RLS)</i> pada <i>database</i> memblokir kueri.	Antarmuka mengembalikan pengguna ke halaman <i>dashboard</i> dengan pesan "Akses Ditolak".	Valid
4	Dokumentasi & Notifikasi	Mengunggah foto berukuran di atas 5MB pada menu dokumentasi.	Fitur kompresi sisi <i>client</i> bekerja sebelum gambar dikirim ke <i>Supabase Storage</i> .	Proses unggah berhasil, ukuran <i>file</i> yang tersimpan di <i>Storage</i> berkurang drastis.	Valid

4.7.2 Hasil User Acceptance Testing

Proses *UAT* melibatkan partisipan yang terdiri dari representasi pengurus dan anggota HIMAPL. Pengujian dilakukan dengan memberikan akses pengoperasian aplikasi secara langsung kepada partisipan, yang kemudian dilanjutkan dengan pengisian kuesioner evaluasi berdasarkan perhitungan Skala

Likert. Aspek penilaian difokuskan pada tiga indikator utama: Desain Antarmuka (*User Interface*), Kemudahan Penggunaan (*User Experience*), dan Manfaat Fungsionalitas.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan keseluruhan tahapan penelitian, perancangan, pengembangan, hingga pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Keberhasilan Pengembangan Sistem: Telah berhasil dirancang dan dibangun sebuah aplikasi Sistem Informasi HIMAPL berbasis *mobile* menggunakan *Framework Ionic* (dengan basis *Vue.js*) yang terintegrasi secara komprehensif dengan layanan komputasi awan *Supabase*. Aplikasi ini telah merangkum seluruh kebutuhan fungsional organisasi, mencakup manajemen *Event*, pendelegasian tugas, publikasi informasi (*News*), sistem dokumentasi, serta notifikasi.
2. Efektivitas Metodologi *Agile Scrum*: Penerapan metode *Agile* dengan kerangka kerja *Scrum* terbukti sangat efektif dalam mengelola pengembangan aplikasi ini. Pembagian kerja ke dalam dua iterasi utama (*Sprint 1* untuk fondasi arsitektur dan *Sprint 2* untuk fungsionalitas operasional) memungkinkan sistem dikembangkan secara terstruktur, adaptif terhadap perubahan kebutuhan, dan dapat dievaluasi secara berkala.
3. Penerapan Keamanan dan Otomatisasi Basis Data: Penggunaan arsitektur *Supabase (PostgreSQL)* terbukti meningkatkan reliabilitas sistem. Implementasi *Row Level Security (RLS)* berhasil mengamankan privasi dan otoritas data antar-pengguna (seperti filter hak akses penugasan), sementara

pemanfaatan *Database Triggers* mampu mengotomatisasi pembuatan notifikasi secara *real-time* tanpa membebani komputasi pada sisi klien (frontend).

4. Hasil Validasi dan Kelayakan Produk: Berdasarkan tahap pengujian fungsionalitas dengan metode *Black-Box Testing*, aplikasi terbukti mampu menangani seluruh validasi *input* dan perlindungan akses data dengan persentase keberhasilan 100%.

5.2 Saran

Mempertimbangkan batasan waktu dan ruang lingkup penelitian, penulis merumuskan beberapa saran bagi pengembangan aplikasi di masa mendatang, yakni:

1. Perluasan Kapasitas Penyimpanan (*Storage*) dan Pengarsipan: Mengingat aplikasi ini secara aktif menampung berkas *visual* (poster *Event*, gambar *News*, dan dokumentasi kegiatan), kapasitas penyimpanan gratis (*Free Tier*) dari *Supabase* berpotensi mencapai batas maksimal seiring berjalannya waktu. Disarankan untuk merencanakan peningkatan layanan (*upgrade plan*) pada *Supabase*, atau mengintegrasikan layanan penyimpanan awan pihak ketiga yang lebih masif (seperti *AWS S3* atau *Google Cloud Storage*). Selain itu, sistem pengarsipan (*auto-archiving*) atau penghapusan berkas usang secara berkala perlu diimplementasikan agar penggunaan ruang penyimpanan lebih efisien.

2. Penambahan Fitur Manajemen Keuangan (Uang Kas): Aplikasi saat ini sangat kuat pada aspek operasional kegiatan dan pendelegasian tugas. Untuk pengembangan selanjutnya, sangat disarankan menambahkan modul Manajemen Keuangan. Fitur ini dapat mencatat pemasukan uang kas anggota, pengeluaran acara, serta menampilkan grafik transparansi dana HIMAPL yang dapat diakses oleh seluruh anggota untuk meningkatkan akuntabilitas organisasi.
3. Implementasi Native Push *Notification*: Saat ini, notifikasi yang berjalan masih bersifat *in-app Notification* (pengguna harus membuka aplikasi untuk melihat pemberitahuan terbaru). Ke depannya, sistem dapat diintegrasikan dengan *Firebase Cloud Messaging (FCM)* atau *APNs (Apple Push Notification service)* agar aplikasi dapat mengirimkan push *Notification* langsung ke layar utama (*lock screen*) perangkat *mobile* anggota saat ada tugas atau berita penting yang baru dipublikasikan.
4. Fitur Rekapitulasi Laporan Pertanggungjawaban (LPJ) Otomatis: Salah satu kendala terbesar pengurus himpunan di akhir masa jabatan adalah menyusun Laporan Pertanggungjawaban (LPJ). Sistem ini memiliki potensi besar untuk dikembangkan dengan fitur Generate Report (Ekspor ke *PDF* atau *Excel*), yang dapat merekap seluruh data kegiatan, penugasan panitia, dan dokumentasi foto dalam satu rentang waktu secara otomatis untuk diserahkan ke pihak kampus UVERS.
5. Modul Presensi Kegiatan Berbasis *QR Code*: Untuk memaksimalkan *Framework Ionic* pada perangkat *mobile*, dapat ditambahkan modul

pemindai *QR Code* (kamera ponsel) untuk mendata presensi (*attendance*) peserta saat *Event* HIMAPL berlangsung. Hal ini akan menggantikan proses absensi manual menjadi sepenuhnya digital dan langsung terekam ke dalam *database Supabase*.

DAFTAR PUSTAKA

- Audy Rivand, I., & Suwandi. (2023). Journal of Culture Accounting and Auditing Dampak Efektivitas Sistem Informasi Akuntansi: Pengaruh Teknologi Informasi Dan Kualitas Sumber Daya Manusia Terhadap Kinerja Perusahaan. *Http://Journal.Umg.Ac.Id/Index.Php/Tiaa*, 2, 119–135. <http://journal.umg.ac.id/index.php/tiaa>
- Azis, Y. M., Susanti, S., & Sarosa, M. (2024). Aplikasi Keuangan Koperasi Simpan Pinjam “Permata Ngijo” Berbasis Teknologi Informasi. *International Journal of Community Service Learning*, 7(3), 370–376. <https://doi.org/10.23887/ijcsl.v7i3.62743>
- Binus University Bekasi. (n.d.). *Bagaimana Melakukan Pengujian dengan User Acceptance Testing (UAT) - BINUS @Bekasi - Kampus Beken Asyik | Business Service and Technology*. Retrieved July 9, 2026, from <https://binus.ac.id/bekasi/2024/12/bagaimana-melakukan-pengujian-dengan-user-acceptance-testing-uat/>
- Computer Soceity, I. (2015). *ISO/IEC/IEEE 29119-4-2015, Software and systems engineering--Software testing--Part 4: Test techniques*.
- Delone, W. H., & Mclean, E. R. (2003). The DeLone and McLean Model of Information Systems Success: A Ten-Year Update. In *Communications of the ACM. DATABASE. Han>aril Business Review* (Vol. 19).

- Gutiérrez, J. D., Jiménez, A. R., Seco, F., Álvarez, F. J., Aguilera, T., Torres-Sospedra, J., & Melchor, F. (2022). GetSensorData: An extensible Android-based application for multi-sensor data registration. *SoftwareX*, 19. <https://doi.org/10.1016/j.softx.2022.101186>
- Hasan, A. N., Liwan, N. A., Sudharma, F. Z., Pontoh, G. T., Indrijawati, A., Penulis, N., Annisa, :, & Hasan, N. (2025). Adopsi Cloud Enterprise Resource Planning dengan Pendekatan Technology, Organization, and Environmental pada UMKM: Tinjauan Literatur. *EL MUHASABA: Jurnal Akuntansi (e-Journal)*, 16(1).
- Ionicframework. (n.d.). *Ionic Framework - The Cross-Platform App Development Leader*. Retrieved July 9, 2026, from <https://ionicframework.com/>
- ISO 9241-11. (2018). *Ergonomics of human-system interaction-Part 11: Usability: Definitions and concepts COPYRIGHT PROTECTED DOCUMENT*. <https://standards.iteh.ai/catalog/standards/sist/d38dc274-d8d4-4fb9-8206->
- ISO 9241-210. (2019). *Ergonomics of human-system interaction-Human-centred design for interactive systems COPYRIGHT PROTECTED DOCUMENT*. www.iso.org

- Joann, Jd. (2015). An Evolution of Android Operating System and Its Version. *International Journal of Engineering and Applied Sciences*, 2(2), 30–33.
www.ijeas.org
- Johannsen, F., Knipp, M., Loy, T., Mirbabaie, M., Möllmann, N. R. J., Voshaar, J., & Zimmermann, J. (2023). What impacts learning effectiveness of a mobile learning app focused on first-year students? *Information Systems and E-Business Management*, 21(3), 629–673.
<https://doi.org/10.1007/s10257-023-00644-0>
- Laudon, K. C. ., & Laudon, J. P. . (2022). *Management information systems : managing the digital firm*. Pearson.
- Microsoft. (n.d.). *Apa itu PostgreSQL? | Microsoft Azure*. Retrieved July 9, 2026, from <https://azure.microsoft.com/id-id/resources/cloud-computing-dictionary/what-is-postgresql>
- Mika, J., Thoe, N., O', J., & Lecturer, S. (2012). *A Mori approach To management: Contrasting traditional and modern Mori management practices in Aotearoa New Zealand*.
- Mumtahana, H. A. (2022). Optimization of Transaction Database Design with MySQL and MongoDB. *Sinkron*, 7(3), 883–890.
<https://doi.org/10.33395/sinkron.v7i3.11528>
- Myers, G. J., Badgett, T., & Sandler, C. (2011). 114-the-art-of-software-testing-3-edition. *John Wiley & Sons, Inc*.

Naiylatan Syirfa, A., Yaa Dzune, D., Octiara Salsabila, C., Hindami Prasetya, R., Zhydan, Z., & Wildani, J. (2023). Analisis Desain Sistem Pendataan Fasilitas Kampus di Program Studi Universitas. *JDBIM (Journal of Digital Business and Innovation Management)*, 2(2), 93–110. <https://doi.org/10.1234/jdbim.v2i2.58087>

Ndukuyu, A. C., Wabwoba, F., & Ikoha, A. (2016). Adoption of *Mobile Computing for Ubiquitous Data Management* within Sugar Companies in Kenya. In *International Journal of Recent Research in Mathematics Computer Science and Information Technology* (Vol. 3). www.paperpublications.org

Popkin Software and Systems. (1998). *Modeling Systems with UML*.

Pšenák, P., & Tibenský, M. (2020). The usage of *Vue JS framework* for web application creation. *Mesterséges Intelligencia*, 2(2), 61–72. <https://doi.org/10.35406/mi.2020.2.61>

Purwandi, N. (2024). Scrum Integration in Cloud-based Human Capital Management (HCM) Project Development. *Journal of Information System and Technology*, 05(02), 45–51. <https://doi.org/10.37253/joint.v5i2.9577>

pwg, & istqborg. (2024). *Certified Tester Foundation Level Syllabus v4.0.1* International Software Testing Qualifications Board.

Quist, C. (2015). *Presented To the Interdisciplinary Studies Program: Benefits of Blending Agile and Waterfall Project Planning Methodologies*.

Rimayanda, & Maknuni, J. (2022). Pelaksanaan Sistem Informasi Manajemen Kepegawaian Berbasis IT di Kantor Kementrian Agama Kabupaten Aceh Timur. *Jurnal Ekonomi Bisnis, Manajemen Dan Akuntansi*, 1(2), 54–58.
<https://doi.org/10.58477/ebima.v1i2.54>

Romi, M. V. (2024). Model Good Governance Berbasis E- Government dan Motivasi Pelayanan Publik Pada Aparatur Sipil Negara Pemerintah Kota Cimahi. *J-MAS (Jurnal Manajemen Dan Sains)*, 9(1), 61.
<https://doi.org/10.33087/jmas.v9i1.1431>

S, A. K., Bharadwaj, R. R., V, T. M., & Patil, P. (2025). Advancements in Mobile Computing: A Comprehensive Review. In *International Journal of Research Publication and Reviews Journal homepage: www.ijrpr.com* (Vol. 6). www.ijrpr.com

Sirojuddin, A., Amirullah, K., Rofiq, M. H., & Kartiko, A. (2022). ZAHRA: Research And Thought Elmentary School Of Islam Journal PERANAN SISTEM INFORMASI MANAJEMEN DALAM PENGAMBILAN KEPUTUSAN DI MADRASAH IBTIDAIYAH DARUSSALAM PACET MOJOKERTO. 3(1), 19–32.

Sopiyan, A., & Darajatun, R. A. (2024). ANALISIS SITUASIONAL DAN PERANCANGAN SISTEM INFORMASI KEUANGAN PADA PT ZMI. *Jurnal Informatika Dan Teknik Elektro Terapan*, 12(1).
<https://doi.org/10.23960/jitet.v12i1.3753>

Supabase. (n.d.). *Architecture* | *Supabase Docs*. Retrieved July 9, 2026, from <https://supabase.com/docs/guides/getting-started/architecture>

Tulvina, K., Syamsiah, N. O., & Dharmawan, W. S. (2022). Penggunaan Extreme Programming Untuk Menunjang Perubahan Kebutuhan Dalam Proses Pembangunan Sistem Informasi Produksi. In *Artikel Ilmiah Sistem Informasi Akuntansi (AKASIA)* (Vol. 2). <https://jurnal.bsi.ac.id/index.php/akasia>

Urbach, N., & Müller, B. (2012). The Updated DeLone and McLean Model of Information Systems Success. In Y. K. Dwivedi, M. R. Wade, & S. L. Schneberger (Eds.), *Information Systems Theory: Explaining and Predicting Our Digital Society, Vol. 1* (pp. 1–18). Springer New York. https://doi.org/10.1007/978-1-4419-6108-2_1

Wirfs-Brock, A., & Eich, B. (2020). JavaScript: The first 20 years. *Proceedings of the ACM on Programming Languages*, 4(HOPL). <https://doi.org/10.1145/3386327>

Wulandari, H., & Raharjo, T. (2023). Systematic Literature and Expert Review of Agile Methodology Usage in Business Intelligence Projects. *Journal of Information Systems Engineering and Business Intelligence*, 9(2), 214–227. <https://doi.org/10.20473/jisebi.9.2.214-227>

LAMPIRAN 01. Kartu Bimbingan Tugas Akhir

KONSULTASI TUGAS AKHIR

Nama Mahasiswa : Alex ferguson

Nomor Induk Mahasiswa : 2022133011

Dosen Pembimbing I/II : Ilwan Syafrinal, S.Kom, M.Kom.

Judul Tugas Akhir : PERANCANGAN APLIKASI SISTEM
INFORMASI HIMAPL BERBASIS *MOBILE* PADA
FAKULTAS KOMPUTER UNIVERSITAS
UNIVERSAL

Tanggal	Komentar	Tanda Tangan
15 Januari 2026		
9 April 2026		
16 April 2026		
23 April 2026		
7 Mei 2026		
22 Mei 2026		
17 Juni 2026		
22 Juni 2026		

Menyetujui *UI*,

Dosen Pembimbing

Mengetahui *UI*,

Koordinator Program Studi

Ilwan Syafrinal, S.Kom, M.Kom.

0427019401

Kaharuddin, S.Kom., M.Kom.

1009059501

LAMPIRAN 02. Daftar Riwayat Hidup

DAFTAR RIWAYAT HIDUP

a. Data Personal

NIM : 2022133011
Nama : Alex Ferguson
Tempat / Tgl. Lahir : Selatpanjang, 13 Oktober 2004
Jenis Kelamin : Laki-laki
Agama : Buddha
Jenjang : Strata 1 (S1)
Program Studi : Teknik Perangkat Lunak
Alamat Rumah : Perumahan Nusajaya Blok A9 No.6, RT 02, RW 05,
Sungai Panas, Kota Batam
Telp : 082287641064
Email : alexferguson5568@gmail.com
Pekerjaan : Mahasiswa

b. Pendidikan

Jenjang	Nama Lembaga	Jurusan	Tahun Lulus
SD	SD Kristen Kalam Kudus Selatpanjang		2016
SMP	SMP Yehonala		2019
SMK	SMK Maitreyawira	AXIOO (Multimedia)	2022

c. Organisasi

Tahun	Organisasi	Jabatan
2023-2024	Himpunan Mahasiswa Teknik Perangkat Lunak	Anggota

2024-2025	Himpunan Mahasiswa Teknik Perangkat Lunak	Ketua Divisi Minat dan Bakat
-----------	--	---------------------------------

Demikianlah daftar riwayat hidup ini dibuat dengan sebenarnya.

Batam, 26 juni 2026

Alex Ferguson

2022133011

LAMPIRAN

<https://github.com/Alex5568/Sistem-Informasi-HIMAPL>

<https://drive.google.com/drive/folders/1hf9yENo5Nmcur1DfAof8AI7Il93vm-sW?usp=sharing>